

DOCUMENT DE RÉVISION — SOUTENANCE

SmartAutoTrack

Les technologies de l'application, expliquées à partir du vrai code : PHP, JavaScript et MySQL en priorité, puis HTML/CSS, outils et services externes

PHP 8.2 · MySQL 8 · JavaScript / jQuery · HTML / CSS · Apache (XAMPP) · Composer · PHPUnit · Git
Version du dépôt : 7cdab83 · 9 octobre 2026

Sommaire

1. Vue d'ensemble : la pile technique
2. PHP : le langage serveur
3. JavaScript : l'interactivité dans le navigateur
4. MySQL : la base de données
5. HTML et CSS : la structure et l'apparence
6. Serveur et outils : XAMPP, Apache, Composer, PHPUnit, Git
7. Services externes et API
8. Sécurité transversale et questions finales

1. Vue d'ensemble : la pile technique

Avant d'entrer dans le détail de chaque langage, il faut savoir présenter **en deux minutes** l'architecture de SmartAutoTrack : quelles technologies, à quoi elles servent, comment une requête traverse l'application, et pourquoi ces choix plutôt que d'autres. C'est presque toujours la première question du jury : « Présentez-nous votre architecture technique. »

SmartAutoTrack est une application web **classique en trois tiers** : un navigateur (HTML, CSS, JavaScript avec jQuery), un serveur web Apache qui exécute du PHP, et une base de données MySQL. Elle tourne en local sous **XAMPP** (le paquet qui regroupe Apache, MySQL/MariaDB et PHP sous Windows) et peut être déployée telle quelle sur un hébergement mutualisé.

1.1 Tableau de la pile technique

Les versions ci-dessous ont été vérifiées sur la machine de développement et dans le projet :

C:\xampp\php\php.exe -v pour PHP, mysql --version du serveur MySQL 8.0 installé comme service Windows (MySQL80), le fichier composer.lock et les balises <script> / <link> de includes/header.php.

Couche	Technologie (version)	Rôle dans l'application	Fichiers principaux
Présentation	HTML5 généré par PHP	Structure des pages des quatre espaces (client, garage, technicien, admin)	includes/header.php , includes/footer.php , client/*.php , garage/*.php ...
Présentation	CSS3 (feuilles maison, variables CSS, thèmes clair/sombre)	Charte graphique commune, mise en page responsive, thèmes	assets/css/style.css , themes.css , client_v2.css , admin_v2.css ...
Présentation	JavaScript + jQuery 3.7.1 (CDN code.jquery.com)	Interactions, fenêtres modales, appels AJAX, assistant IA, changement de thème	assets/js/main.js , assistant.js , themes.js
Présentation	Chart.js (CDN jsDelivr, version non figée)	Graphiques des statistiques (inscriptions, anomalies)	admin/statistics.php , client/anomalies_recentes.php
Présentation	Font Awesome 6.4.0, polices Google Fonts « Inter » (et « Sora » / « Manrope » sur les pages v2)	Icônes et typographie	includes/header.php (variable \$extraFonts pour les polices en plus)
Serveur web	Apache (XAMPP) + .htaccess	Reçoit les requêtes HTTP, exécute PHP, ajoute les entêtes de sécurité et de cache, bloque les fichiers sensibles	.htaccess , config/.htaccess , uploads/.htaccess
Application	PHP 8.2 (8.2.12) sans framework	Logique métier, sessions, contrôle d'accès par rôle, CSRF, génération HTML et JSON	config/config.php , includes/*.php , ajax/*.php
Accès aux données	PDO (extension PHP), driver pdo_mysql	Connexion unique, requêtes préparées, exceptions	config/database.php

Couche	Technologie (version)	Rôle dans l'application	Fichiers principaux
Données	MySQL 8 (8.0.41, moteur InnoDB, utf8mb4)	Stockage : tables <code>utilisateur</code> , <code>client</code> , <code>vehicule</code> , <code>intervention</code> , <code>anomalie</code> , <code>paiement</code> , <code>abonnement</code> , <code>rappel_envoye</code> ...	<code>sql/SCHEMA_ACTUEL.md</code> (référence du schéma), <code>scripts/migrate_structure.php</code>
Services externes	cURL (extension PHP) + API REST JSON	Appels sortants vers CamPay, OpenRouter, Brevo, Google	<code>includes/http_client.php</code>
Paiement	API CamPay (Mobile Money MTN / Orange, XAF)	Paiement des réparations et des abonnements Premium, confirmation par webhook	<code>includes/campay.php</code> , <code>ajax/campay_collect.php</code> , <code>ajax/subscription_collect.php</code> , <code>webhooks/campay.php</code>
Intelligence artificielle	API OpenRouter (modèle principal + modèle de secours)	Assistant conversationnel client et admin	<code>includes/ai.php</code> , <code>ajax/assistant_chat.php</code>
Authentification	Google OAuth 2.0 (Authorization Code Flow, sans SDK)	Connexion « Se connecter avec Google »	<code>includes/google_oauth.php</code> , <code>auth/</code>
E-mails	API Brevo (HTTP)	E-mails transactionnels, dont les rappels d'échéance (paliers J7 et retard)	<code>includes/mailer.php</code>
Documents	PdfLite (générateur PDF maison, PHP pur)	Rapports d'intervention et reçus de paiement au format PDF	<code>includes/pdf_lite.php</code> , <code>includes/repair_report.php</code> , <code>includes/receipt.php</code>
Tâches planifiées	Scripts PHP en ligne de commande (CLI)	Rappels d'échéances (J30, J7, J0, retard), migrations de schéma	<code>scripts/send_reminders.php</code> , <code>scripts/migrate_structure.php</code>
Tests	PHPUnit 9.6 (9.6.29) + Guzzle 7.10 via Composer (développement uniquement)	Tests unitaires, d'intégration et fonctionnels (HTTP)	<code>composer.json</code> , <code>phpunit.xml</code> , <code>tests/Unit</code> , <code>tests/Integration</code> , <code>tests/Functional</code>

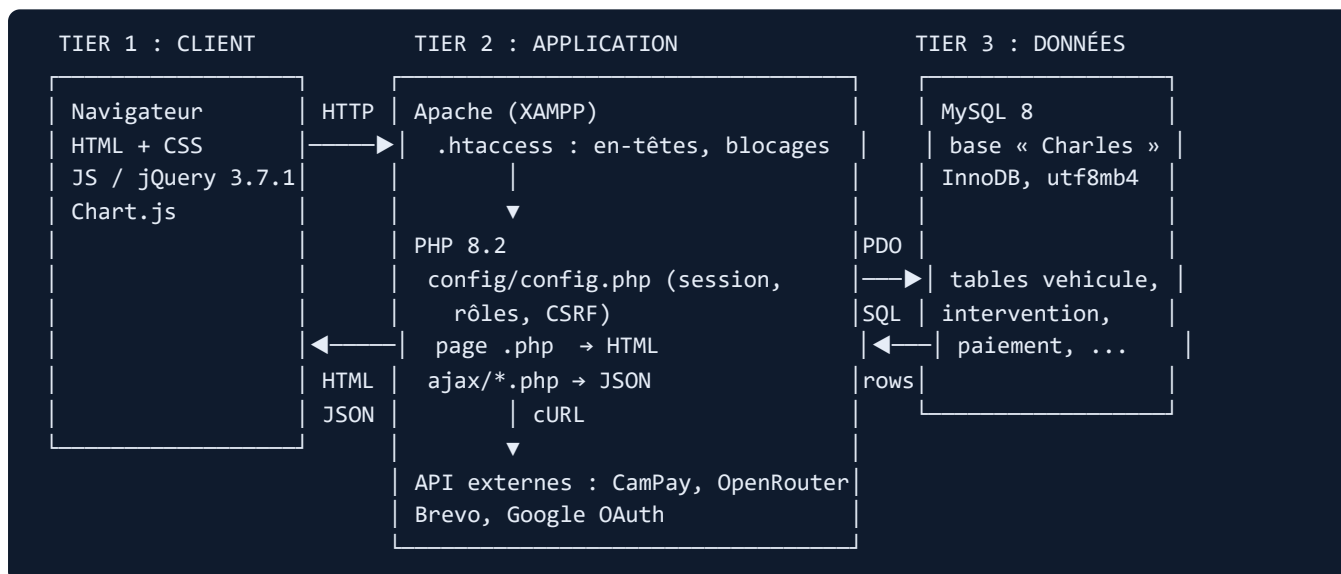
Un point important à souligner : Composer n'est utilisé **que pour les tests**. Le fichier `composer.json` a une section `require` vide ; tout est dans `require-dev`. L'application en production ne charge jamais le dossier `vendor/`.

```
{
    "name": "hch/smartautotrack",
    "description": "Suite de tests (unitaires, intégration, fonctionnels) pour SmartAutoTrack",
    "require": {},
    "require-dev": {
        "phpunit/phpunit": "^9.6",
        "guzzlehttp/guzzle": "^7.9"
    },
    "scripts": {
        "test": "phpunit -c phpunit.xml",
        "test:unit": "phpunit -c phpunit.xml --testsuite Unit"
    }
}
```

Extrait de `composer.json` (abrégé). Le `^9.6` signifie « toute version 9.x à partir de 9.6 » ; la version réellement installée, figée dans `composer.lock`, est 9.6.29.

1.2 Les trois tiers et le trajet d'une requête

L'architecture **trois tiers** sépare : (1) le **client** (le navigateur, qui affiche), (2) le **serveur d'application** (Apache + PHP, qui décide), (3) le **serveur de données** (MySQL, qui stocke). Chaque tier ne parle qu'à son voisin : le navigateur ne touche jamais directement la base.



Suivons une requête concrète : le client ouvre la fenêtre de modification d'un véhicule, et le JavaScript demande les données de ce véhicule.

1. Navigateur : jQuery envoie GET `/HCH/ajax/get_vehicle.php?id=12`
(le cookie de session `HCHSESSID` part avec la requête)
2. Apache : trouve le fichier `.php`, applique `.htaccess`
(`Cache-Control: no-store, X-Frame-Options...`), passe la main à PHP
3. PHP : require `config/config.php` → démarre la session, charge les rôles
vérifie `$_SESSION['user_id']` (sinon : JSON "Non authentifié")
4. PDO : `$conn = (new Database())->getConnection();`
`prepare("SELECT ... WHERE idVehicule = ? AND idClient = ?")`
5. MySQL : exécute la requête préparée, renvoie la ligne
6. PHP : `json_encode(...)` avec `Content-Type: application/json`
7. Navigateur : jQuery reçoit l'objet JSON et préremplit le formulaire

Voici le début réel de cet endpoint, qui montre les étapes 3 et 4 :

```
require_once '../config/config.php';
require_once '../config/database.php';
header('Content-Type: application/json');

if (!isset($_SESSION['user_id'])) {
    echo json_encode(['success' => false, 'message' => 'Non authentifié']);
    exit;
}
$vehicle_id = $_GET['id'] ?? null;
...
$db = new Database();
$conn = $db->getConnection();
if ($_SESSION['role'] === 'client') {
    $stmt = $conn->prepare("SELECT $cols FROM vehicule v WHERE v.idVehicule = ? AND v.idClient
= ?");
    $stmt->execute([$vehicle_id, $_SESSION['user_id']]);
```

Extrait de `ajax/get_vehicule.php`. Notez le cloisonnement : un client ne peut lire que **ses** véhicules (AND `v.idClient = ?`).

Et la connexion PDO elle-même, centralisée dans une seule classe :

```
$this->conn = new PDO(
    "mysql:host=" . $this->host . ";dbname=" . $this->db_name . ";charset=utf8mb4",
    $this->username,
    $this->password,
    array(
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false
    )
);
```

Extrait de `config/database.php`. Les identifiants ne sont pas écrits dans le code : ils viennent de variables d'environnement ou du fichier non versionné `config/local.php`.

Deux types de réponses : HTML et JSON

- **Pages complètes** (`client/dashboard.php`, `admin/statistics.php` ...) : PHP assemble le HTML (en incluant `includes/header.php` et `footer.php`) et le navigateur recharge la page entière.
- **Endpoints AJAX** (dossier `ajax/`) : PHP renvoie seulement des données JSON ; jQuery met à jour un morceau de page sans rechargement (notifications, détails d'une anomalie, assistant IA, statut d'un paiement CamPay).
- **Webhook** (`webhooks/campay.php`) : c'est le serveur de CamPay, et non un navigateur, qui appelle l'application pour confirmer un paiement.
- **Scripts CLI** (`scripts/`) : lancés en ligne de commande (par exemple par une tâche planifiée) ; pas de navigateur du tout.

Le trajet des appels vers l'extérieur

Quand PHP doit parler à CamPay, OpenRouter, Brevo ou Google, il devient lui-même **client HTTP** grâce à `cURL`. Pour CamPay et OpenRouter, une fonction commune centralise ces appels ; `includes/mailer.php` (Brevo) et `includes/google_oauth.php` font leur propre appel `curl_init()`, sur le même principe :

```
function httpJsonRequest(string $method, string $url, array $headers = [], $body = null, int
$timeout = 30): array {
    $ch = curl_init($url);
    $httpHeaders = ['Accept: application/json'];
    ...
    if ($body !== null) {
        $httpHeaders[] = 'Content-Type: application/json';
        curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode($body, JSON_UNESCAPED_UNICODE));
    }
    curl_setopt_array($ch, [
        CURLOPT_CUSTOMREQUEST => strtoupper($method),
        CURLOPT_HTTPHEADER => $httpHeaders,
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_CONNECTTIMEOUT => 10,
        CURLOPT_TIMEOUT => $timeout,
    ]);
}
```

Extrait de `includes/http_client.php`. Les clés d'API sont lues par `appConfig()` et ne transitent jamais par le navigateur.

À retenir

- Trois tiers : navigateur (HTML/CSS/jQuery) → Apache + PHP 8.2 → MySQL 8 via PDO.
- Deux formats de réponse : HTML pour les pages, JSON pour les appels AJAX.
- Le navigateur ne voit jamais la base de données ni les clés d'API : tout passe par PHP.
- Les services externes (CamPay, OpenRouter, Brevo, Google) sont appelés côté serveur par cURL.

1.3 Pourquoi ces technologies ?

Le coût

Toute la pile est **gratuite et open source** : PHP, Apache, MySQL, jQuery, Chart.js, PHPUnit. Il n'y a ni licence à payer, ni serveur spécialisé à louer. Pour un projet étudiant destiné à des garages camerounais au budget limité, c'est décisif.

L'hébergement mutualisé au Cameroun

La quasi-totalité des hébergeurs mutualisés (locaux comme internationaux) proposent d'office « **Apache + PHP + MySQL + phpMyAdmin** » pour quelques milliers de FCFA par mois. On dépose les fichiers par FTP, on importe le fichier SQL, et l'application fonctionne. À l'inverse, une application Node.js, Django ou Spring demanderait un VPS (serveur privé), plus cher et plus difficile à administrer. Le projet a été pensé pour cela : `config/config.php` calcule lui-même l'URL de base, ce qui permet de fonctionner aussi bien sous `/HCH/` qu'à la racine d'un domaine, et le PDF est produit par un générateur en PHP pur parce que `vendor/` n'est pas déployé.

```
/**
 * PdfLite : petit générateur PDF autonome, en PHP pur, sans aucune
 * dépendance (vendor/ n'est pas déployé), suffisant pour un document texte
 * comme le rapport de fin d'intervention (ajax/download_report.php?format=pdf).
```

Extrait du docblock de `includes/pdf_lite.php`.

La simplicité

Un fichier PHP correspond à une page ou à un endpoint : `client/vehicles.php` affiche les véhicules, `ajax/get_vehicle.php` renvoie un véhicule. Pas d'étape de compilation, pas de « build » JavaScript, pas de routeur à configurer. On modifie, on rafraîchit, on voit le résultat. Le développement et le débogage se font avec XAMPP et phpMyAdmin.

Les compétences

PHP, MySQL et jQuery sont les technologies enseignées dans la formation et très répandues au Cameroun ; un futur mainteneur (un autre étudiant, un prestataire local) les connaîtra très probablement. La documentation et les réponses en ligne sont abondantes.

Le contexte réseau

Les pages sont générées côté serveur : le téléphone de l'utilisateur reçoit du HTML prêt à afficher, sans gros paquet JavaScript à télécharger. C'est un avantage sur des connexions mobiles lentes ou chères. Le paiement par **Mobile Money** (CamPay : MTN et Orange) correspond aux habitudes locales, là où une carte bancaire serait peu utilisée.

1.4 Ce qui n'est PAS utilisé, et pourquoi

Non utilisé	Ce que c'est	Pourquoi on s'en passe	Ce qui le remplace
Framework PHP (Laravel, Symfony)	Cadre complet : routeur, ORM, moteur de templates, conteneur	Courbe d'apprentissage, dépendances Composer à déployer, exigences d'hébergement (accès SSH, version PHP précise), surdimensionné pour la taille du projet	PHP « natif » organisé : <code>config/config.php</code> (session, CSRF, rôles), fonctions dans <code>includes/</code> , classe <code>Database</code>
ORM (Eloquent, Doctrine)	Couche qui transforme les tables en objets	Ajoute de l'abstraction ; écrire le SQL à la main permet de le maîtriser et de l'expliquer	PDO + requêtes préparées écrites en SQL
React, Vue, Angular	Frameworks JavaScript pour applications « monopage » (SPA)	Exigent une chaîne de build (Node, npm, bundler), une API séparée, un rendu côté client plus lourd pour les téléphones	HTML rendu par PHP + jQuery pour les parties dynamiques
Bootstrap, Tailwind	Frameworks CSS	Le projet a sa propre charte graphique	Feuilles CSS maison (<code>assets/css/</code>) et variables de thème
SDK officiels (Google, CamPay, OpenRouter)	Bibliothèques fournies par les éditeurs	Il faudrait <code>vendor/</code> en production	Appels REST directs avec cURL (<code>includes/http_client.php</code> , <code>includes/google_oauth.php</code>)
Bibliothèque PDF (TCPDF, Dompdf)	Génération de PDF	Même raison : aucune dépendance déployée	<code>includes/pdf_lite.php</code>

Piège fréquent

Ne dites pas « je n'ai pas utilisé de framework parce que c'est plus simple » sans nuance : le jury peut répondre que Laravel rend justement les choses plus simples. Dites plutôt que c'est un **choix adapté au contexte** (hébergement mutualisé, aucune dépendance en production, équipe d'une personne) et montrez que vous avez **réimplémenté vous-même les protections** qu'un framework offre : requêtes préparées PDO, jetons CSRF, échappement HTML avec `h()`, contrôle d'accès par rôle, mots de passe hachés. Reconnaissez aussi la limite : plus de code à maintenir soi-même, et un framework serait envisageable si l'application grandit.

Piège fréquent

XAMPP n'est pas une technologie de production : c'est l'environnement de développement local. En production, on aurait le même trio Apache + PHP + MySQL chez l'hébergeur. Autre précision : le XAMPP installé fournit **MariaDB 10.4.24** (et non MySQL), mais la machine a aussi **MySQL Server 8.0.41** installé comme service Windows (`MySQL80`) ; c'est cette version qu'annonce le dump phpMyAdmin (« Version du serveur : 8.0.41 »). Sachez dire lequel des deux tourne pendant la démonstration : phpMyAdmin l'affiche sur sa page d'accueil. MariaDB est une variante de MySQL, et les deux conviennent pour le SQL qu'utilise l'application.

Piège fréquent

Les fichiers `db_init.sql`, `sql/schema.sql` et `charles (1).sql` décrivent l'**ancien** schéma (tables en anglais : `users`, `vehicles`, `interventions` ...) et sont marqués « OBSOLÈTE » en tête de fichier. Depuis le 23/09/2026, la base `charles` utilise un nouveau schéma en français et en camelCase (`utilisateur`, `vehicule`, `idVehicule` ...). Il est décrit dans `sql/SCHEMA_ACTUEL.md` et complété par `scripts/migrate_structure.php`. Ne présentez pas au jury un de ces anciens fichiers SQL comme le schéma actuel.

À retenir

- Choix guidés par : coût nul, hébergement mutualisé courant au Cameroun, simplicité de déploiement (FTP + import SQL), compétences disponibles.
- Pas de framework PHP ni de React : aucune dépendance en production ; Composer sert uniquement aux tests.
- Les sécurités qu'apporterait un framework sont codées à la main dans `config/config.php` et `config/database.php`.
- Versions vérifiées : PHP 8.2.12, MySQL 8.0.41 (MariaDB 10.4.24 dans XAMPP), jQuery 3.7.1, PHPUnit 9.6.29.

1.5 Questions possibles du jury

Q. Présentez l'architecture technique de votre application ?

C'est une architecture trois tiers. Le navigateur affiche des pages HTML/CSS avec du JavaScript (jQuery 3.7.1) ; Apache exécute PHP 8.2, qui gère les sessions, les rôles et la logique métier ; MySQL 8 stocke les données, et PHP y accède par PDO avec des requêtes préparées. PHP renvoie soit des pages HTML

complètes, soit du JSON pour les appels AJAX, et appelle en cURL les services externes : CamPay pour le Mobile Money, OpenRouter pour l'assistant IA, Brevo pour les e-mails et Google pour la connexion.

Q. Pourquoi ne pas avoir utilisé un framework comme Laravel ?

Pour pouvoir déployer sur n'importe quel hébergement mutualisé sans dépendances : l'application ne charge pas `vendor/`. Cela m'a aussi obligé à comprendre et à coder moi-même les mécanismes de base (session, CSRF, requêtes préparées, contrôle d'accès). La contrepartie est davantage de code à maintenir ; si le projet grandit, migrer vers un framework serait une évolution logique.

Q. Décrivez le trajet d'une requête AJAX dans votre application ?

Le JavaScript appelle par exemple `ajax/get_vehicle.php?id=12` avec `$.ajax` ; le cookie de session part avec la requête. Apache transmet à PHP, qui charge `config/config.php`, vérifie que l'utilisateur est connecté et son rôle, puis interroge MySQL par une requête préparée PDO. Le résultat est encodé en JSON avec `json_encode` et renvoyé avec l'en-tête `application/json` ; jQuery met alors à jour la page sans la recharger.

Q. À quoi sert Composer dans votre projet si l'application n'a pas de dépendances ?

Uniquement à installer les outils de test : PHPUnit 9.6 et Guzzle (pour les tests fonctionnels qui envoient de vraies requêtes HTTP). Ils sont déclarés en `require-dev` dans `composer.json`, la section `require` est vide. Le dossier `vendor/` n'est donc pas nécessaire en production, et `.htaccess` interdit même l'accès web à `composer.json` et au `.lock`.

Q. Pourquoi jQuery plutôt que React ou du JavaScript moderne seul ?

Les pages sont déjà construites côté serveur par PHP ; on a seulement besoin d'ajouter de l'interactivité (modales, AJAX, thème). jQuery le fait en quelques lignes, se charge depuis un CDN sans étape de compilation et est bien connu. React demanderait Node, npm, une API séparée et un rendu côté client plus lourd, ce qui n'apporte rien à ce périmètre et pénalise les connexions mobiles lentes.

Q. Où sont stockés les mots de passe de la base et les clés d'API ?

Jamais dans le code versionné. La fonction `appConfig()` et la classe `Database` lisent d'abord les variables d'environnement, puis le fichier `config/local.php` qui n'est pas versionné (un modèle `local.example.php` est fourni), et le dossier `config/` est protégé par son propre `.htaccess`. Les clés ne sont utilisées que côté serveur, le navigateur ne les voit jamais.

Q. Votre application peut-elle être hébergée facilement au Cameroun ?

Oui, c'est l'un des critères du choix. Elle ne demande que Apache, PHP 8 avec les extensions PDO MySQL et cURL, et une base MySQL : c'est l'offre standard de tout hébergement mutualisé. On transfère les fichiers par FTP, on importe le SQL via phpMyAdmin et on renseigne `local.php` ; l'URL de base est calculée automatiquement. Seuls les rappels d'échéance nécessitent une tâche planifiée (cron sous Linux, Planificateur de tâches sous Windows) pour lancer `scripts/send_reminders.php`, comme l'indique l'en-tête de ce script.

Q. Comment les PDF sont-ils générés sans bibliothèque ?

Par une classe maison, `PdfLite`, écrite en PHP pur dans `includes/pdf_lite.php`. Elle écrit directement le format PDF 1.4 : pages A4, polices standard Helvetica, coupure des lignes selon la largeur du texte, pied de page « Page n / N », table des positions (xref) et compression quand zlib est disponible. Ce choix évite toute dépendance externe, au prix de fonctions limitées (pas d'images ni de polices personnalisées).

2. PHP : le langage serveur

PHP est le cœur de SmartAutoTrack. Chaque page que voit un client, un garage, un technicien ou un administrateur est d'abord **fabriquée sur le serveur** par un script PHP. Ce script lit la session, interroge MySQL et contrôle les droits, puis renvoie du HTML (ou du JSON, ou un PDF) au navigateur. Ce chapitre suit le chemin d'une requête dans le projet et présente chaque outil du langage au moment où il intervient.

2.1 Rôle de PHP dans l'application

Ce que c'est. PHP (PHP: Hypertext Preprocessor) est un langage de script **interprété côté serveur**. Le navigateur demande une URL (par exemple `/HCH/client/vehicles.php`). Apache, le serveur web de XAMPP, confie le fichier à l'interpréteur PHP, qui exécute le code et renvoie **uniquement le résultat**. Le client ne voit jamais le code source PHP. Le projet tourne avec **PHP 8.2** (version fournie par XAMPP).

Pourquoi ici. PHP est installé de base avec XAMPP et s'intègre directement à Apache et MySQL. On peut mélanger le HTML et la logique dans un même fichier, ce qui simplifie un projet de cette taille. Il gère nativement tout ce dont l'application a besoin : sessions, accès base de données (PDO), envoi de fichiers, cURL pour appeler CamPay et OpenRouter, compression zlib pour les PDF.

Ce qui est fait en PHP dans SmartAutoTrack :

- l'authentification (connexion classique et Google OAuth), les rôles et le cloisonnement des données ;
- toutes les lectures et écritures en base MySQL ;
- la validation **serveur** des formulaires : le JavaScript filtre aussi, mais on ne s'y fie jamais ;
- les points d'accès AJAX du dossier `ajax/`, qui répondent en JSON ;
- les appels sortants vers CamPay (Mobile Money) et OpenRouter (assistant IA) ;
- la génération des rapports et des reçus en PDF ;
- les scripts en ligne de commande : migrations et envoi quotidien des rappels d'échéance.

2.2 Organisation des fichiers

Dossier	Contenu	Exemple
<code>config/</code>	Configuration, session, fonctions transverses (sécurité, validation), connexion à la base	<code>config.php</code> , <code>database.php</code> , <code>roles.php</code>
<code>includes/</code>	Logique métier partagée , réutilisée par plusieurs pages	<code>payments.php</code> , <code>campay.php</code> , <code>ai.php</code> , <code>maintenance.php</code> , <code>pdf_lite.php</code> , <code>header.php</code> , <code>footer.php</code>
<code>client/</code> , <code>garage/</code> , <code>technicien/</code> , <code>admin/</code>	Pages de chaque espace (une par fonctionnalité)	<code>client/vehicles.php</code>
<code>auth/</code>	Connexion, inscription, mot de passe oublié, Google	<code>auth/login.php</code>
<code>ajax/</code>	Points d'accès appelés en JavaScript, qui répondent en JSON ou envoient un fichier	<code>ajax/campay_collect.php</code>

Dossier	Contenu	Exemple
webhooks/	URL appelées directement par CamPay (sans session ni CSRF, mais avec une signature à vérifier)	webhooks/campay.php : confirmation de paiement
scripts/	Scripts en ligne de commande, jamais lancés depuis le web	scripts/send_reminders.php

Une page de `client/` contient peu de logique métier : elle appelle des fonctions de `includes/`. Le paiement, par exemple, est écrit une seule fois dans `includes/payments.php` et sert à la fois aux réparations et aux abonnements Premium.

2.3 Le cycle d'une page PHP

Toutes les pages suivent le même déroulé. Prenons `client/vehicles.php` :

1. **Inclusion de la configuration** : `require_once '../config/config.php'` démarre la session et charge les fonctions communes.
2. **Contrôle d'accès** : `requireRole('client')`.
3. **Traitement du formulaire** si la requête est un POST : jeton CSRF, nettoyage, validation, écriture en base, puis redirection.
4. **Requêtes de lecture** pour préparer l'affichage (liste des véhicules, badges d'échéances...).
5. **Affichage** : `include '../includes/header.php'`, puis le HTML de la page, puis `include '../includes/footer.php'`.

```
<?php
require_once '../config/config.php';
require_once '../config/database.php';
require_once '../config/roles.php';
require_once '../includes/subscription.php';
require_once '../includes/maintenance.php';
// ... docblock de la page ...
requireRole('client');

$db = new Database();
$conn = $db->getConnection();
```

Début de `client/vehicles.php`.

Le contrôle d'accès est défini dans `config/config.php`. Il lit uniquement la session : le rôle n'est **jamais** fourni par le navigateur.

```
function requireAuth() {
    if (!isset($_SESSION['user_id']) || !isset($_SESSION['role'])) {
        redirect('auth/login.php');
    }
}

function requireRole($role) {
    requireAuth();
    if ($_SESSION['role'] !== $role) {
        redirect('dashboard.php');
    }
}
```

config/config.php.

La logique s'exécute **avant** d'inclure `header.php`, pour une raison technique : `header('Location: ...')` et `http_response_code()` ne fonctionnent que si aucun octet de HTML n'est encore parti vers le navigateur. Le docblock de `includes/header.php` le précise : « En-tête HTML commun à toutes les pages (inclus après la logique de la page) ». La page appelante peut régler l'en-tête avec des variables définies juste avant l'inclusion :

```
$pageTitle = 'Connexion';
$bodyClass = 'authv2';
$hideNavbar = true;
$extraStylesheets = ['assets/css/auth_v2.css'];
$extraFonts = [/* ... polices Google ... */];
include '../includes/header.php';
?>
<div class="authv2-shell"> ... HTML de la page ...
<?php include '../includes/footer.php'; ?>
```

`auth/login.php`. Une variable définie dans la page est visible dans le fichier inclus, car `include` partage la portée courante.

À retenir

- Ordre fixe : config → contrôle d'accès → traitement du POST → lectures → header → HTML → footer.
- Tout traitement qui peut rediriger passe avant le premier `echo` ou le premier HTML.
- `config.php` est inclus **en premier** par toutes les pages : c'est lui qui démarre la session.

2.4 Syntaxe essentielle utilisée dans le projet

Variables et tableaux associatifs

Une variable commence par `$` et n'a pas de type déclaré : PHP est **dynamiquement typé**. Le **tableau associatif** (clé => valeur) sert partout dans le projet : lignes renvoyées par la base, réponses JSON, options PDO, correspondances de statuts.

```
$statutMap = ['VALIDE' => 'actif', 'EN_ATTENTE' => 'pending', 'REJETE' => 'rejete'];
if ($profile['role'] === ROLE_TECHNICIEN) {
    $_SESSION['statut'] = $statutMap[$profile['statutValidation']] ?? 'pending';
}
```

`auth/login.php` : convertit le statut de la base vers l'ancienne convention.

Comparaisons strictes

Le projet utilise `===` et `!==`, qui comparent la valeur **et** le type. Avec `==`, PHP convertit les types avant de comparer, ce qui peut donner des résultats surprenants (par exemple `"abc" == 0` était vrai avant PHP 8). On écrit donc `$_SESSION['role'] !== $role`, et `in_array($format, ['html', 'pdf'], true)` avec le troisième paramètre `true` pour une comparaison stricte.

Constantes : `define()` et `const`

Une constante ne change plus une fois définie. On l'utilise pour éviter les « chaînes magiques » écrites à plusieurs endroits.

```

if (!defined('SITE_NAME')) {
    // ... calcul de $basePath ...
    define('SITE_NAME', 'SmartAutoTrack');
    define('BASE_PATH', $basePath);
    define('SITE_URL', $protocol . '://' . $host . $basePath);
    define('ROLE_ADMIN', 'admin');
    define('ROLE_CLIENT', 'client');
}

```

`config/config.php`. La garde `defined()` évite une erreur si le fichier est inclus deux fois.

Le mot-clé `const` fait la même chose, mais au niveau du fichier ou d'une classe :

```

/** Au-delà, un paiement resté EN_ATTENTE n'empêche plus une nouvelle tentative. */
const PAYMENT_PENDING_LOCK_SECONDS = 900;

```

`includes/payments.php`. PdfLite déclare aussi `public const PAGE_WIDTH = 595.28;`.

Fonctions et typage des paramètres

Les fonctions récentes du projet déclarent le **type** de leurs paramètres et de leur valeur de retour. Si on passe un mauvais type, PHP lève une `TypeError` au lieu de continuer avec une valeur fausse. Le `?` devant un type (`?array`, `?int`) signifie « ce type ou `null` ». `void` signifie « ne renvoie rien ».

```

/** Réparation terminée, au coût non nul, appartenant à ce client ; sinon null. */
function paymentPayableRepair(PDO $conn, int $clientId, int $repairId, bool $forUpdate = false): ?array {
    $stmt = $conn->prepare("
        SELECT r.idReparation, r.titre, r.cout, i.idIntervention, i.idClient
        FROM reparation r
        JOIN intervention i ON i.idIntervention = r.idIntervention
        WHERE r.idReparation = ? AND i.idClient = ? AND r.statut = 'TERMINEE' AND r.cout > 0
    " . ($forUpdate ? ' FOR UPDATE' : ''));
    $stmt->execute([$repairId, $clientId]);
    $row = $stmt->fetch(PDO::FETCH_ASSOC);
    return $row ?: null;
}

```

`includes/payments.php` : paramètre facultatif avec valeur par défaut (`$forUpdate = false`) et retour `?array`.

Le projet mélange deux époques : les anciennes fonctions de `config.php` (`requireRole($role)`, `sanitize($data)`) ne sont pas typées, alors que les modules plus récents (`payments.php`, `http_client.php`, `maintenance.php`) le sont.

Variable static

Une variable `static` dans une fonction garde sa valeur d'un appel à l'autre pendant la requête. Le projet s'en sert comme **cache** : `appConfig()` ne lit `config/local.php` qu'une fois, et `paymentsReady()` n'interroge `information_schema` qu'une fois.

```
function paymentsReady(PDO $conn): bool {
    static $ready = null;
    if ($ready === null) {
        $stmt = $conn->query("SELECT COUNT(*) FROM information_schema.COLUMNS ...");
        $ready = (int)$stmt->fetchColumn() === 3;
    }
    return $ready;
}
```

includes/payments.php (requête abrégée).

require_once, require, include

Instruction	Fichier absent	Inclus deux fois ?	Usage dans le projet
require_once	Erreur fatale, le script s'arrête	Non, ignoré la 2e fois	Fichiers de fonctions et de classes (config.php , includes/*.php) : les redéclarer provoquerait une erreur « Cannot redeclare »
require	Erreur fatale	Oui	\$loaded = require \$localFile; (config/database.php) pour lire le tableau renvoyé par config/local.php
include	Simple avertissement, le script continue	Oui	Gabarits d'affichage header.php / footer.php

Les chemins sont écrits de deux façons. '../config/config.php' est relatif au dossier de la page. __DIR__ . '/campay.php' part du dossier du fichier courant, quel que soit l'appelant : c'est le cas de includes/payments.php, et c'est plus robuste.

Opérateurs ?? et ?:

- \$a ?? \$b (**coalescence nulle**) renvoie \$a s'il **existe et n'est pas null**, sinon \$b, sans avertissement « Undefined index ». C'est l'opérateur le plus courant du projet pour lire \$_POST et \$_GET : \$_POST['email'] ?? ''.
- \$a ?: \$b (**ternaire court**) renvoie \$a s'il est « **vrai** », sinon \$b. Or '', 0, '0' et [] sont « faux ».

```
$this->host      = getenv('HCH_DB_HOST') ?: ($local['db_host'] ?? 'localhost');
$this->db_name    = getenv('HCH_DB_NAME') ?: ($local['db_name'] ?? 'Charles');
```

config/database.php : getenv() renvoie false si la variable n'existe pas, d'où ?; ; on se replie ensuite sur local.php, puis sur une valeur par défaut.

Piège fréquent

Confondre ?? et ?:. Le docblock de Database::__construct() en donne un exemple : un mot de passe MySQL **vide** est une valeur légitime sous XAMPP, mais ?: le considérerait comme absent. Pour ce paramètre, le code teste donc explicitement !== false au lieu d'utiliser ?:.

Fonctions fléchées fn et match

Les **fonctions fléchées** (fn, PHP 7.4) sont des fonctions anonymes courtes qui capturent automatiquement les variables extérieures. Le projet s'en sert avec array_filter et array_map pour calculer des statistiques :

```
$stats['en_cours'] = count(array_filter($reparations, fn($r) => in_array($r['statut'],
['planifiee', 'en_cours'])));
$stats['terminees'] = count(array_filter($reparations, fn($r) => $r['statut'] === 'terminee'));
```

client/reparations.php.

L'expression `match` (PHP 8) **n'est pas utilisée** dans le projet. Pour choisir entre plusieurs cas, on trouve un `switch` (redirection selon le rôle dans `auth/login.php`) ou un tableau de correspondance (`$statutMap`). Si le jury pose la question : `match` compare strictement (`===`), renvoie une valeur et n'a pas besoin de `break`.

Exceptions : try / catch / throw

Une **exception** est une erreur « lancée » (`throw`) qui remonte la pile d'appels jusqu'à un bloc `catch` qui la traite. Le projet en définit une à lui : `class CampayException extends RuntimeException {}` (`includes/campay.php`). Elle sert aux erreurs **affichables au client**, comme « Numéro Mobile Money invalide ». Les autres erreurs (base, réseau) sont journalisées et remplacées par un message générique :

```
try {
    $conn = (new Database())->getConnection();
    $result = paymentStart($conn, (int)$SESSION['user_id'], $repairId, (string)
($POST['telephone'] ?? ''));
    echo json_encode(['success' => true, 'paiement_id' => $result['idPaielement'], /* ... */]);
} catch (CampayException $e) {
    http_response_code(422);
    echo json_encode(['success' => false, 'message' => $e->getMessage()]);
} catch (Throwable $e) {
    error_log('[SmartAutoTrack] campay_collect : ' . $e->getMessage());
    http_response_code(500);
    echo json_encode(['success' => false, 'message' => 'Erreur lors du lancement du paiement.']);
}
```

`ajax/campay_collect.php` : les blocs `catch` vont du plus précis au plus général. `Throwable` attrape tout, y compris les `Error` de PHP.

En production, les erreurs ne sont jamais montrées au visiteur. `config.php` commence par `ini_set('display_errors', getenv('HCH_DEBUG') ? '1' : '0');` et `ini_set('log_errors', '1');` : les détails partent dans les journaux du serveur.

2.5 Les sessions

Ce que c'est. HTTP est **sans état** : chaque requête arrive sans mémoire des précédentes. Une **session** PHP fait le lien entre ces requêtes. Au démarrage, le serveur crée un identifiant aléatoire, l'envoie au navigateur dans un **cookie**, et garde de son côté un fichier de données associé. Le tableau `$_SESSION` lit et écrit ce fichier. Le navigateur ne possède que l'identifiant, jamais les données : il ne peut donc pas modifier son rôle.

```

if (session_status() === PHP_SESSION_NONE) {
    session_name('HCHSESSID');
    session_set_cookie_params([
        'lifetime' => 0,
        'path' => BASE_PATH,
        'domain' => '',
        'secure' => (isset($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off'),
        'httponly' => true,
        'samesite' => 'Lax',
    ]);
    session_start();
}

```

config/config.php.

Option	Effet
<code>session_name('HCHSESSID') + path</code>	Cookie propre à l'application, pour ne pas partager la session avec d'autres projets du même XAMPP (l'ancien cookie global <code>PHPSESSID</code> provoquait des « connexions fantômes »)
<code>lifetime => 0</code>	Cookie supprimé à la fermeture du navigateur
<code>httponly</code>	Le JavaScript ne peut pas lire le cookie, ce qui limite le vol de session par XSS
<code>secure</code>	Cookie envoyé uniquement en HTTPS, quand le site est en HTTPS
<code>samesite => 'Lax'</code>	Cookie non envoyé sur les requêtes POST (et les sous-requêtes) venant d'un autre site, mais toujours envoyé quand l'utilisateur suit un lien GET vers le site : protection complémentaire contre le CSRF, pas suffisante seule

À la connexion réussie, `auth/login.php` régénère l'identifiant **avant** de remplir la session :

```

session_regenerate_id(true);
loginThrottleClear($email);

$_SESSION['user_id'] = $user['idUtilisateur'];
$_SESSION['nom'] = $user['nom'];
$_SESSION['prenom'] = $user['prenom'];
$_SESSION['email'] = $user['email'];
$_SESSION['role'] = $profile['role'];

```

`auth/login.php`.

`session_regenerate_id(true)` protège contre la **fixation de session**. Dans cette attaque, l'attaquant fait d'abord utiliser à la victime un identifiant qu'il connaît, puis attend qu'elle se connecte avec. Si l'identifiant change au moment de la connexion, celui que connaît l'attaquant ne sert plus à rien. Le paramètre `true` supprime aussi l'ancien fichier de session.

Deux autres fonctions complètent le dispositif :

- `enforceActiveSession()`, exécutée à chaque inclusion de `config.php`, revérifie le compte en base. Un compte supprimé, suspendu ou rejeté perd sa session **dès sa requête suivante**, sans attendre qu'il se déconnecte.
- `destroySession()`, appelée à la déconnexion, vide `$_SESSION`, fait expirer le cookie (date dans le passé avec `setcookie(..., time() - 42000, ...)`) puis appelle `session_destroy()`.

2.6 Données reçues : \$_GET, \$_POST, \$_FILES, \$_SERVER et filter_var

Ces tableaux **superglobaux** sont remplis automatiquement par PHP :

- `$_GET` : paramètres de l'URL (`vehicles.php?action=edit&id=4`);
- `$_POST` : champs d'un formulaire envoyé en POST ;
- `$_FILES` : fichiers envoyés (nom d'origine, taille, chemin temporaire, code d'erreur) ;
- `$_SERVER` : informations sur la requête (`REQUEST_METHOD` , `REMOTE_ADDR` , `HTTP_HOST` , en-têtes comme `HTTP_X_CSRF_TOKEN`).

Toutes ces données viennent de l'utilisateur et peuvent être falsifiées, même un champ caché ou un menu déroulant. Le projet les lit avec `??`, les convertit (`(int)`) ou les valide. `filter_var` est la fonction de validation intégrée à PHP :

```
$reparation_id = filter_var($_GET['id'] ?? null, FILTER_VALIDATE_INT, ['options' => ['min_range' => 1]]);

if (!$reparation_id) {
    http_response_code(400);
    exit('ID de réparation invalide');
}

// Format de sortie (liste blanche) : HTML historique par défaut, ou PDF.
$format = (string)($_GET['format'] ?? 'html');
if (!in_array($format, ['html', 'pdf'], true)) {
    http_response_code(400);
    exit('Format de rapport invalide');
}
```

`ajax/download_report.php` : `FILTER_VALIDATE_INT` renvoie l'entier, ou `false` si la valeur n'en est pas un ou vaut moins de 1. Le format est vérifié par **liste blanche**.

Le projet utilise aussi `FILTER_VALIDATE_EMAIL` (`validateEmail()`) et `FILTER_VALIDATE_BOOLEAN` (`appConfigBool()`). Pour les noms, les plaques et les modèles, il utilise des expressions régulières (`preg_match`) : `validateLettersOnly()`, `validatePlate()`, `validateModel()`. Pour les nombres, un simple `(int)($_POST['annee'] ?? 0)` suffit.

2.7 PDO et les requêtes préparées

Ce que c'est. PDO (PHP Data Objects) est l'interface orientée objet de PHP pour parler à une base de données. La connexion est créée dans la classe `Database` :

```
$this->conn = new PDO(
    "mysql:host=" . $this->host . ";dbname=" . $this->db_name . ";charset=utf8mb4",
    $this->username,
    $this->password,
    array(
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false
    )
);
```

`config/database.php`. Les identifiants ne sont pas écrits dans le code : ils viennent des variables d'environnement ou de `config/local.php`, qui n'est pas versionné.

- `charset=utf8mb4` : UTF-8 complet, pour les accents et les emojis.
- `ERRMODE_EXCEPTION` : une erreur SQL lève une `PDOException` au lieu de passer inaperçue (c'est le comportement par défaut depuis PHP 8, mais le projet le déclare explicitement).
- `FETCH_ASSOC` : chaque ligne est un tableau associatif `['marque' => 'Toyota', ...]`.
- `EMULATE_PREPARES => false` : ce sont de **vraies** requêtes préparées, exécutées par MySQL lui-même.

Si la connexion échoue, le détail va dans `error_log` et le visiteur reçoit un code 503 avec un message neutre.

Requête préparée. On envoie d'abord à MySQL le texte SQL avec des marqueurs `?` (`prepare`), puis les valeurs **séparément** (`execute`). Ces valeurs ne sont jamais interprétées comme du SQL : c'est la protection contre l'**injection SQL**.

```
$stmt = $conn->prepare("
    UPDATE vehicule
    SET marque = ?, modele = ?, immatriculation = ?, annee = ?, couleur = ?, kilometrage = ?
    WHERE idVehicule = ? AND idClient = ?
");
$stmt->execute([$marque, $modele, $immatriculation, $annee, $couleur, $kilometrage, $vehicule_id,
$_SESSION['user_id']]);
header('Location: vehicles.php?success=updated');
exit;
```

`client/vehicles.php` : la condition `AND idClient = ?` avec l'identifiant pris dans la session empêche un client de modifier le véhicule d'un autre client en changeant l'`id` dans l'URL.

Méthodes de lecture utilisées : `fetch()` (une ligne, ou `false`), `fetchAll()` (toutes les lignes), `fetchColumn()` (une seule valeur, par exemple un `COUNT(*)`), et `lastInsertId()` (identifiant de la ligne qui vient d'être insérée).

Pour une liste de longueur variable (`IN (...)`), le projet génère autant de `?` que de valeurs :

```
$in = implode(',', array_fill(0, count($repairIds), '?'));
$stmt = $conn->prepare("SELECT idReparation, statut, datePaiement FROM paiement WHERE idReparation
IN ($in) AND statut IN ('PAYE', 'EN_ATTENTE')");
$stmt->execute($repairIds);
```

`includes/payments.php`. Seuls des `?` sont insérés dans la chaîne, jamais les valeurs elles-mêmes (qui ont en plus été converties en entiers par `array_map('intval', ...)`).

2.8 Les transactions PDO

Ce que c'est. Une **transaction** regroupe plusieurs requêtes en un bloc « tout ou rien ». `beginTransaction()` ouvre le bloc. `commit()` valide tout. `rollback()` annule tout. Combinée à `SELECT ... FOR UPDATE`, qui verrouille des lignes jusqu'à la fin de la transaction, elle empêche aussi deux requêtes simultanées de se marcher dessus : c'est ce qu'on appelle une **condition de concurrence** (race condition).

```

$conn->beginTransaction();
try {
    $repair = paymentPayableRepair($conn, $clientId, $repairId, true); // FOR UPDATE
    if (!$repair) {
        throw new CampayException('Cette réparation ne peut pas être payée en ligne.');
```

includes/payments.php, fonction `paymentStart()` (abrégée). Si le client clique deux fois sur « Payer », la deuxième requête attend le verrou, puis voit le paiement EN_ATTENTE et est refusée. On ne crée donc jamais deux paiements pour une même réparation.

client/vehicles.php applique le même principe à la limite de véhicules de la formule : `SELECT idClient FROM client WHERE idClient = ? FOR UPDATE`, puis on recompte les véhicules sous verrou avant l'INSERT. Dans le `catch`, le code vérifie `if ($conn->inTransaction())` avant d'appeler `rollback()`, car une transaction déjà annulée ne peut pas l'être une seconde fois.

Piège fréquent

Une transaction ne protège rien si la vérification (« y a-t-il déjà un paiement ? ») a lieu **en dehors** de la transaction ou **sans** `FOR UPDATE`. Deux requêtes peuvent alors lire « non » en même temps, puis insérer toutes les deux. Le verrou doit être posé **avant** la vérification. Autre condition : les tables doivent utiliser le moteur InnoDB, car MyISAM ne gère pas les transactions.

2.9 Mots de passe : `password_hash` et `password_verify`

```

/** Hache un mot de passe avec l'algorithme par défaut de PHP (bcrypt). */
function hashPassword($password) {
    return password_hash($password, PASSWORD_DEFAULT);
}

/** Vérifie un mot de passe en clair contre le hachage stocké (colonne motDePasse). */
function verifyPassword($password, $hash) {
    return password_verify($password, $hash);
}
```

config/config.php.

Un **hachage** est une transformation à **sens unique** : on ne peut pas retrouver le mot de passe à partir du résultat. `password_hash` utilise aujourd'hui **bcrypt**. Il ajoute automatiquement un **sel** aléatoire (deux mots de passe identiques donnent deux hachages différents) et il est volontairement **lent**, ce qui freine les attaques par force brute. Le résultat (environ 60 caractères, commençant par `$2y$`) contient l'algorithme, le coût et le

sel. `password_verify` relit ces informations et refait le calcul. On ne compare donc jamais deux hachages avec `==`.

À la connexion, le message d'erreur est **volontairement générique** (« Email ou mot de passe incorrect ») pour ne pas révéler quels emails sont inscrits. Les échecs répétés sont limités par `includes/login_throttle.php`, avec un compteur par email et un autre par adresse IP. Au-delà du seuil, la page répond `http_response_code(429)` avec un en-tête `Retry-After`. La robustesse des nouveaux mots de passe est vérifiée par `includes/password_policy.php`.

2.10 Protection CSRF

Ce que c'est. Le CSRF (Cross-Site Request Forgery) consiste à faire envoyer, à l'insu d'un utilisateur connecté, un formulaire vers SmartAutoTrack depuis un site piégé. Le navigateur joint automatiquement le cookie de session. La parade est un **jeton secret** : il est stocké en session, inclus dans chaque formulaire et vérifié à la réception. Le site piégé ne peut pas le connaître.

```
function generateCSRFToken() {
    if (!isset($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

/** Compare le jeton reçu à celui de la session, en temps constant. */
function verifyCSRFToken($token) {
    return isset($_SESSION['csrf_token']) && is_string($token) &&
    hash_equals($_SESSION['csrf_token'], $token);
}

function verifyRequestCSRF() {
    return verifyCSRFToken($_POST['csrf_token'] ?? ($_SERVER['HTTP_X_CSRF_TOKEN'] ?? ''));
}
```

`config/config.php`.

- `random_bytes(32)` produit 32 octets cryptographiquement sûrs, et `bin2hex` les écrit en 64 caractères hexadécimaux.
- `hash_equals` compare en **temps constant**, ce qui évite de deviner le jeton caractère par caractère en mesurant le temps de réponse.
- Formulaire classique : `<input type="hidden" name="csrf_token" value="<?php echo generateCSRFToken(); ?>">` (dans `auth/login.php`).
- AJAX : `header.php` publie le jeton dans `<meta name="csrf-token">`, et `assets/js/main.js` l'envoie dans l'en-tête `X-CSRF-Token`. PHP le lit dans `$_SERVER['HTTP_X_CSRF_TOKEN']`.

Piège fréquent

Le jeton ne protège que les actions qui le vérifient. Dans `client/vehicles.php`, l'ajout et la modification vérifient le CSRF, mais la **suppression** se fait par un simple lien GET (`?action=delete&id=N`) sans jeton. Le `confirm()` JavaScript affiché avant la suppression ne protège de rien : un lien piégé ouvert depuis un autre site ne passe pas par ce script, et `SameSite=Lax` laisse passer le cookie sur une navigation GET. Le filtre `idClient` empêche bien de toucher au véhicule d'un autre client, mais un lien piégé pourrait faire supprimer un véhicule de la victime (s'il n'a pas d'intervention liée). La bonne pratique : toute action qui modifie des données passe en POST avec un jeton. Mieux vaut le reconnaître soi-même si le jury le remarque.

2.11 Échappement à l'affichage : `h()`, `htmlspecialchars` et `sanitize`

Ce que c'est. La faille **XSS** (Cross-Site Scripting) apparaît quand une donnée saisie par un utilisateur est réaffichée telle quelle. Si un utilisateur écrit `<script>...</script>` comme nom de véhicule, ce script s'exécuterait chez tous ceux qui affichent la page. `htmlspecialchars` transforme les caractères spéciaux en **entités HTML** : `<` devient `<`, `"` devient `"`, l'apostrophe devient `'`. Le navigateur les affiche alors comme du texte, sans les exécuter.

```
function h($value) {
    return htmlspecialchars((string)($value ?? ''), ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8', false);
}

function sanitize($data) {
    return htmlspecialchars(strip_tags(trim($data)), ENT_QUOTES, 'UTF-8');
}
```

`config/config.php`.

- `sanitize()` s'applique **à l'entrée**, avant l'enregistrement : il retire les espaces de bord et les balises, puis encode les caractères. Les valeurs sont donc stockées **déjà encodées**.
- `h()` s'applique **à la sortie**, à chaque affichage : `<?php echo h($v['immatriculation']); ?>` dans `client/vehicles.php`. `ENT_QUOTES` encode les deux types de guillemets (indispensable dans un attribut `value="..."`). Le dernier paramètre `false` (`double_encode`) évite de ré-encoder ce que `sanitize()` a déjà encodé : on n'affiche pas `&#039;`.

Piège fréquent

L'échappement dépend du contexte. `h()` protège le HTML, mais pas une URL ni du JavaScript. L'encodage à l'entrée par `sanitize()` a aussi un effet de bord : une apostrophe arrive sous la forme `'` dans les fonctions de validation. C'est pour cela que `validateLettersOnly()` commence par `html_entity_decode()`. Sans ce décodage, « O'Brien » serait rejeté. La règle à citer devant le jury : **valider à l'entrée, échapper à la sortie**.

2.12 Envoi de fichiers : `$FILES` et `finfo`

Lors de l'inscription, un technicien joint ses diplômes et certificats. Le nom et l'extension d'un fichier sont fournis par le navigateur et donc falsifiables : un `virus.php` renommé en `diplome.pdf` passerait un contrôle sur l'extension. Le projet lit le **contenu réel** du fichier avec `finfo`, qui détecte le type MIME d'après les premiers octets :

```

$allowed_mimes = [
    'application/pdf' => 'pdf',
    'image/jpeg'      => 'jpg',
    'image/png'       => 'png',
];
$finfo = new finfo(FILEINFO_MIME_TYPE);
$max_size = 5 * 1024 * 1024; // 5MB
// ... boucle sur $_FILES['documents'] ...
$file_mime = is_uploaded_file($file_tmp) ? $finfo->file($file_tmp) : false;
if (!$file_mime || !isset($allowed_mimes[$file_mime])) {
    $errors[] = "Type de fichier non autorisé pour $file_name. Types autorisés: PDF, JPG, PNG";
    continue;
}
$file_ext = $allowed_mimes[$file_mime];
// Nom de fichier aléatoire (non prédictible), extension issue du type réel
$new_file_name = $user_id . '_' . bin2hex(random_bytes(16)) . '.' . $file_ext;
move_uploaded_file($file_tmp, $file_path);

```

auth/register.php (abrégé).

Le contrôle se fait en plusieurs couches :

1. `UPLOAD_ERR_OK` : vérifie que l'envoi s'est bien passé ;
2. `is_uploaded_file` : vérifie que le fichier vient vraiment de l'envoi HTTP ;
3. type MIME réel comparé à une liste blanche ;
4. taille maximale de 5 Mo ;
5. **nouveau nom aléatoire**, avec une extension déduite du type réel et non du nom d'origine ;
6. `move_uploaded_file` : déplace le fichier hors du dossier temporaire.

Le téléchargement (`ajax/download_document.php`) refait la détection avec `finfo` pour envoyer le bon Content-Type . Le même contrôle sert au logo d'un garage (`garage/parametres.php`).

2.13 Redirections : `header('Location')` + `exit`

```

function redirect($url) {
    header('Location: ' . SITE_URL . ltrim($url, '/'));
    exit();
}

```

config/config.php .

`header('Location: ...')` ajoute un en-tête HTTP qui demande au navigateur d'aller à une autre adresse, avec le code 302 par défaut. Ce n'est qu'une **demande** : le script PHP continue de s'exécuter tant qu'on ne l'arrête pas. Le `exit` est donc **indispensable**. Sans lui, dans `requireRole()`, la suite de la page protégée s'exécuterait quand même pour un visiteur non autorisé, et son contenu partirait dans la réponse.

Après un formulaire traité avec succès, le projet applique le motif **Post/Redirect/Get** : `header('Location: vehicles.php?success=added');` `exit;` . Si l'utilisateur rafraîchit la page, le navigateur recharge la page en GET au lieu de renvoyer le formulaire, ce qui évite un deuxième ajout.

2.14 Réponses JSON des fichiers ajax/

Les fichiers de `ajax/` ne renvoient pas une page : ils renvoient des **données** que le JavaScript exploite (paiement CamPay, assistant IA, notifications, messagerie). Le schéma est toujours le même :

1. déclarer le type de réponse : `header('Content-Type: application/json');` ;

2. contrôler l'accès avec `requireJsonAuth()` ;
3. vérifier la méthode HTTP et le jeton CSRF ;
4. valider les paramètres ;
5. répondre avec `echo json_encode([...])` et un **code HTTP** qui a du sens.

```
function requireJsonAuth($role = null) {
    if (!isset($_SESSION['user_id']) || !isset($_SESSION['role'])) {
        http_response_code(401);
        header('Content-Type: application/json');
        exit(json_encode(['success' => false, 'message' => 'Non authentifié']));
    }
    if ($role !== null && $_SESSION['role'] !== $role) {
        http_response_code(403);
        header('Content-Type: application/json');
        exit(json_encode(['success' => false, 'message' => 'Accès refusé']));
    }
}
```

`config/config.php`. Une redirection vers la page de connexion n'aurait aucun sens pour un appel `fetch()` : on renvoie plutôt une erreur JSON que le JavaScript peut afficher.

Code	Sens	Exemple dans le projet
200	Succès (valeur par défaut)	paiement lancé
400	Requête mal formée	« Réparation invalide » (identifiant non entier)
401 / 403	Non connecté / pas le droit	<code>requireJsonAuth()</code> , jeton CSRF invalide
404	Introuvable (ou hors du périmètre de l'utilisateur)	rapport d'une réparation d'un autre client
422	Données valides en forme mais refusées par la règle métier	<code>CampayException</code> (« déjà payée »)
429	Trop de requêtes	tentatives de connexion, quota de l'assistant IA
500 / 503	Erreur serveur / service indisponible	exception inattendue / base de données injoignable

Le même mécanisme d'en-têtes sert à envoyer des fichiers. `ajax/download_report.php` envoie `Content-Type: application/pdf`, `Content-Disposition: attachment; filename="..."` (qui force le téléchargement), `X-Content-Type-Options: nosniff` et `Cache-Control: private, no-store`, puis fait `echo $pdf; exit;`.

2.15 Appels HTTP sortants avec cURL

Pour parler à CamPay (paiement) et à OpenRouter (IA), c'est le serveur PHP qui devient **client HTTP**. L'appel part du serveur et non du navigateur, pour que les clés d'API ne quittent jamais le serveur. L'extension **cURL** est encapsulée dans une seule fonction réutilisable :

```
function httpJsonRequest(string $method, string $url, array $headers = [], $body = null, int
$timeout = 30): array {
    $ch = curl_init($url);
    $httpHeaders = ['Accept: application/json'];
    foreach ($headers as $name => $value) {
        $httpHeaders[] = $name . ': ' . $value;
    }
    if ($body !== null) {
        $httpHeaders[] = 'Content-Type: application/json';
        curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode($body, JSON_UNESCAPED_UNICODE));
    }
    curl_setopt_array($ch, [
        CURLOPT_CUSTOMREQUEST => strtoupper($method),
        CURLOPT_HTTPHEADER => $httpHeaders,
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_CONNECTTIMEOUT => 10,
        CURLOPT_TIMEOUT => $timeout,
    ]);
    $raw = curl_exec($ch);
    // ... curl_error, curl_getinfo(CURLINFO_HTTP_CODE), curl_close ...
    if ($raw === false) {
        throw new RuntimeException('Requête HTTP impossible vers ' . parse_url($url, PHP_URL_HOST)
. ' : ' . $error);
    }
    return ['status' => $status, 'data' => json_decode($raw, true), 'raw' => (string)$raw];
}
```

`includes/http_client.php` (abrégé).

- `CURLOPT_RETURNTRANSFER` : `curl_exec` renvoie la réponse sous forme de chaîne au lieu de l'afficher.
- Les **délais** (10 s pour se connecter, 30 s au total) évitent qu'une page reste bloquée si CamPay ne répond pas.
- `json_decode($raw, true)` : le `true` donne des tableaux associatifs plutôt que des objets.
- Le message d'erreur ne contient que le nom d'hôte (`parse_url`), jamais l'URL complète ni les en-têtes, qui pourraient contenir un jeton.

Les clés (jeton CamPay, clé OpenRouter) sont lues par `appConfig()`, qui cherche d'abord dans les variables d'environnement puis dans `config/local.php`, fichier non versionné. Elles ne sont jamais écrites dans le code source.

2.16 Génération de PDF en PHP pur : `includes/pdf_lite.php`

Les rapports de fin d'intervention et les reçus de paiement sont produits par une classe écrite pour le projet, `PdfLite`, **sans bibliothèque externe**. Le docblock explique ce choix : « vendor/ n'est pas déployé ». Un PDF est au fond un fichier texte structuré. Il contient des **objets numérotés** (catalogue, arbre des pages, polices, pages, flux de dessin), une table `xref` qui donne la position exacte en octets de chaque objet, puis un trailer et la ligne `%%EOF`.

```

$pdf = "%PDF-1.4\n%\xE2\xE3\xCF\xD3\n";
$offsets = [];
foreach ($objects as $n => $body) {
    $offsets[$n] = strlen($pdf);
    $pdf .= $n . " 0 obj\n" . $body . "\nendobj\n";
}
$xref = strlen($pdf);
$size = count($objects) + 1;
// Chaque entrée fait exactement 20 octets (fin de ligne « espace + LF »).
$pdf .= "xref\n0 " . $size . "\n0000000000 65535 f\n";
for ($n = 1; $n < $size; $n++) $pdf .= sprintf('%010d 00000 n ', $offsets[$n]) . "\n";
$pdf .= "trailer\n<< /Size " . $size . " /Root 1 0 R /Info 5 0 R >>\nstartxref\n" . $xref .
"\n%%EOF\n";
return $pdf;

```

`includes/pdf_lite.php`, fin de `output()`.

Ce que la classe gère :

- pages A4 et saut de page automatique ;
- polices standard Helvetica et Helvetica-Bold, avec conversion du texte UTF-8 en Windows-1252 (`toWinAnsi()`) pour les accents ;
- mesure de la largeur du texte d'après les tables Adobe AFM, pour couper les lignes (`wrapText()`) ;
- rectangles et lignes de couleur ;
- pied de page « Page n / N » ;
- compression `gzcompress` (FlateDecode) quand `zlib` est disponible.

Côté utilisation, l'interface est simple :

```

$pdf = new PdfLite();
$pdf->setTitle('Rapport de fin d\'intervention n° ' . $id);
$pdf->setFooterText('Document généré le ' . date('d/m/Y à H:i') . ' par SmartAutoTrack');
$pdf->addPage();
$pdf->rect(0, 0, PdfLite::PAGE_WIDTH, 74, $blue);
$pdf->setTextColor(255, 255, 255);
$pdf->setFont(true, 17);
$pdf->text($left, 36, 'SmartAutoTrack – Rapport de fin d\'intervention');

```

`ajax/download_report.php`, `generateReportPDF()`. `includes/receipt.php` utilise la même classe pour les reçus.

C'est aussi le seul vrai exemple de **programmation orientée objet** écrite pour le projet, avec `Database`. On y trouve une classe `final` (qui ne peut pas être héritée), des propriétés `private`, des méthodes publiques et privées, des méthodes `static` et des constantes de classe.

2.17 Scripts en ligne de commande (CLI)

PHP ne sert pas qu'au web : `php fichier.php` exécute un script dans un terminal. Le projet en a deux familles :

- les **migrations** de schéma (`scripts/migrate_structure.php --apply`) ;
- l'envoi quotidien des **rappels d'échéance** (`scripts/send_reminders.php`), à programmer avec le Planificateur de tâches Windows ou `cron` sous Linux.

```

if (PHP_SAPI !== 'cli') {
    http_response_code(403);
    exit('Accès interdit');
}

require_once __DIR__ . '/../config/database.php';
require_once __DIR__ . '/../includes/maintenance.php';

$idClient = null;
foreach (array_slice($argv, 1) as $arg) {
    if (preg_match('/^--client=(\d+)/', $arg, $m)) {
        $idClient = (int)$m[1];
    } else {
        fwrite(STDERR, "Option inconnue : $arg\nUsage : php scripts/send_reminders.php [--client=ID]\n");
        exit(2);
    }
}

```

scripts/send_reminders.php .

- `PHP_SAPI` indique comment PHP a été lancé : `'cli'` en terminal, `'apache2handler'` sous Apache. Le test empêche qu'un visiteur déclenche l'envoi des rappels en tapant l'URL du script.
- `$argv` contient les arguments de la ligne de commande. `STDERR` reçoit les messages d'erreur, et `exit(0/1/2)` renvoie un **code de sortie** que le planificateur peut surveiller.
- Le script est **idempotent** : chaque rappel est noté dans la table `rappel_envoye`, donc le relancer ne crée pas de doublon.
- `Database::getConnection()` s'adapte au contexte : en CLI, elle renvoie `null` au lieu de répondre 503, et le script décide lui-même quoi faire.

À retenir

- PHP exécute le code sur le serveur et n'envoie que le résultat : HTML, JSON ou PDF.
- Sécurité en couches : session durcie et régénérée, rôle lu en session, requêtes préparées, jeton CSRF, `h()` à l'affichage, `password_hash`, `finfo` pour les fichiers, limitation des tentatives.
- Transactions + `FOR UPDATE` contre les doubles paiements et le dépassement de la limite de véhicules.
- `header('Location')` doit toujours être suivi de `exit`.
- `includes/` contient la logique métier partagée, et les pages restent fines.
- `cURL` pour CamPay et OpenRouter, `pdfLite` pour les PDF, CLI pour les tâches planifiées.

2.18 Questions du jury

Q. Pourquoi avoir choisi PHP plutôt que Node.js ou Python ?

PHP est livré avec XAMPP et s'intègre directement à Apache et MySQL, sans serveur applicatif à part. Il fournit en natif tout ce dont l'application a besoin : sessions, PDO, envoi de fichiers, `cURL`, `zlib`. C'est aussi le langage le plus répandu chez les hébergeurs mutualisés, donc le déploiement au Cameroun est simple et peu coûteux. Le projet utilise PHP 8.2 avec typage des paramètres et fonctions fléchées : c'est du PHP moderne.

Q. Comment empêchez-vous l'injection SQL ?

Toutes les requêtes qui reçoivent une donnée extérieure passent par des requêtes préparées PDO : `prepare()` avec des `?`, puis `execute([...])`. Les valeurs sont transmises séparément du texte SQL et ne sont donc jamais interprétées comme du code. J'ai désactivé l'émulation (`ATTR_EMULATE_PREPARES => false`) pour que ce soit MySQL lui-même qui prépare. Pour les listes `IN (...)`, je génère autant de `?` que de valeurs.

Q. Un client peut-il voir ou modifier les données d'un autre client en changeant l'id dans l'URL ?

Non. Le rôle et l'identifiant viennent de la session, côté serveur, jamais de la requête. Chaque requête sensible filtre sur le propriétaire, par exemple `WHERE idVehicule = ? AND idClient = ?` avec `$_SESSION['user_id']`. Pour les téléchargements de rapports, une réparation hors du périmètre répond 404, comme si elle n'existait pas, pour ne pas révéler son existence.

Q. Comment sont stockés les mots de passe ?

Avec `password_hash(..., PASSWORD_DEFAULT)`, donc `bcrypt` : un hachage à sens unique, salé et volontairement lent. À la connexion, `password_verify` recalcule et compare. Même avec un accès à la base, on ne peut pas lire les mots de passe. En plus, les tentatives sont limitées par email et par IP (réponse 429), et le message d'erreur reste générique.

Q. À quoi sert `session_regenerate_id(true)` dans `login.php` ?

Il protège contre la fixation de session. Sans lui, un attaquant qui aurait imposé un identifiant de session à la victime avant sa connexion pourrait ensuite réutiliser cet identifiant, désormais authentifié. En changeant l'identifiant au moment de la connexion, celui que l'attaquant connaît devient inutile ; `true` supprime en plus l'ancien fichier de session. Le cookie est aussi `httponly` et `SameSite=Lax`.

Q. Qu'est-ce que le CSRF et comment vous en protégez-vous ?

C'est le fait de faire exécuter une action à un utilisateur connecté depuis un site tiers, en profitant de son cookie de session. Chaque session reçoit un jeton aléatoire (`random_bytes(32)`). Il est mis dans les formulaires en champ caché, ou dans l'en-tête `x-CSRF-Token` pour l'AJAX, puis vérifié avec `hash_equals`. Un site tiers ne connaît pas ce jeton. Une limite à reconnaître : la suppression d'un véhicule passe encore par un lien GET sans jeton, ce que je corrigerais en la passant en POST.

Q. Pourquoi une transaction dans le paiement ?

Pour éviter qu'un double clic ou deux onglets créent deux paiements pour la même réparation. `paymentStart()` ouvre une transaction et verrouille la réparation avec `SELECT ... FOR UPDATE`. Ensuite, elle vérifie qu'aucun paiement n'est en cours et insère le paiement `EN_ATTENTE` avant de valider avec `commit()`. La seconde requête attend la fin de la première et voit alors le paiement existant. En cas d'erreur, `rollback()` annule tout.

Q. Pourquoi avoir écrit votre propre générateur PDF au lieu d'utiliser FPDF ou Dompdf ?

Le dossier `vendor/` de Composer n'est pas déployé sur le serveur cible. Un rapport et un reçu ne demandent que du texte, des rectangles et des lignes. `PdfLite` fait environ 570 lignes de PHP pur : il construit les objets PDF, la table xref aux offsets exacts et compresse les flux avec zlib. C'est aussi l'occasion de montrer que je comprends la structure interne d'un PDF. La limite : pas d'images ni de polices incorporées, et les caractères hors Windows-1252 deviennent « ? ».

3. JavaScript : l'interactivité dans le navigateur

PHP fabrique la page sur le serveur, puis l'envoie au navigateur. À partir de ce moment, PHP ne fait plus rien : si l'on veut qu'un menu s'ouvre, qu'un compteur de notifications se mette à jour ou qu'un paiement Mobile Money soit suivi sans recharger la page, il faut un langage qui s'exécute **dans le navigateur de l'utilisateur**. Ce langage, c'est JavaScript (JS). Ce chapitre explique comment SmartAutoTrack l'utilise, fichier par fichier, avec de vrais extraits du code.

3.1 Le rôle du JavaScript côté client

Ce que c'est. JavaScript est un langage interprété par le navigateur (Chrome, Firefox, le navigateur du téléphone...). Il a accès à la page affichée (le DOM), aux actions de l'utilisateur (clics, frappes au clavier) et peut envoyer des requêtes HTTP au serveur en arrière-plan.

Pourquoi on l'utilise ici. Dans SmartAutoTrack, le JS sert à quatre choses :

- **le confort de saisie** : filtres de caractères (`data-only`), règles de mot de passe cochées en direct ;
- **l'ergonomie** : menu mobile en tiroir, menus déroulants, fenêtres modales, petits messages éphémères (« toasts ») ;
- **les échanges sans rechargement (AJAX)** : notifications, détails d'une réparation, assistant IA, enregistrement du thème ;
- **le suivi en temps (presque) réel** : interrogation périodique du statut d'un paiement CamPay et des notifications.

Comment c'est organisé. Le code JS commun est rassemblé dans trois fichiers du dossier `assets/js/` :

Fichier	Rôle	Chargé où ?
<code>main.js</code> (~1000 lignes)	Jeton CSRF des requêtes AJAX, objet global <code>SmartAutoTrack</code> (utilitaires, notifications, modale, toasts, formulaires, tableaux), menu mobile, règles de mot de passe, filtres <code>data-only</code> , fenêtre « Voir le rapport »	Toutes les pages, via <code>includes/header.php</code>
<code>themes.js</code>	Choix du thème de couleurs et enregistrement côté serveur (avec <code>fetch</code>)	Toutes les pages
<code>assistant.js</code>	Conversation avec l'assistant IA (<code>ajax/assistant_chat.php</code>)	Seulement <code>admin/assistant.php</code> et <code>client/assistant.php</code>

Certaines pages ont en plus un petit script qui leur est propre, écrit dans une balise `<script>` en bas de page (par exemple `client/reparations.php` pour le paiement, `garage/demandes.php` pour la fenêtre « Affecter la demande »).

L'ordre de chargement est fixé dans `includes/header.php` : jQuery d'abord, puis `main.js` qui en dépend.

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
<script src="<?php echo h(asset_url('assets/js/main.js')); ?>"></script>
...
<meta name="csrf-token" content="<?php echo generateCSRFToken(); ?>">
```

Le pied de page `includes/footer.php` transmet ensuite au JavaScript les informations dont il a besoin, en les « imprimant » depuis PHP :

```
<script>
  // Configuration globale
  const SITE_URL = '<?php echo SITE_URL; ?>';
  const USER_ROLE = '<?php echo h($userRole); ?>';
  const USER_ID = '<?php echo h($isLoggedIn ? $_SESSION['user_id'] : 'null'); ?>';
</script>
```

C'est le pont entre les deux mondes : PHP connaît la session, JavaScript la lit sous forme de constantes. Comme ces constantes sont déclarées après `main.js`, celui-ci ne les utilise qu'une fois la page entièrement chargée (dans `$(document).ready(...)`), comme l'explique son en-tête.

À retenir

- PHP s'exécute sur le serveur **avant** l'affichage ; JavaScript s'exécute dans le navigateur **après**.
- Le JS commun est dans `assets/js/main.js`, chargé sur toutes les pages juste après jQuery.
- PHP transmet au JS l'adresse du site et l'identifiant de l'utilisateur (`SITE_URL`, `USER_ID`) et le jeton CSRF (balise `<meta>`).

3.2 JavaScript natif et jQuery 3.7.1

Ce que c'est. Le JavaScript natif (on dit aussi « vanilla JS ») est le langage tel que le navigateur le fournit : `document.querySelector`, `addEventListener`, `fetch` ... **jQuery** est une bibliothèque, c'est-à-dire un fichier JS écrit par d'autres, qui propose des raccourcis plus courts pour les mêmes opérations. Elle est chargée depuis le CDN officiel `code.jquery.com`, en version 3.7.1.

Pourquoi jQuery ici. Le projet est parti d'un modèle qui utilisait déjà jQuery ; il a été conservé parce qu'il simplifie trois choses très présentes dans l'application :

- sélectionner et modifier des éléments en une ligne (`$('#messageCounter').text(5)`) ;
- les animations simples (`fadeIn()` / `fadeOut()` des modales) ;
- les requêtes AJAX avec `$.ajax`, et surtout la possibilité d'ajouter **un réglage commun à toutes les requêtes** (`$.ajaxSetup`) et **un traitement d'erreur commun** (`$(document).ajaxError`).

Le code plus récent (menu mobile, mots de passe, filtres `data-only`, fenêtre du rapport, affichage de l'assistant IA) est écrit en JS natif ; l'assistant garde toutefois `$.ajax` pour ses requêtes. Les navigateurs actuels offrent maintenant presque tout ce que jQuery apportait. Les deux styles cohabitent sans problème, car jQuery n'est qu'une couche au-dessus du JS natif.

Les quatre outils jQuery à savoir expliquer

jQuery	Équivalent natif	Sens
<code>\$('#notificationList')</code>	<code>document.querySelector('#notificationList')</code>	Sélectionne un élément de la page avec un sélecteur CSS
<code>\$(el).on('click', fn)</code>	<code>el.addEventListener('click', fn)</code>	Exécute <code>fn</code> à chaque clic
<code>\$.ajax({url, method, data, success})</code>	<code>fetch(url, {method, body}).then(...)</code>	Envoie une requête HTTP sans recharger la page
<code>\$(document).ready(fn)</code> ou <code>\$(fn)</code>	<code>document.addEventListener('DOMContentLoaded', fn)</code>	Attend que la page soit construite

Exemple réel : le point d'entrée de `main.js`, lancé dès que la page est prête.

```
// Initialisation au chargement de la page
$(document).ready(function() {
  SmartAutoTrack.init();
});
```

Et un exemple de `.on` avec jQuery : le bouton du menu mobile historique (`main.js`, fonction `initGlobalEvents`).

```
$('.navbar-toggle').on('click', function() {
  const $btn = $(this);
  const $menu = $('#navbarMenu');
  const isOpen = $menu.hasClass('open');
  $menu.toggleClass('open', !isOpen);
  $btn.attr('aria-expanded', String(!isOpen));
});
```

Convention de nommage : une variable qui contient un objet jQuery commence par `$` (`$btn`, `$menu`). C'est une simple habitude de lecture, pas une obligation du langage.

Un espace de noms unique : `window.SmartAutoTrack`

Plutôt que de déclarer des dizaines de fonctions globales, `main.js` regroupe tout dans un seul objet :

```
// Espace de noms unique : évite de multiplier les fonctions globales.
window.SmartAutoTrack = {
  utils: { formatDate: ..., escapeHtml: ..., validateEmail: ... },
  notifications: { load: ..., updateCounter: ..., display: ..., markAsRead: ... },
  messages: { loadCount: ... },
  modal: { show: ..., hide: ... },
  toast: { show: ..., getIcon: ... },
  forms: { validate: ..., reset: ... },
  tables: { init: ... },
  init: function() { ... }
};
```

(Extrait résumé : chaque `...` est une vraie fonction dans le fichier.) On appelle ensuite `SmartAutoTrack.toast.show('Paieement reçu', 'success')`. Avantage : un seul nom global, donc pas de risque qu'une autre bibliothèque écrase une fonction appelée par exemple `load`.

Piège fréquent

Dire « jQuery est un autre langage ». Non : jQuery est une **bibliothèque écrite en JavaScript**. `$` n'est qu'un nom de fonction. Tout ce que fait jQuery peut être fait en JS natif, simplement avec plus de lignes.

3.3 Le DOM : lire et modifier la page

Ce que c'est. Quand le navigateur reçoit le HTML, il le transforme en un arbre d'objets en mémoire : le **DOM** (Document Object Model). Chaque balise devient un « nœud » que JavaScript peut lire, modifier, créer ou supprimer. Modifier le DOM, c'est modifier ce que l'utilisateur voit, sans recharger la page.

Les opérations de base

- `document.querySelector(sel)` : renvoie le premier élément correspondant au sélecteur CSS (ou `null`) ;

- `document.querySelectorAll(sel)` : renvoie tous les éléments, qu'on parcourt avec `forEach` ;
- `document.createElement('div')` puis `appendChild` : crée et insère un nouvel élément ;
- `el.classList.add / remove / toggle / contains` : manipule les classes CSS (c'est le CSS qui décide de l'apparence, le JS ne fait qu'ajouter ou retirer une classe) ;
- `el.setAttribute('aria-expanded', 'true')` : modifie un attribut.

Exemple réel : la première fonction de `main.js` lit le jeton CSRF dans la balise `<meta>`.

```
function smartautotrackCsrfToken() {
  const meta = document.querySelector('meta[name="csrf-token"]');
  return meta ? meta.getAttribute('content') : '';
}
```

Exemple réel avec `classList` : l'ouverture du tiroir de navigation des tableaux de bord sur mobile (`main.js`).

Le JS n'anime rien lui-même : il pose la classe `is-open`, et la feuille CSS fait glisser la barre latérale.

```
/** Ouvre le tiroir de navigation (classe nav-open : bloque le défilement du fond). */
function openNav() {
  sidebar.classList.add('is-open');
  backdrop.classList.add('is-open');
  document.body.classList.add('nav-open');
  toggle.setAttribute('aria-expanded', 'true');
}

toggle.addEventListener('click', function () {
  if (sidebar.classList.contains('is-open')) closeNav(); else openNav();
});
```

Ce script est intéressant à présenter : il **détecte** lequel des quatre gabarits est présent (`v2` client, `av2` admin, `gv2` garage, `tv2` technicien), puis **crée** lui-même le bouton « hamburger » et le rideau sombre avec `createElement`. Ainsi, une seule fonction sert les quelque 55 pages des tableaux de bord sans les modifier une par une.

`textContent` contre `innerHTML` : la question de sécurité XSS

Il existe deux façons de mettre du texte dans un élément :

- `el.textContent = valeur` : la valeur est affichée **comme du texte**. Si elle contient `<script>`, l'utilisateur voit littéralement « `<script>` », rien ne s'exécute ;
- `el.innerHTML = valeur` (ou `.html()` en jQuery) : la valeur est **interprétée comme du HTML**. Si elle contient une balise piégée, le navigateur l'exécute.

Une attaque **XSS** (Cross-Site Scripting) consiste justement à faire enregistrer dans la base un texte contenant du code (par exemple dans le titre d'une anomalie), pour qu'il s'exécute chez les autres utilisateurs qui l'affichent. Le code malveillant pourrait alors agir avec la session de la victime.

La fenêtre « Voir le rapport » des journaux d'activité applique la règle la plus sûre : **jamais d'innerHTML**. Sa petite fonction utilitaire pose tout le texte avec `textContent` (`main.js`) :

```

/**
 * Crée un élément avec une classe et, éventuellement, un texte.
 * @param {string} [text] posé avec textContent.
 */
function el(tag, className, text) {
    const node = document.createElement(tag);
    if (className) node.className = className;
    if (text !== undefined) node.textContent = text;
    return node;
}

```

Quand on a vraiment besoin de construire du HTML (une liste de notifications avec icônes, par exemple), on **échappe** d'abord les données venant de la base. C'est le rôle de `SmartAutoTrack.utils.escapeHtml`, qui remplace les caractères dangereux par leurs entités HTML :

```

escapeHtml: function(value) {
    if (value === null || value === undefined) return '';
    return String(value)
        .replace(/&(?!(?:[a-zA-Z][a-zA-Z0-9]*|[0-9]+|#[xX][0-9a-fA-F]+)/g, '&amp;')
        .replace(/</g, '&lt;')
        .replace(/>/g, '&gt;')
        .replace(/"/g, '&quot;')
        .replace(/'/g, '&#039;');
},

```

Utilisation dans l'affichage des notifications (`SmartAutoTrack.notifications.display`) :

```

<div class="notification-title">${SmartAutoTrack.utils.escapeHtml(notification.titre)}</div>
<div class="notification-message">${SmartAutoTrack.utils.escapeHtml(notification.message)}</div>

```

La première expression régulière est subtile : elle n'échappe un `&` que s'il ne fait pas déjà partie d'une entité (`&`, `'` ...). En effet, côté PHP, les saisies passent par `sanitize()` et sont déjà encodées en base ; sans cette précaution, une apostrophe s'afficherait « `'` ».

Cette règle doit être appliquée partout, et un oubli reste dans le code : dans `client/reparations.php`, la fonction `displayReparationDetails()` échappe le statut, la description ou le véhicule, mais insère `reparation.titre` tel quel. Le risque est limité, car le titre a été encodé par `sanitize()` à l'enregistrement, mais il faut savoir le reconnaître et proposer la correction :

```

${SmartAutoTrack.utils.escapeHtml(reparation.titre)}.

```

L'assistant IA (`assets/js/assistant.js`) combine les deux approches : le message de l'utilisateur est posé avec `textContent` ; la réponse de l'IA est d'abord entièrement échappée (par une petite fonction `escapeHtml` propre à ce fichier, distincte de celle de `main.js`), puis seul le gras Markdown (`**texte**`) est transformé en `<strong>`. Ainsi, même si le modèle renvoyait du HTML, il ne serait jamais exécuté.

```

function setText(el, text, formatted) {
    if (!formatted) {
        el.textContent = text;
        return;
    }
    el.innerHTML = escapeHtml(text)
        .replace(/\*\*(.+?)\*/g, '<strong>$1</strong>')
        .replace(/^#{1,4}\s*(.+)$/gm, '<strong>$1</strong>');
}

```

À retenir

- Le DOM est la représentation de la page en mémoire ; le JS le modifie pour changer l'affichage.
- `classList` : le JS change une classe, le CSS décide de l'apparence.
- `textContent` = texte sûr ; `innerHTML` = HTML interprété, à n'utiliser qu'avec des données échappées.
- Double protection XSS : `h()` côté PHP à l'affichage, `escapeHtml()` / `textContent` côté JS.

3.4 Les événements et la délégation

Ce que c'est. Un événement est un signal émis par le navigateur : `click`, `input` (le contenu d'un champ change), `keydown` (touche enfoncée), `submit` (formulaire envoyé), `DOMContentLoaded` (page construite), `resize` (fenêtre redimensionnée)... On y attache une fonction, appelée « gestionnaire » ou `listener`, avec `addEventListener`. La fonction reçoit un objet `e` (l'événement) qui indique notamment `e.target` (l'élément concerné) et `e.key` (la touche).

Deux méthodes de cet objet reviennent sans cesse dans le projet :

- `e.preventDefault()` : annule le comportement normal (ne pas envoyer le formulaire, ne pas suivre le lien, ne pas taper la lettre) ;
- `e.stopPropagation()` : empêche l'événement de « remonter » vers les éléments parents.

La propagation (le « bouillonnement »)

Quand on clique sur un bouton, l'événement `click` est d'abord reçu par le bouton, puis par son parent, puis par le parent du parent... jusqu'à `document`. C'est ce qu'on appelle le bubbling. Les menus déroulants de `main.js` l'utilisent : un clic n'importe où sur la page (donc sur `document`) ferme les menus, sauf si le clic vient du bouton du menu, qui arrête la remontée :

```
// Gestion des dropdowns
$('.dropdown-toggle').click(function(e) {
  e.stopPropagation();
  $(this).next('.dropdown-menu').toggle();
});

// Fermer les dropdowns en cliquant ailleurs
$(document).click(function() {
  $('.dropdown-menu').hide();
});
```

La délégation d'événements

Le problème. `$('.btn').click(fn)` n'attache le gestionnaire qu'aux boutons présents au moment de l'appel. Un bouton créé plus tard (dans une modale, une liste chargée en AJAX) ne réagira pas.

La solution. Grâce à la propagation, on place **un seul** gestionnaire sur `document`, et l'on regarde dans `e.target` d'où vient l'événement. C'est la délégation. Exemple réel : la fenêtre « Voir le rapport » (`main.js`, dernières lignes) :

```
// Délégation : couvre tous les boutons de la page, même ajoutés plus tard.
document.addEventListener('click', function (e) {
    const trigger = e.target && e.target.closest ? e.target.closest('.report-modal-trigger') :
    null;
    if (!trigger) return;
    e.preventDefault();
    show(trigger);
});
```

`closest()` remonte de l'élément cliqué vers ses parents jusqu'à trouver un `.report-modal-trigger`. C'est nécessaire si l'utilisateur clique sur une icône à l'intérieur du bouton.

La version jQuery de la délégation s'écrit avec un deuxième argument sélecteur dans `.on` :

```
$(document).on('input', 'input[name="telephone"]', function () {
    smartautotrackFilterInput(this, /^[^0-9]/g);
});
```

Les données dans le HTML : attributs `data-*`

Le PHP prépare les informations dont le JS aura besoin dans des attributs `data-*`. Les boutons du journal d'activité portent tout le contenu du rapport, en JSON, dans `data-report` (`client/journal.php`) :

```
<button type="button" class="report-modal-trigger"
    data-report="<?php echo h(activity_log_report_json($j)); ?>">Voir le rapport</button>
```

Le JS le relit avec `JSON.parse(trigger.getAttribute('data-report'))`, dans un `try / catch` pour ne pas planter si le JSON était mal formé. Aucun aller-retour avec le serveur n'est nécessaire pour ouvrir la fenêtre.

Piège fréquent

Attacher un gestionnaire à un élément qui n'existe pas encore. Si un bouton est ajouté par JavaScript après le chargement, il faut soit attacher le gestionnaire après sa création, soit utiliser la délégation sur `document`. C'est pour cela que les filtres `data-only` et la fenêtre du rapport utilisent `document.addEventListener`.

3.5 AJAX : dialoguer avec le serveur sans recharger la page

Ce que c'est. AJAX (Asynchronous JavaScript And XML) désigne le fait d'envoyer une requête HTTP depuis le JS, en arrière-plan, puis de mettre à jour une partie de la page avec la réponse. Aujourd'hui, la réponse n'est presque jamais du XML mais du **JSON** (JavaScript Object Notation), un format texte léger que PHP produit avec `json_encode()` et que JS lit directement comme un objet.

« Asynchrone » signifie que le navigateur ne se bloque pas pendant l'attente : la page reste utilisable, et une fonction de rappel (callback), ici `success` ou `error`, est appelée quand la réponse arrive.

Le côté serveur : le dossier `ajax/`

Chaque action AJAX a son propre fichier PHP dans `ajax/` (23 fichiers : `notifications.php`, `get_reparation_details.php`, `campay_collect.php`, `campay_status.php`, `assistant_chat.php`, `save_theme.php` ...). Ces fichiers ne renvoient pas une page HTML mais du JSON. Exemple : le début de `ajax/campay_status.php`.

```
header('Content-Type: application/json');

requireJsonAuth(ROLE_CLIENT);

if ($_SERVER['REQUEST_METHOD'] !== 'POST' || !verifyRequestCSRF()) {
    http_response_code(403);
    exit(json_encode(['success' => false, 'message' => 'Session expirée, merci de recharger la
page.']));
}

$paieementId = filter_var($_POST['paieement_id'] ?? null, FILTER_VALIDATE_INT);
```

On remarque : l'en-tête `Content-Type: application/json`, le contrôle du rôle (seul un client connecté peut demander), la vérification du jeton CSRF, la validation de l'identifiant reçu, et des **codes HTTP** parlants (403 interdit, 400 requête invalide, 404 introuvable).

Le côté navigateur : `$.ajax`

Exemple réel : le chargement des notifications (`main.js`, `SmartAutoTrack.notifications.load`).

```
$.ajax({
    url: SITE_URL + 'ajax/notifications.php',
    method: 'GET',
    dataType: 'json',
    success: function(response) {
        if (response.success) {
            SmartAutoTrack.notifications.updateCounter(response.unread_count);
            SmartAutoTrack.notifications.display(response.notifications);
        }
    },
    error: function() {
        console.error('Erreur lors du chargement des notifications');
    }
});
```

`dataType: 'json'` demande à jQuery de transformer automatiquement le texte reçu en objet JS : on écrit ensuite simplement `response.unread_count`.

L'en-tête `X-CSRF-Token` ajouté automatiquement

Rappel CSRF. Une attaque CSRF (Cross-Site Request Forgery) consiste à piéger un utilisateur connecté pour que son navigateur envoie, à son insu, une requête vers notre site (par exemple « marquer comme payé »). Le navigateur joint automatiquement le cookie de session, donc le serveur croirait que la demande est légitime. La parade : un **jeton secret** généré par le serveur, stocké en session et que seule une vraie page du site connaît.

Plutôt que d'ajouter ce jeton à la main dans chacun des dizaines d'appels `$.ajax`, `main.js` le fait **une fois pour toutes** grâce à `jQuery.ajaxSetup` :

```
// Toute requête jQuery vers le site porte l'en-tête X-CSRF-Token, vérifié
// côté serveur (config/config.php). Les requêtes vers un autre domaine ne le
// reçoivent pas, pour ne jamais divulguer le jeton.
if (window.jQuery) {
    jQuery.ajaxSetup({
        beforeSend: function(xhr, settings) {
            if (!settings.crossDomain) {
                xhr.setRequestHeader('X-CSRF-Token', smartautotrackCsrfToken());
            }
        }
    });
}
```

Côté serveur, `config/config.php` accepte le jeton soit dans un champ de formulaire, soit dans cet en-tête (PHP transforme l'en-tête `X-CSRF-Token` en `$_SERVER['HTTP_X_CSRF_TOKEN']`), et le compare en temps constant avec `hash_equals` :

```
function verifyCSRFToken($token) {
    return isset($_SESSION['csrf_token']) && is_string($token) &&
    hash_equals($_SESSION['csrf_token'], $token);
}

function verifyRequestCSRF() {
    return verifyCSRFToken($_POST['csrf_token'] ?? ($_SERVER['HTTP_X_CSRF_TOKEN'] ?? ''));
}
```

La gestion d'erreurs commune

De la même façon, `main.js` centralise la réaction aux erreurs HTTP de toutes les requêtes jQuery :

```
$(document).ajaxError(function(event, xhr, settings, thrownError) {
    if (xhr.status === 401) {
        showToast('Session expirée. Veuillez vous reconnecter.', 'error');
        setTimeout(function() {
            window.location.href = SITE_URL + 'auth/login.php';
        }, 2000);
    } else if (xhr.status === 403) {
        showToast('Accès refusé.', 'error');
    } else if (xhr.status >= 500) {
        showToast('Erreur serveur. Veuillez réessayer plus tard.', 'error');
    }
});
```

Le détail technique de l'erreur n'est jamais montré à l'utilisateur : il reste dans les journaux du serveur (`error_log`).

fetch et les promesses

`fetch` est la fonction AJAX **native** des navigateurs modernes. Elle renvoie une **promesse** (Promise) : un objet qui représente un résultat « à venir ». On y enchaîne `.then(...)` pour le succès et `.catch(...)` pour l'échec. Le projet l'utilise une fois, dans `assets/js/themes.js`, pour enregistrer le thème choisi :

```

fetch((typeof SITE_URL !== 'undefined' ? SITE_URL : '') + 'ajax/save_theme.php', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/x-www-form-urlencoded',
    'X-CSRF-Token': typeof smartautotrackCsrfToken === 'function' ? smartautotrackCsrfToken()
: '',
  },
  body: `theme=${encodeURIComponent(theme)}`
}).catch(error => {
  console.log('Impossible de sauvegarder le thème sur le serveur:', error);
});

```

Point important à expliquer : `fetch` ne passe **pas** par jQuery, donc `ajaxSetup` ne s'applique pas. C'est pourquoi le jeton CSRF est ajouté à la main dans `headers`, en réutilisant la fonction `smartautotrackCsrfToken()` de `main.js`. `encodeURIComponent` protège la valeur envoyée (espaces, &...).

Les objets renvoyés par `$.ajax` se comportent aussi comme des promesses : l'assistant IA utilise `.always(...)` pour vider l'affichage de la conversation, que la requête de réinitialisation réussisse ou non. Le projet n'utilise pas la syntaxe `async / await`.

À retenir

- AJAX = requête en arrière-plan + mise à jour partielle de la page ; la réponse est en JSON.
- Un fichier `ajax/*.php` par action : il vérifie le rôle, le jeton CSRF et les données, puis renvoie du JSON avec un code HTTP adapté.
- `$.ajaxSetup` ajoute `X-CSRF-Token` à toutes les requêtes jQuery vers le site ; `fetch` doit l'ajouter lui-même.
- `$(document).ajaxError` gère en un seul endroit les erreurs 401, 403 et 500.

3.6 L'interrogation périodique (polling)

Ce que c'est. HTTP fonctionne en « question / réponse » : le serveur ne peut pas, de lui-même, prévenir le navigateur qu'une chose a changé. Pour suivre une information qui évolue, le plus simple est que le navigateur **redemande régulièrement** : c'est le polling. Deux fonctions natives servent à programmer du code dans le temps :

- `setTimeout(fn, ms)` : exécute `fn` **une fois** après `ms` millisecondes ;
- `setInterval(fn, ms)` : exécute `fn` **en boucle** toutes les `ms` millisecondes.

Les notifications : toutes les 30 secondes

```

init: function() {
  if (typeof USER_ID !== 'undefined' && USER_ID) {
    SmartAutoTrack.notifications.load();
    SmartAutoTrack.messages.loadCount();

    // Actualiser toutes les 30 secondes
    setInterval(function() {
      SmartAutoTrack.notifications.load();
      SmartAutoTrack.messages.loadCount();
    }, 30000);
  }
  ...
}

```

La cloche des notifications et le compteur de messages se mettent ainsi à jour sans que l'utilisateur recharge la page. 30 secondes est un compromis : assez fréquent pour paraître réactif, assez rare pour ne pas surcharger le serveur.

Point honnête à connaître : `includes/footer.php`, inclus par les mêmes pages, contient une version plus ancienne de cette boucle (`loadNotifications()` et `loadMessageCount()` dans son propre `setInterval` de 30 s). Les deux boucles tournent donc en même temps, et chaque page envoie deux séries de requêtes toutes les 30 secondes. Le résultat affiché reste juste, mais le serveur travaille deux fois. L'amélioration évidente consiste à supprimer l'une des deux boucles.

Le paiement CamPay : suivi toutes les 5 secondes

Le paiement Mobile Money se déroule en deux temps, et c'est le meilleur exemple d'AJAX du projet (`client/reparations.php`) :

1. le client saisit son numéro dans la fenêtre de paiement ; un appel à `ajax/campay_collect.php` demande à CamPay d'envoyer une demande de validation sur son téléphone ;
2. le client valide sur son téléphone (code secret Mobile Money) : cela peut prendre quelques secondes ou quelques minutes. Pendant ce temps, la page interroge `ajax/campay_status.php` toutes les 5 secondes.

```
function pollPayment(paiementId, attempt) {
    if (attempt >= 36) {
        $('#paymentWaitText').text('Paiement toujours en attente. Si vous l\'avez validé, il apparaîtra comme payé d\'ici quelques minutes.');
```

```
        return;
    }
    paymentPollTimer = setTimeout(function() {
        $.ajax({
            url: SITE_URL + 'ajax/campay_status.php',
            method: 'POST',
            dataType: 'json',
            data: { paiement_id: paiementId },
            success: function(response) {
                if (response.statut === 'PAYE') { /* reçu proposé, étape « Terminé » */ }
                else if (response.statut === 'ECHOUE') { /* message d'échec, retour au formulaire
*/ }

                else { pollPayment(paiementId, attempt + 1); }
            },
            error: function() { pollPayment(paiementId, attempt + 1); }
        });
    }, 5000);
}
```

(Extrait légèrement condensé dans les deux premières branches.) Trois choix à savoir justifier :

- **setTimeout récursif plutôt que setInterval** : la requête suivante n'est programmée qu'après la réponse de la précédente. Si le réseau est lent, les requêtes ne s'empilent jamais.
- **Une limite de 36 essais** : $36 \times 5 \text{ s} = 3 \text{ minutes}$. Au-delà, on arrête d'interroger et on rassure l'utilisateur ; le paiement sera de toute façon mis à jour plus tard, car CamPay prévient aussi le serveur directement par un webhook (`webhooks/campay.php`).
- **Arrêt propre** : fermer la fenêtre appelle `clearTimeout(paymentPollTimer)`, ce qui annule l'interrogation en cours.

Côté serveur, `campay_status.php` vérifie que le paiement **appartient bien au client connecté** (sinon 404) ; un client ne peut donc pas espionner le paiement d'un autre en changeant l'identifiant. Et en cas d'erreur

passagère de CamPay, il répond `EN_ATTENTE` pour que la page réessaie au tour suivant.

Piège fréquent

Croire que le navigateur « décide » que le paiement est réussi. Non : le JS ne fait qu'**afficher** le statut que le serveur lui renvoie. C'est le serveur qui interroge CamPay (`paymentRefresh()`) et met à jour la table `paiement`. Si quelqu'un trafiquait le JS pour afficher « Paiement reçu », rien ne changerait en base.

3.7 Les fenêtres modales

Ce que c'est. Une modale est une fenêtre qui s'affiche au-dessus de la page, sur un fond assombri, et qui demande l'attention de l'utilisateur avant de revenir à la page. Techniquement, c'est un bloc HTML caché, rendu visible par JavaScript (classe CSS, attribut `hidden` ou `fadeIn()`).

L'application en utilise trois sortes, de complexité croissante :

Modale	Fichier	Technique
Affecter la demande (garage)	<code>garage/demandes.php</code>	HTML déjà présent, JS natif, classe <code>show</code>
Paiement d'une réparation	<code>client/reparations.php</code>	jQuery <code>fadeIn/fadeOut</code> , trois étapes (formulaire, attente, terminé)
Rapport de fin d'intervention	<code>assets/js/main.js</code>	Construite entièrement par JS, accessible au clavier

La modale « Affecter la demande »

Le formulaire est écrit en HTML par PHP ; le JS se contente de remplir l'identifiant caché et d'ajouter la classe `show` :

```
window.openAssign = function (id, label) {
    document.getElementById('assignInterventionId').value = id;
    document.getElementById('assignModalSub').textContent = label;
    assignOverlay.classList.add('show');
};
document.getElementById('closeAssignModal').addEventListener('click', function () {
    assignOverlay.classList.remove('show');
});
assignOverlay.addEventListener('click', function (e) { if (e.target === assignOverlay)
    assignOverlay.classList.remove('show'); });

<?php if (!empty($formErrors)): ?>
assignOverlay.classList.add('show');
<?php endif; ?>
```

Deux détails : le test `e.target === assignOverlay` ferme la fenêtre seulement si l'on clique sur le fond, pas à l'intérieur ; et le bloc PHP final rouvre la fenêtre automatiquement si le serveur a refusé le formulaire, pour que le garagiste voie les erreurs. Le formulaire, lui, est envoyé normalement (pas en AJAX) et c'est PHP qui vérifie que la demande appartient bien au garage.

La modale « Rapport de fin d'intervention »

C'est la plus aboutie. Elle est construite une seule fois, au premier clic, et respecte les règles d'accessibilité : `role="dialog"`, `aria-modal="true"`, fermeture par la touche Échap, et **piège du focus** (la touche Tab reste à

l'intérieur de la fenêtre). À la fermeture, le focus revient sur le bouton qui l'a ouverte, ce qui est indispensable pour une personne qui navigue au clavier ou avec un lecteur d'écran.

```
/** Ferme la fenêtre et rend le focus au bouton qui l'a ouverte. */
function hide() {
  if (!overlay || overlay.hidden) return;
  overlay.hidden = true;
  document.documentElement.classList.remove('report-modal-open');
  if (opener && document.contains(opener)) opener.focus();
  opener = null;
}
```

Sécurité : les liens « Télécharger en PDF » et « Payer cette réparation » ne sont affichés que si leur adresse pointe vers **notre propre site** (fonction `setSameSiteLink`, qui compare `new URL(url).origin` à `window.location.origin`). Une donnée trafiquée ne peut donc pas transformer ce bouton en lien vers un site externe.

3.8 Aides à la saisie : mots de passe et filtres `data-only`

Règles de mot de passe en direct (`data-password-policy`)

Sur l'inscription, la réinitialisation et la création de comptes par l'admin, le champ porte l'attribut `data-password-policy` (`auth/register.php`) :

```
<input type="password" name="mot_de_passe" class="form-control" required
      data-password-policy autocomplete="new-password" placeholder="...">
```

`main.js` trouve tous ces champs, insère sous chacun une liste des cinq règles, et coche chaque règle respectée à chaque frappe (`input`) :

```
const rules = [
  { label: '8 caractères minimum', test: function (v) { return v.length >= 8; } },
  { label: 'Une lettre minuscule', test: function (v) { return /[a-z]/.test(v); } },
  { label: 'Une lettre majuscule', test: function (v) { return /[A-Z]/.test(v); } },
  { label: 'Un chiffre', test: function (v) { return /[0-9]/.test(v); } },
  { label: 'Un caractère spécial (ex. ! @ # $ %)', test: function (v) { return /^[A-Za-z0-9]/.test(v); } }
];
...
function update() {
  ...
  rules.forEach(function (rule, i) {
    const ok = rule.test(value);
    items[i].classList.toggle('ok', ok);
    if (!ok) missing++;
  });
  const blocking = missing > 0 && (value !== '' || input.required);
  input.setCustomValidity(blocking ? 'Le mot de passe ne respecte pas toutes les règles indiquées.' : '');
}
```

Chaque règle est un petit objet contenant un libellé et une fonction de test (souvent une expression régulière). `setCustomValidity` utilise la validation HTML5 native : tant que le message n'est pas vide, le navigateur refuse d'envoyer le formulaire et affiche ce message. La liste porte `aria-live="polite"` pour que

les lecteurs d'écran annoncent les changements. Ces règles sont **les mêmes** que celles de `includes/password_policy.php`, qui reste le contrôle qui fait foi.

Filtres de saisie (data-only)

Un champ marqué `data-only="letters"`, `"digits"`, `"plate"` ou `"model"` n'accepte que certains caractères ; les autres sont supprimés dès la frappe (ou le collage). Exemple d'utilisation dans `admin/garages.php` :

```
<input type="text" name="nom" id="ngNom" data-only="letters" required placeholder="Ex. Mbarga">
```

Et le code, dans `main.js` :

```
(function () {
  const forbidden = {
    digits: /^[0-9]/g,
    letters: /^[A-Za-zÀ-ÖØ-öø-ÿ' -]/g,
    plate: /^[A-Za-z0-9 -]/g,
    model: /^[A-Za-zÀ-ÖØ-öø-ÿ0-9 .\/+!-]/g
  };

  // Délégation sur document : couvre aussi les champs ajoutés après le
  // changement (modales, formulaires dynamiques).
  document.addEventListener('input', function (e) {
    const input = e.target;
    const pattern = input.dataset && forbidden[input.dataset.only];
    if (!pattern || input.type === 'number') return;
    let cleaned = input.value.replace(pattern, '');
    if (input.dataset.only === 'plate') cleaned = cleaned.toUpperCase();
    ...
  });
});
```

À expliquer : `^[0-9]` signifie « tout caractère qui n'est pas un chiffre » ; le drapeau `g` remplace toutes les occurrences. `input.dataset.only` lit l'attribut `data-only`. Le code recalcule aussi la position du curseur, sinon il sauterait à la fin du champ à chaque caractère supprimé. Un second gestionnaire `keydown` bloque les touches `e`, `E`, `+`, `-` sur les champs `type="number"`, que le navigateur accepte normalement (notation scientifique comme `1e5`).

Les immatriculations camerounaises (par exemple `LT 123 AB`) ont des formats variés : le filtre `plate` est donc volontairement souple (lettres, chiffres, espace, tiret), et c'est `validatePlate()` côté PHP qui tranche.

3.9 IIFE et portée des variables

La portée (scope) d'une variable est la zone du code où elle existe. En JS :

- une variable déclarée hors de toute fonction est **globale** : visible partout, y compris par les autres scripts de la page, avec un risque de conflit de noms ;
- `let` et `const` sont limités au **bloc** `{ ... }` où ils sont déclarés ; `const` interdit en plus la réaffectation ;
- `var` (ancienne syntaxe, encore présente dans le menu mobile) est limité à la **fonction**, pas au bloc.

Une IIFE (Immediately Invoked Function Expression, fonction exécutée immédiatement) est une fonction qu'on déclare et qu'on appelle aussitôt :

```
(function () {
  let overlay = null; // invisible en dehors de cette fonction
  let opener = null;

  function el(tag, className, text) { ... }
  function show(trigger) { ... }
  function hide() { ... }

  document.addEventListener('click', function (e) { ... });
})();
```

C'est la structure de la fenêtre du rapport, des filtres `data-only` et de tout `assistant.js`. Les variables `overlay`, `opener` et les fonctions `el`, `show`, `hide` sont **privées** : un autre script ne peut ni les lire ni les écraser. Pourtant, les gestionnaires d'événements, eux, continuent à y avoir accès longtemps après la fin de l'IIFE : c'est ce qu'on appelle une **fermeture** (closure), une fonction qui « se souvient » des variables de l'endroit où elle a été créée.

À l'inverse, quand une fonction doit être appelée depuis le HTML (par exemple `onclick="openAssign(...)"`), le code l'attache volontairement à `window` : `window.openAssign = function (...) {...}`.

À retenir

- IIFE = `(function () { ... })()` : crée une portée privée et s'exécute tout de suite.
- Objectif : ne pas polluer l'espace global et éviter les conflits entre scripts.
- Les fonctions définies à l'intérieur gardent l'accès à ses variables (fermeture).
- Préférer `const` et `let` à `var`.

3.10 Pourquoi le JavaScript ne remplace jamais la validation serveur

C'est probablement **la** question de sécurité que posera le jury. Le JavaScript s'exécute sur l'ordinateur de l'utilisateur, donc **sous son contrôle**. N'importe qui peut :

- désactiver JavaScript dans son navigateur ;
- modifier le code ou le HTML avec les outils de développement (touche F12), par exemple supprimer l'attribut `data-only` ou `required` ;
- ne pas utiliser de navigateur du tout et envoyer directement une requête HTTP forgée (avec `curl` ou Postman) vers `ajax/campay_collect.php`.

Le JavaScript du projet est donc un **confort** : il aide l'utilisateur honnête à corriger ses erreurs tout de suite, sans aller-retour avec le serveur. La **sécurité**, elle, est toujours assurée par PHP. Le code le dit explicitement à plusieurs endroits, par exemple dans `main.js` :

```
/**
 * Contrôle minimal avant envoi : champs [required] non vides et format des
 * champs email. Simple confort de saisie ; la validation qui fait foi est
 * faite côté serveur par chaque page.
 */
validate: function(form) { ... }
```

Contrôle dans le navigateur (confort)	Contrôle sur le serveur (fait foi)
Règles de mot de passe cochées en direct	<code>includes/password_policy.php</code>
Filtres <code>data-only</code>	<code>validateDigitsOnly</code> , <code>validateLettersOnly</code> , <code>validatePlate</code> , <code>validateModel</code> (<code>config/config.php</code>)
Champs <code>required</code> , format e-mail	Contrôles de chaque page PHP avant l'écriture en base
Affichage « Paiement reçu »	<code>paymentRefresh()</code> interroge CamPay ; webhook signé
Bouton « Payer » visible pour le propriétaire	<code>paymentFindForClient()</code> vérifie la propriété du paiement

Piège fréquent

Répondre « j'ai mis une validation JavaScript, donc la donnée est sûre ». Le jury attend l'inverse : **toute donnée reçue par le serveur est suspecte**, quelle que soit la validation faite dans le navigateur. On valide côté client pour l'ergonomie, côté serveur pour la sécurité.

Questions du jury

Q. Quelle est la différence entre PHP et JavaScript dans votre application ?

PHP s'exécute sur le serveur : il lit la base de données, vérifie les droits et produit le HTML envoyé au navigateur. JavaScript s'exécute ensuite dans le navigateur de l'utilisateur : il rend la page interactive (menus, modales, filtres de saisie) et dialogue avec le serveur en AJAX. PHP transmet à JS les informations utiles via des constantes (`SITE_URL`, `USER_ID` dans `includes/footer.php`) et le jeton CSRF via une balise `<meta>`.

Q. Pourquoi utiliser jQuery alors que le JavaScript moderne sait tout faire ?

jQuery venait du modèle de départ et reste pratique pour les sélections courtes, les animations `fadeIn/fadeOut` et surtout les requêtes AJAX. Son principal intérêt ici est `$.ajaxSetup`, qui ajoute le jeton CSRF à toutes les requêtes en un seul endroit, et `ajaxError`, qui gère toutes les erreurs HTTP de façon commune. Le code récent est écrit en JS natif ; une évolution possible serait de remplacer progressivement jQuery par `fetch` pour alléger les pages.

Q. Qu'est-ce qu'une faille XSS et comment l'évitez-vous côté JavaScript ?

Une XSS consiste à faire exécuter du code JavaScript malveillant dans le navigateur d'un autre utilisateur, en l'injectant dans un contenu affiché (titre, message...). Côté JS, je pose le texte avec `textContent`, qui n'interprète jamais le HTML, comme dans la fenêtre du rapport. Quand je dois construire du HTML, j'échappe d'abord les données avec `escapeHtml()`, par exemple pour les notifications. Côté PHP, la fonction `h()` échappe aussi tout ce qui est affiché.

Q. Comment protégez-vous vos requêtes AJAX contre le CSRF ?

Le serveur génère un jeton secret stocké en session et l'écrit dans `<meta name="csrf-token">`. `main.js` configure `jQuery.ajaxSetup` pour ajouter ce jeton dans l'en-tête `X-CSRF-Token` de chaque requête vers notre domaine, jamais vers un domaine étranger. Les fichiers `ajax/*.php` appellent `verifyRequestCSRF()`, qui le compare à la session avec `hash_equals`, et renvoient 403 en cas d'échec. Un site pirate ne connaît pas ce jeton et ne peut donc pas forger une requête valide.

Q. Comment la page sait-elle que le client a validé le paiement sur son téléphone ?

Après le lancement du paiement, la page appelle `ajax/campay_status.php` toutes les 5 secondes : c'est du polling. Le serveur interroge CamPay, met à jour la table `paiement` et renvoie le statut (`PAYE`, `ECHOUÉ` ou en attente). J'utilise un `setTimeout` relancé après chaque réponse plutôt qu'un `setInterval`, pour ne jamais empiler les requêtes, et je m'arrête après 36 essais, soit 3 minutes. En parallèle, le webhook de CamPay peut mettre à jour le paiement même si le client a fermé la page.

Q. Qu'est-ce que la délégation d'événements et pourquoi l'utiliser ?

Au lieu d'attacher un gestionnaire à chaque bouton, on en attache un seul à `document` et on regarde, grâce à `e.target.closest(...)`, quel élément a été cliqué. Cela fonctionne parce que les événements remontent dans l'arbre du DOM. L'avantage est que les éléments ajoutés après le chargement, dans une modale ou une liste chargée en AJAX, sont aussi couverts, et qu'un seul gestionnaire consomme moins de mémoire. Je l'utilise pour les boutons « Voir le rapport » et les filtres `data-only`.

Q. Si l'utilisateur désactive JavaScript, votre application est-elle encore sûre ?

Oui, car aucune règle de sécurité ne repose sur le JavaScript. Les filtres de saisie, les règles de mot de passe et la validation des formulaires sont doublés par des contrôles PHP qui font foi. Sans JS, l'utilisateur perd le confort (pas de menu mobile, pas de mise à jour automatique des notifications, pas de suivi du paiement en direct), mais il ne peut contourner aucun contrôle. C'est le principe : « ne jamais faire confiance au client ».

Q. À quoi sert l'écriture `(function () { ... })()` ; que l'on voit dans vos fichiers ?

C'est une IIFE, une fonction exécutée immédiatement. Elle crée une portée privée : les variables et fonctions déclarées dedans ne deviennent pas globales et ne risquent pas d'entrer en conflit avec d'autres scripts. Les gestionnaires d'événements créés à l'intérieur gardent malgré tout l'accès à ces variables grâce au mécanisme de fermeture. Je l'utilise pour la fenêtre du rapport, les filtres de saisie et l'assistant IA.

4. MySQL : la base de données

Toutes les données de SmartAutoTrack (comptes, véhicules, interventions, rapports, paiements, abonnements, rappels...) sont rangées dans une base **MySQL** appelée `charles`. Le code PHP n'écrit jamais dans des fichiers pour garder ces informations : il envoie des requêtes SQL à MySQL par l'intermédiaire de PDO (voir le chapitre PHP). Ce chapitre explique comment la base est organisée, comment le projet l'interroge, et comment il protège les données quand plusieurs personnes agissent en même temps.

4.1 Qu'est-ce qu'un SGBD relationnel ?

Un **SGBD** (Système de Gestion de Base de Données) est un logiciel serveur qui stocke des données, les retrouve vite et garantit leur cohérence. **Relationnel** veut dire que les données sont rangées dans des **tables** (lignes et colonnes, comme un tableur), et que les tables sont **reliées** entre elles par des identifiants : un véhicule contient l'identifiant de son propriétaire (`vehicule.idClient`), une intervention celui du véhicule (`intervention.idVehicule`), etc. On interroge la base avec le langage **SQL** (Structured Query Language).

Sur le poste de développement, Apache et PHP viennent de **XAMPP**. Attention à un détail que le jury peut relever : le « MySQL » livré par XAMPP est en réalité **MariaDB** (ici la version 10.4.24), un fork de MySQL compatible avec lui. Le poste a aussi un **MySQL Server 8.0** installé à part, et l'export de la base présent dans le dépôt (`charles (1).sql`) a été produit par un serveur **MySQL 8.0.41** via phpMyAdmin 5.2.0. Le code n'utilise que du SQL standard (et quelques extensions communes aux deux, comme `INSERT IGNORE` ou `ON DUPLICATE KEY UPDATE`) : il fonctionne avec l'un comme avec l'autre.

Pourquoi MySQL ici ? Les données de l'application sont très structurées et très liées entre elles (un client possède des véhicules, qui ont des anomalies, qui donnent lieu à des interventions, qui produisent des réparations, qui sont payées). Une base relationnelle est exactement faite pour ce genre de données. MySQL est gratuit, installé en local avec XAMPP (sous la forme de MariaDB, compatible), très bien supporté par PHP, et proposé par presque tous les hébergeurs.

Le moteur InnoDB

MySQL peut stocker une table avec différents « moteurs de stockage ». Les tables du projet utilisent **InnoDB**, le moteur par défaut de MySQL : chaque table créée par `scripts/migrate_structure.php` le déclare (`ENGINE=InnoDB`), et les clés étrangères du schéma de base n'existeraient pas sans lui. C'est lui qui apporte les trois fonctionnalités dont SmartAutoTrack a absolument besoin :

- les **clés étrangères** (le moteur ancien MyISAM ne les vérifie pas) ;
- les **transactions** (tout ou rien : `COMMIT` / `ROLLBACK`) ;
- le **verrouillage par ligne** (`SELECT ... FOR UPDATE`), qui évite les doubles paiements ou les doubles affectations.

Le jeu de caractères utf8mb4

Chaque table créée par les scripts de migration déclare `DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci`. **utf8mb4** est le vrai UTF-8 complet (jusqu'à 4 octets par caractère) : il accepte les accents français (é, è, ç), les symboles et même les emojis. L'ancien « utf8 » de MySQL (alias de utf8mb3) s'arrête à 3 octets et refuse certains caractères. La **collation** `utf8mb4_unicode_ci` définit l'ordre de tri et la comparaison : `ci` = **case insensitive**, donc « Toyota » et « toyota » sont considérés égaux dans un `WHERE` (cette collation ignore aussi les accents : « é » = « e »). Le même jeu de caractères est demandé à la connexion, dans `config/database.php` :

```
$this->conn = new PDO(
    "mysql:host=" . $this->host . ";dbname=" . $this->db_name . ";charset=utf8mb4",
    $this->username,
    $this->password,
    array(
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false
    )
);
```

Les identifiants de connexion ne sont pas écrits dans le code : ils viennent des variables d'environnement `HCH_DB_HOST`, `HCH_DB_NAME`, `HCH_DB_USER` ... ou du fichier non versionné `config/local.php`. Si rien n'est configuré, le nom de base par défaut est `'charles'` : sous Windows, MySQL ignore la casse des noms de base, mais pas sous Linux, où il faut écrire le nom exact.

Piège fréquent

Avoir des tables en `utf8mb4` mais une connexion dans un autre jeu de caractères donne des accents cassés (« `Ã©` » au lieu de « `é` »). Il faut que la table **et** la connexion (`charset=utf8mb4` dans le DSN) soient en `utf8mb4`. C'est le cas dans le projet.

4.2 La base `charles` et ses tables

Le schéma actuel est documenté dans `sql/SCHEMA_ACTUEL.md`. Les noms suivent le style **camelCase** (`idUtilisateur`, `dateIntervention`). Les fichiers `sql/schema.sql` et `charles (1).sql` décrivent l'**ancien** schéma (tables `users`, `vehicules` ...) et sont marqués obsolètes.

Les personnes : un utilisateur, plusieurs rôles

La table `utilisateur` contient ce qui est commun à tout le monde (nom, prénom, email, mot de passe haché, téléphone, thème, photo, `googleId`). Le **rôle** n'est pas une colonne : il se déduit de la table dans laquelle l'identifiant apparaît. C'est un modèle d'**héritage** (spécialisation) :

```
utilisateur (idUtilisateur, nom, prenom, motDePasse, email, telephone,
            themePreference, photoProfil, dateCreation)
├─ administrateur (idAdministrateur)
├─ client (idClient, typeClient)
│   ├─ particulier (idClient, adresse)
│   └─ entreprise (idClient, idEntreprise, raisonSociale, adresse)
└─ technicien (idTechnicien, idGarage, specialite, statutValidation,
               competences, experience)
```

(extrait de `sql/SCHEMA_ACTUEL.md`). Ainsi `client.idClient` a la même valeur que `utilisateur.idUtilisateur` du même compte. La fonction `getUserRole()` de `config/roles.php` centralise cette déduction.

Tableau des tables principales

Table	Contenu
<code>utilisateur</code>	Tous les comptes (données communes).
<code>client</code> , <code>particulier</code> , <code>entreprise</code>	Propriétaires de véhicules (personne physique ou société).
<code>technicien</code>	Mécaniciens : spécialité, statut de validation par l'admin, garage éventuel.

Table	Contenu
administrateur	Comptes d'administration.
garage	Garages partenaires (nomGarage , statutGarage , compte de connexion idUtilisateur).
vehicule	Véhicules : marque, modèle, immatriculation, kilométrage, dates d'assurance et de visite technique.
anomalie	Problèmes détectés sur un véhicule (niveau FAIBLE / MOYEN / CRITIQUE).
intervention	Demande de prise en charge : client, véhicule, technicien, garage, priorité, statut.
reparation	Rapport de fin d'intervention : diagnostic, travaux, pièces, coût, kilométrage relevé.
paiement	Paiements Mobile Money CamPay (réparation ou abonnement).
abonnement , ia_usage	Formule Premium ; compteur journalier de messages à l'assistant IA.
notifications , messages	Notifications internes et messagerie entre utilisateurs.
journalactivites	Journal de toutes les actions importantes (qui, quoi, quand).
onglet_vu	Dernière ouverture des onglets « Journal » et « Interventions » (pastilles de nouveautés).
entretien , entretien_regle , rappel_envoye	Historique des vidanges / freins / pneus, règles de périodicité, rappels déjà envoyés.
password_resets	Jetons (hachés) de réinitialisation du mot de passe.
technician_documents , documentgarage	Justificatifs déposés par les techniciens et les garages.

La chaîne métier principale se lit ainsi : **client** → **vehicule** → **anomalie / intervention** → **reparation** → **paiement**, avec `entretien` et `rappel_envoye` accrochés au véhicule.

4.3 Clés primaires et AUTO_INCREMENT

Une **clé primaire** (`PRIMARY KEY`) identifie chaque ligne de façon unique : deux lignes ne peuvent pas avoir la même valeur, et elle ne peut pas être NULL. Avec `AUTO_INCREMENT` , MySQL attribue lui-même le numéro suivant (1, 2, 3...) à chaque `INSERT` . Exemple de la table `paiement` dans `scripts/migrate_structure.php` :

```
CREATE TABLE paiement (
  idPaiement INT AUTO_INCREMENT PRIMARY KEY,
  idClient INT NOT NULL,
  idIntervention INT NULL,
  montant DECIMAL(10,2) NOT NULL,
  datePaiement DATETIME NULL,
  typePaiement VARCHAR(30) NULL,
  statut ENUM('EN_ATTENTE','PAYE','ECHOUE','ANNULE') NOT NULL DEFAULT 'EN_ATTENTE',
  CONSTRAINT fk_paiement_client FOREIGN KEY (idClient) REFERENCES client(idClient) ON DELETE RESTRICT,
  CONSTRAINT fk_paiement_intervention FOREIGN KEY (idIntervention) REFERENCES intervention(idIntervention) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
```

Après l'insertion, PHP récupère le numéro attribué avec `$conn->lastInsertId()` (utilisé dans `includes/payments.php` pour connaître l'`idPaiement` créé).

Clés primaires composées

Certaines tables n'ont pas de numéro automatique : leur clé primaire est formée de **plusieurs colonnes**. C'est un choix volontaire, qui sert de règle d'unicité métier :

```
CREATE TABLE rappel_envoye (
    idVehicule INT NOT NULL,
    type VARCHAR(20) NOT NULL,
    echeance DATE NOT NULL,
    palier ENUM('J30','J7','J0','RETARD') NOT NULL,
    canal ENUM('NOTIF','EMAIL') NOT NULL,
    dateEnvoi DATETIME NOT NULL,
    PRIMARY KEY (idVehicule, type, echeance, palier, canal),
    CONSTRAINT fk_rappel_envoye_vehicule FOREIGN KEY (idVehicule) REFERENCES vehicule(idVehicule)
ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
```

Ici, la base elle-même garantit qu'on n'enregistre **jamais deux fois** le même rappel (même véhicule, même type d'échéance, même date, même palier J-30/J-7/J0/retard, même canal). De même, `ia_usage` a pour clé (`idUtilisateur`, `jour`) : une seule ligne de compteur par utilisateur et par jour, et `onglet_vu` a pour clé (`idUtilisateur`, `onglet`).

4.4 Clés étrangères et comportements ON DELETE

Une **clé étrangère** (`FOREIGN KEY ... REFERENCES`) oblige une colonne à contenir une valeur qui existe dans une autre table. Exemple : on ne peut pas créer de paiement pour un `idClient` qui n'existe pas. C'est l'**intégrité référentielle** : la base refuse les données « orphelines ». L'option `ON DELETE` dit ce qui se passe quand on supprime la ligne référencée. Le projet utilise les trois comportements :

Option	Effet	Exemple réel
CASCADE	Les lignes dépendantes sont supprimées automatiquement.	<code>notifications.user_id</code> , <code>messages</code> , <code>onglet_vu</code> , <code>ia_usage</code> , <code>password_resets</code> → <code>utilisateur</code> ; <code>entretien</code> et <code>rappel_envoye</code> → <code>vehicule</code> ; <code>abonnement</code> → <code>client</code> .
SET NULL	La ligne dépendante est conservée, la colonne devient NULL.	<code>paiement.idAbonnement</code> → <code>abonnement</code> ; <code>entretien.idReparation</code> → <code>reparation</code> ; <code>garage.idUtilisateur</code> → <code>utilisateur</code> ; <code>anomalie.idIntervention</code> → <code>intervention</code> ; liens de <code>journalactivites</code> .
RESTRICT	La suppression est refusée tant qu'il reste des lignes dépendantes.	<code>paiement.idClient</code> , <code>paiement.idIntervention</code> , et selon <code>SCHEMA_ACTUEL.md</code> <code>vehicule.idClient</code> , <code>intervention.idClient</code> , <code>intervention.idVehicule</code> .

Le choix suit une logique simple : ce qui n'a **aucun sens sans son parent** part avec lui (CASCADE : une notification sans destinataire est inutile) ; ce qui doit **survivre** à son parent perd juste le lien (SET NULL : l'historique d'une vidange reste même si le rapport de réparation est supprimé) ; ce qui a une **valeur comptable** est protégé (RESTRICT : on ne fait pas disparaître un paiement par accident). Exemple réel tiré de `scripts/migrate_structure.php` :

```

CONSTRAINT fk_entretien_vehicule FOREIGN KEY (idVehicule)
REFERENCES vehicule(idVehicule) ON DELETE CASCADE,
CONSTRAINT fk_entretien_reparation FOREIGN KEY (idReparation)
REFERENCES reparation(idReparation) ON DELETE SET NULL

```

Conséquence du RESTRICT : pour supprimer un client, l'administrateur doit effacer les dépendances **dans le bon ordre**, à l'intérieur d'une transaction (admin/clients.php) :

```

$conn->beginTransaction();
// Le schéma protège paiements/interventions par des clés RESTRICT (pas
// de cascade automatique) : suppression explicite dans l'ordre. Messages
// et notifications, eux, sont en CASCADE depuis utilisateur.
$conn->prepare("DELETE FROM paiement WHERE idClient = ?")->execute([$client_id]);
$conn->prepare("DELETE FROM intervention WHERE idClient = ?")->execute([$client_id]);
$conn->prepare("DELETE FROM vehicule WHERE idClient = ?")->execute([$client_id]);
$conn->prepare("DELETE FROM utilisateur WHERE idUtilisateur = ?")->execute([$client_id]);
$conn->commit();

```

Piège fréquent

Une colonne en `ON DELETE SET NULL` doit être déclarée `NULL` (facultative), sinon MySQL refuse de créer la contrainte. C'est pour cela que `paiement.idAbonnement` et `entretien.idReparation` sont en `INT NULL`. De même, `paiement.idIntervention` a été rendu facultatif pour les paiements d'abonnement : une valeur `NULL` n'est simplement pas contrôlée par la clé étrangère.

4.5 Les types de colonnes

Type	Usage	Exemple dans le projet
INT	Entiers : identifiants, kilométrage, compteurs.	vehicule.kilometrage INT NOT NULL DEFAULT 0, ia_usage.nb
VARCHAR(n)	Texte court de longueur maximale n.	paiement.telephone VARCHAR(20), referenceExterne VARCHAR(64)
TEXT	Texte long sans limite pratique.	reparation.diagnostic, travauxEffectues, messages.contenu
DECIMAL(p,s)	Nombre décimal exact : argent, durées.	paiement.montant DECIMAL(10,2), reparation.dureeIntervention DECIMAL(5,2)
DATE	Une date sans heure.	vehicule.dateExpirationAssurance, entretien.dateEntretien, ia_usage.jour
DATETIME / TIMESTAMP	Date et heure.	utilisateur.dateCreation DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP, messages.date_envoi TIMESTAMP, abonnement.dateFin
ENUM(...)	Liste fermée de valeurs autorisées : statuts.	anomalie.niveau ENUM('FAIBLE','MOYEN','CRITIQUE')
TINYINT(1)	Booléen (0/1).	entretien_regle.actif, password_resets.used

Pourquoi DECIMAL pour l'argent ? Les types `FLOAT` / `DOUBLE` stockent une approximation binaire : $0,1 + 0,2$ n'y vaut pas exactement $0,3$. Pour des montants, on veut une valeur exacte, donc `DECIMAL(10,2)` : 10 chiffres au total dont 2 après la virgule. En pratique le franc CFA (XAF) n'a pas de centimes, et le code arrondit le montant à l'entier avant de l'envoyer à CamPay (`((int)round((float)$repair['cout']))` dans `includes/payments.php`).

Pourquoi ENUM pour les statuts ? La base refuse toute valeur qui n'est pas dans la liste (en mode strict) : impossible d'enregistrer par erreur « PAYEE » ou « REGLE » à la place de `PAYE`. (Avec la collation `_ci`, « paye » en minuscules serait en revanche accepté et rangé comme `PAYE`.) Exemples de cycles de vie : intervention `PLANIFIEE` → `EN_COURS` → `TERMINEE` (ou `ANNULEE`), anomalie `NOUVELLE` / `EN_COURS` / `TRAITEE` / `IGNOREE`, paiement `EN_ATTENTE` / `PAYE` / `ECHUE` / `ANNULE`, technicien `EN_ATTENTE` / `VALIDE` / `REJETE`.

`NOT NULL` rend une colonne obligatoire ; `DEFAULT` donne la valeur utilisée si l'`INSERT` ne la précise pas (par exemple `statut` vaut `'EN_ATTENTE'` à la création d'un paiement).

4.6 Les index

Un **index** est une structure triée (un arbre B+ dans InnoDB) qui permet à MySQL de retrouver des lignes sans lire toute la table, comme l'index à la fin d'un livre. La clé primaire est automatiquement un index ; InnoDB exige aussi un index sur chaque colonne de clé étrangère et le crée lui-même si aucun index utilisable n'existe déjà. Le projet en ajoute là où il cherche souvent :

```
$indexes = [
    ['uq_paiement_reference_externe', 'ADD UNIQUE INDEX uq_paiement_reference_externe
(referenceExterne)'],
    ['idx_paiement_reference_campay', 'ADD INDEX idx_paiement_reference_campay
(referenceCampay)'],
    ['idx_paiement_reparation', 'ADD INDEX idx_paiement_reparation (idReparation)'],
];
```

(`scripts/migrate_structure.php`). Le webhook CamPay retrouve un paiement par sa référence : sans index, MySQL devrait parcourir toute la table `paiement` à chaque notification. L'index **UNIQUE** sur `referenceExterne` ajoute une garantie : deux paiements ne peuvent pas avoir la même référence.

Le projet utilise aussi des **index composites** (sur plusieurs colonnes), pensés pour une requête précise : `INDEX idx_abonnement_client_statut_fin (idClient, statut, dateFin)` sert la question « ce client a-t-il un abonnement ACTIF qui n'est pas encore terminé ? », et `idx_entretien_vehicule_type_date` (`idVehicule, type, dateEntretien`) sert « quelle est la dernière vidange de ce véhicule ? ». D'autres index : `idx_journal_dateheure` (comptage des activités récentes), `idx_utilisateur_google_id` (connexion Google), `idx_pwd_resets_token`.

Piège fréquent

Un index composite ne sert que si la requête utilise ses colonnes **en partant de la gauche** : (`idClient, statut, dateFin`) aide une recherche sur `idClient`, mais pas une recherche sur `dateFin` seule. Et les index ont un coût : ils accélèrent les lectures mais ralentissent un peu les écritures et prennent de la place. On n'indexe donc pas toutes les colonnes.

4.7 Les requêtes types du projet

Toutes les requêtes passent par des **requêtes préparées** PDO : la requête contient des `?`, et les valeurs sont envoyées séparément avec `execute([...])`. MySQL ne peut alors jamais interpréter une valeur saisie par

l'utilisateur comme du code SQL : c'est la protection contre l'**injection SQL**.

SELECT avec JOIN

Un **JOIN** (jointure interne) assemble des lignes de deux tables qui correspondent ; une ligne sans correspondance disparaît du résultat. Pour savoir si une réparation peut être payée, `includes/payments.php` joint la réparation à son intervention afin de vérifier qu'elle appartient bien au client connecté :

```
SELECT r.idReparation, r.titre, r.cout, i.idIntervention, i.idClient
FROM reparation r
JOIN intervention i ON i.idIntervention = r.idIntervention
WHERE r.idReparation = ? AND i.idClient = ? AND r.statut = 'TERMINEE' AND r.cout > 0
```

LEFT JOIN, GROUP BY et COUNT

Un **LEFT JOIN** garde **toutes** les lignes de la table de gauche, même sans correspondance à droite (les colonnes de droite valent alors NULL). **GROUP BY** regroupe les lignes et **COUNT** les compte par groupe. La liste des clients de l'admin (`admin/clients.php`) affiche, pour chacun, son nombre de véhicules et d'anomalies ouvertes :

```
SELECT u.idUtilisateur AS id, u.nom, u.prenom, u.email, u.telephone, u.dateCreation AS created_at,
       c.typeClient, e.raisonSociale,
       COUNT(DISTINCT v.idVehicule) AS vehicles_count,
       COUNT(DISTINCT CASE WHEN a.statut IN ('NOUVELLE', 'EN_COURS') THEN a.idAnomalie END) AS
active_anomalies
FROM utilisateur u
JOIN client c ON c.idClient = u.idUtilisateur
LEFT JOIN entreprise e ON e.idClient = c.idClient
LEFT JOIN vehicule v ON u.idUtilisateur = v.idClient
LEFT JOIN anomalie a ON v.idVehicule = a.idVehicule
WHERE $whereSql
GROUP BY u.idUtilisateur
ORDER BY u.dateCreation DESC
```

Le **JOIN client** ne garde que les utilisateurs qui sont des clients. Les **LEFT JOIN** font qu'un client **sans véhicule** apparaît quand même, avec 0. `COUNT(DISTINCT ...)` évite de compter plusieurs fois le même véhicule (un véhicule avec 3 anomalies produit 3 lignes après jointure). `$whereSql` n'est construit qu'à partir de morceaux fixes écrits dans le code ; les mots recherchés passent par des `?`.

Autre exemple, le graphique de tendance du tableau de bord admin (`admin/dashboard.php`) :

```
SELECT DATE(dateIntervention) AS jour, COUNT(*) AS total
FROM intervention
WHERE dateIntervention >= DATE_SUB(CURDATE(), INTERVAL 6 DAY)
GROUP BY DATE(dateIntervention)
```

Les jours sans intervention n'apparaissent pas dans le résultat d'un **GROUP BY** : le code PHP complète donc les jours manquants à 0 pour toujours dessiner 7 points.

Sous-requête EXISTS / NOT EXISTS

EXISTS (`SELECT 1 ...`) est vrai dès qu'**au moins une** ligne correspond ; MySQL peut s'arrêter à la première trouvée. Le projet s'en sert pour filtrer sans dupliquer les lignes. Dans `client/reparations.php`, filtre « réparations restant à payer » :

```
$where_conditions[] = "NOT EXISTS (SELECT 1 FROM paiement p WHERE p.idReparation = r.idReparation AND p.statut = 'PAYE')";
```

Et dans `ajax/get_vehicule.php`, un technicien ne peut voir un véhicule que s'il a une intervention dessus (contrôle d'accès directement en SQL) :

```
WHERE v.idVehicule = ? AND EXISTS (SELECT 1 FROM intervention i WHERE i.idVehicule = v.idVehicule AND i.idTechnicien = ?)
```

INSERT et UPDATE

Création d'un paiement en attente (`includes/payments.php`), puis mise à jour en cas d'échec de CamPay :

```
INSERT INTO paiement (idClient, idIntervention, idReparation, montant, datePaiement, typePaiement, statut, referenceExterne, telephone)
VALUES (?, ?, ?, ?, ?, 'MOBILE_MONEY', 'EN_ATTENTE', ?, ?)

UPDATE paiement SET statut = 'ECHOUE', messageErreur = ? WHERE idPaiement = ?
```

UPDATE conditionnel et rowCount()

Un **UPDATE conditionnel** répète dans son `WHERE` les conditions qui rendent l'action valide. Si quelqu'un a changé la ligne entre-temps, l'UPDATE ne touche aucune ligne, et `rowCount()` le révèle. Extrait de l'affectation d'une demande par l'admin (`admin/includes/helpers.php`) :

```
// Mêmes conditions que la lecture : rowCount = 0 signifie que la
// demande a changé entre-temps.
$stmt = $conn->prepare("
    UPDATE intervention SET idTechnicien = ?, idGarage = ?, statut = 'PLANIFIEE'
    WHERE idIntervention = ? AND idTechnicien IS NULL AND statut IN ('PLANIFIEE', 'ANNULEE')
    AND (idGarage IS NULL OR statut = 'ANNULEE')
");
$stmt->execute([$technicienId, $garageId, $interventionId]);
if ($stmt->rowCount() !== 1) {
    $conn->rollBack();
    return 'Cette demande vient d'être modifiée, merci de recharger la page.';
}
```

INSERT IGNORE

INSERT IGNORE tente l'insertion et, si elle viole une clé primaire ou unique, l'abandonne **sans erreur** (`rowCount` vaut alors 0). Il transforme aussi d'autres erreurs en simples avertissements : on le réserve donc à des insertions dont les valeurs sont déjà contrôlées, comme ici. Le projet s'en sert pour « réserver » un rappel d'entretien une seule fois (`includes/maintenance.php`) :

```
function maintenanceClaimReminder(PDO $conn, int $idVehicule, string $type, string $echeance,
string $palier, string $canal): bool {
    $stmt = $conn->prepare('
        INSERT IGNORE INTO rappel_envoye (idVehicule, type, echeance, palier, canal, dateEnvoi)
        VALUES (?, ?, ?, ?, ?, ?)
    ');
    $stmt->execute([$idVehicule, $type, $echeance, $palier, $canal, date('Y-m-d H:i:s')]);
    return $stmt->rowCount() === 1;
}
```

Si le script `scripts/send_reminders.php` tourne deux fois le même jour, la deuxième insertion est ignorée grâce à la clé primaire composée : le client ne reçoit pas deux fois le même email. C'est atomique : pas besoin de « vérifier puis insérer » (deux étapes qui pourraient se croiser).

INSERT ... ON DUPLICATE KEY UPDATE

Cette forme insère la ligne si elle n'existe pas, et la **met à jour** sinon (on parle d'« upsert »). Le compteur de messages IA d'un client gratuit (`includes/subscription.php`) :

```
INSERT INTO ia_usage (idUser, jour, nb) VALUES (?, ?, 1)
ON DUPLICATE KEY UPDATE nb = LEAST(nb + 1, ?)
```

Premier message du jour : la ligne est créée avec `nb = 1`. Messages suivants : la clé (`idUser`, `jour`) existe déjà, donc `nb` est incrémenté, plafonné par `LEAST()` à la limite + 1. Le tout en une seule instruction atomique : deux messages envoyés en même temps ne peuvent pas « perdre » un incrément. Le même principe sert à `onglet_vu` (`includes/activity_log.php`) et aux règles d'entretien modifiées par l'admin (`includes/maintenance.php`).

4.8 Transactions et SELECT ... FOR UPDATE

Une **transaction** regroupe plusieurs requêtes en un bloc **tout ou rien** : soit tout est validé (`COMMIT`), soit tout est annulé (`ROLLBACK`). C'est la propriété **ACID** d'InnoDB : Atomicité, Cohérence, Isolation, Durabilité. En PHP : `$conn->beginTransaction()`, `$conn->commit()`, `$conn->rollback()`.

`SELECT ... FOR UPDATE`, à l'intérieur d'une transaction, **verrouille** les lignes lues : une autre transaction qui veut les verrouiller à son tour doit **attendre** le `COMMIT` ou le `ROLLBACK` de la première. Cela règle le problème des **accès concurrents** (« race condition ») : deux clics, deux onglets ou deux administrateurs qui agissent en même temps.

Cas 1 : le paiement (`includes/payments.php`)

```
// Transaction + FOR UPDATE sur la réparation : deux clics simultanés ne
// peuvent pas créer deux paiements EN_ATTENTE pour la même réparation.
$conn->beginTransaction();
try {
    $repair = paymentPayableRepair($conn, $clientId, $repairId, true);
    if (!$repair) {
        throw new CampayException('Cette réparation ne peut pas être payée en ligne.');
```

Le paramètre `true` ajoute `FOR UPDATE` à la lecture de la réparation. Le deuxième clic attend donc la fin du premier, puis voit le paiement `EN_ATTENTE` qui vient d'être créé et s'arrête. Point important : l'appel réseau à `CamPay` se fait **après** le `COMMIT`, hors transaction, pour ne pas garder un verrou pendant une requête HTTP lente.

Cas 2 : la clôture d'une intervention (includes/repair_report.php)

```
$conn->beginTransaction();
try {
    $stmt = $conn->prepare("
        SELECT i.idIntervention, i.idClient, i.idTechnicien, i.idVehicule, v.kilometrage
        FROM intervention i
        JOIN vehicule v ON v.idVehicule = i.idVehicule
        WHERE i.idIntervention = ? AND i.$scopeCol = ? AND i.statut = 'EN_COURS'
        FOR UPDATE
    ");
```

Dans la même transaction, le code insère la ligne `reparation`, passe l'intervention à `TERMINEE`, met à jour le kilométrage et l'état du véhicule, et marque `TRAITEE` les anomalies liées. Si une étape échoue, rien n'est enregistré : pas d'intervention terminée sans rapport. Le verrou empêche aussi une double clôture.

(`$scopeCol` vient d'une liste blanche : `idTechnicien` ou `idGarage`.)

Cas 3 : l'affectation par l'admin (admin/includes/helpers.php)

La fonction `admin_assign_internal()` lit l'intervention avec `FOR UPDATE`, puis fait l'`UPDATE` conditionnel vu plus haut. Deux administrateurs qui affectent la même demande au même moment ne peuvent pas l'affecter deux fois. Le même schéma (verrou sur la ligne `client`) est utilisé dans `client/vehicles.php` pour que deux ajouts simultanés ne dépassent pas la limite de véhicules de la formule gratuite, et dans `includes/subscription.php` pour les abonnements.

Piège fréquent

`FOR UPDATE` ne sert à rien **en dehors** d'une transaction : en mode autocommit, le verrou est relâché dès la fin de l'instruction. Il faut aussi toujours appeler `rollback()` dans le `catch`, sinon la transaction reste ouverte et garde ses verrous jusqu'à la fin du script.

4.9 Migrations idempotentes : scripts/migrate_structure.php

Une **migration** est un script qui fait évoluer la structure de la base (ajouter une colonne, une table, un index) sans perdre les données existantes. Le schéma de `charles` a été créé directement en base le 23/09/2026 ; ensuite, chaque nouvelle fonctionnalité (CamPay, abonnements, rapport de fin, pastilles, Google OAuth, rappels d'entretien) a ajouté sa section dans `scripts/migrate_structure.php`.

Idempotent veut dire qu'on peut le lancer une, deux ou dix fois : le résultat est le même. Pour cela, avant chaque modification, le script vérifie si elle est déjà faite en interrogeant `information_schema`, la base « catalogue » où MySQL décrit toutes les tables :

```
function colExists(PDO $c, string $t, string $col): bool {
    $s = $c->prepare("SELECT COUNT(*) FROM information_schema.COLUMNS WHERE
TABLE_SCHEMA=DATABASE() AND TABLE_NAME=? AND COLUMN_NAME=?");
    $s->execute([$t, $col]);
    return (bool)$s->fetchColumn();
}
/* ... */
foreach ($columns as [$t, $c, $def]) {
    if (!colExists($conn, $t, $c)) {
        announce($apply, "ajouter $t.$c");
        if ($apply) $conn->exec("ALTER TABLE `$t` ADD COLUMN `$c` $def");
    }
}
```

Il existe de même `tableExists()`, `indexExists()`, `foreignKeyExists()` et `ensureEnumValues()` (qui ajoute des valeurs à un ENUM en gardant les anciennes, pour que les lignes existantes restent valides).

Simulation puis --apply

```
php scripts/migrate_structure.php          simulation (aucune écriture)
php scripts/migrate_structure.php --apply  applique
```

Sans option, le script affiche seulement la liste [à faire] : on voit ce qui va changer avant de toucher la base. Avec `--apply`, il exécute et affiche [fait]. Le script refuse d'être appelé depuis un navigateur (`if (PHP_SAPI !== 'cli')` → 403) : seule la ligne de commande peut modifier la structure.

Le code s'adapte si la migration manque. Des fonctions comme `paymentsReady()`, `maintenanceReady()` ou `repairReportReady()` vérifient dans `information_schema` que les colonnes existent ; sinon le site désactive simplement la fonctionnalité au lieu de planter.

Piège fréquent

En MySQL, les instructions de structure (`CREATE TABLE`, `ALTER TABLE`) provoquent un **COMMIT implicite** : on ne peut pas les annuler avec un `ROLLBACK`. C'est une raison de plus pour faire d'abord la simulation, et une sauvegarde avant `--apply`.

4.10 phpMyAdmin et sauvegarde

phpMyAdmin est une interface web d'administration de MySQL, fournie avec XAMPP

(`http://localhost/phpmyadmin`). Elle permet de parcourir les tables, voir leur structure, lancer des requêtes SQL dans l'onglet « SQL », vérifier les clés étrangères (vue « Concepteur » / « Relations ») et faire des exports. Ce n'est pas une partie de l'application : c'est l'outil du développeur.

Sauvegarde (export). L'onglet « Exporter » de phpMyAdmin produit un fichier `.sql` qui contient les `CREATE TABLE` et les `INSERT` de toutes les données ; l'onglet « Importer » le recharge. Le fichier `charles (1).sql` du dépôt est justement un tel export (il commence par `-- phpMyAdmin SQL Dump`), de l'ancien schéma. En ligne de commande, l'équivalent est l'outil `mysqldump` :

```
mysqldump -u root -p charles > sauvegarde_charles.sql    (export)
mysql -u root -p charles < sauvegarde_charles.sql        (restauration)
```

La bonne pratique : exporter la base **avant** chaque migration avec `--apply` et avant chaque mise en production.

À retenir

- MySQL est un SGBD relationnel ; toutes les tables sont en **InnoDB** (clés étrangères, transactions, verrous par ligne) et en **utf8mb4** (accents et emojis).
- Le rôle d'un utilisateur n'est pas une colonne : il dépend de la table (`client` , `technicien` , `administrateur`) où apparaît son identifiant.
- Clés étrangères : **CASCADE** pour ce qui n'a pas de sens sans parent, **SET NULL** pour garder l'historique, **RESTRICT** pour protéger les paiements et interventions.
- **DECIMAL** pour l'argent (exact), **ENUM** pour les statuts (liste fermée), **DATE** pour les échéances.
- Requêtes préparées partout (contre l'injection SQL) ; `LEFT JOIN + COUNT(DISTINCT)` pour les compteurs ; `EXISTS` pour filtrer.
- `INSERT IGNORE` et `ON DUPLICATE KEY UPDATE` s'appuient sur une clé primaire/unique pour être atomiques.
- Transaction + `SELECT ... FOR UPDATE` + `UPDATE` conditionnel = pas de double paiement, de double clôture ou de double affectation.
- Migrations idempotentes (vérification dans `information_schema`), simulation puis `--apply` , sauvegarde avant.

4.11 Questions du jury

Q. Pourquoi avoir choisi MySQL plutôt qu'une base NoSQL comme MongoDB ?

Les données de SmartAutoTrack sont fortement liées : un client a des véhicules, qui ont des interventions, qui ont des réparations, qui ont des paiements. Une base relationnelle garantit ces liens par des clés étrangères et permet des jointures. Elle offre aussi des transactions ACID, indispensables pour les paiements. MySQL est gratuit, disponible en local avec XAMPP (via MariaDB, compatible) et chez presque tous les hébergeurs.

Q. Quelle est la différence entre JOIN et LEFT JOIN ? Donnez un exemple de votre projet.

Un JOIN ne garde que les lignes qui ont une correspondance dans les deux tables. Un LEFT JOIN garde toutes les lignes de la table de gauche ; si rien ne correspond à droite, les colonnes de droite valent NULL. Dans la liste des clients de l'admin, j'utilise `LEFT JOIN vehicule` pour qu'un client sans véhicule apparaisse quand même, avec un compteur à 0. Avec un JOIN simple, il disparaîtrait de la liste.

Q. Que se passe-t-il si un client clique deux fois en même temps (deux clics, deux onglets) sur « Payer » pour la même réparation ?

Le paiement s'exécute dans une transaction qui lit la réparation avec `SELECT ... FOR UPDATE` . La deuxième requête attend que la première ait validé (COMMIT). Quand elle reprend, elle voit le paiement `EN_ATTENTE` qui vient d'être créé et renvoie « un paiement est déjà en cours ». Il ne peut donc pas y avoir deux paiements en attente pour la même réparation. L'appel à CamPay est fait après le COMMIT pour ne pas garder le verrou pendant un appel réseau.

Q. Pourquoi stocker les montants en DECIMAL et pas en FLOAT ?

FLOAT stocke une approximation binaire : certains nombres décimaux ne sont pas représentés exactement, ce qui produit des erreurs d'arrondi. Pour de l'argent, on veut une valeur exacte.

`DECIMAL(10,2)` stocke exactement 10 chiffres dont 2 décimales. Pour CamPay, le montant en francs CFA est en plus arrondi à l'entier dans le code PHP.

Q. Expliquez la différence entre ON DELETE CASCADE, SET NULL et RESTRICT dans votre base.

CASCADE supprime automatiquement les lignes dépendantes : si on supprime un utilisateur, ses notifications et messages disparaissent. SET NULL garde la ligne mais vide le lien : si une réparation est supprimée, l'historique d'entretien reste avec `idReparation` à NULL. RESTRICT refuse la suppression tant qu'il reste des dépendances : on ne peut pas supprimer un client qui a des paiements. C'est pour cela que l'admin supprime explicitement paiements, interventions puis véhicules, dans une transaction, avant de supprimer le compte.

Q. Comment évitez-vous d'envoyer deux fois le même rappel d'entretien ?

La table `rappel_envoye` a une clé primaire composée (véhicule, type, échéance, palier, canal). Avant d'envoyer, le code fait un `INSERT IGNORE` dans cette table : si la ligne existe déjà, l'insertion est ignorée et `rowCount()` vaut 0, donc on n'envoie pas. C'est la base qui garantit l'unicité, même si le script de rappels est lancé deux fois en même temps.

Q. Qu'est-ce qu'une migration idempotente et comment la mettez-vous en œuvre ?

C'est un script de modification de structure qu'on peut relancer sans effet si tout est déjà en place.

`scripts/migrate_structure.php` vérifie dans `information_schema` si chaque colonne, table, index ou clé étrangère existe avant de la créer. On le lance d'abord sans option pour une simulation qui liste les actions, puis avec `--apply` pour les exécuter. Il ne peut être lancé qu'en ligne de commande, jamais depuis le navigateur.

Q. Comment votre application se protège-t-elle contre l'injection SQL ?

Toutes les valeurs venant de l'utilisateur passent par des requêtes préparées PDO avec des `?` et `execute([...])`. La requête et les données sont envoyées séparément à MySQL, et l'émulation est désactivée (`ATTR_EMULATE_PREPARES => false`), donc une valeur ne peut jamais être interprétée comme du SQL. Les rares morceaux insérés directement dans la requête (noms de colonnes, filtres) viennent de listes blanches écrites dans le code.

5. HTML et CSS : la structure et l'apparence

Chaque page de SmartAutoTrack est produite par PHP, mais ce que le navigateur reçoit, ce sont deux langages : **HTML**, qui décrit la structure (titres, formulaires, menus, tableaux), et **CSS**, qui décrit l'apparence (couleurs, polices, disposition, adaptation au téléphone). Une règle simple à garder en tête pour la maintenance : **HTML dit ce que c'est, CSS dit à quoi ça ressemble**. Les deux sont séparés, ce qui permet de changer toute la charte graphique sans toucher au contenu.

5.1 Le squelette commun : includes/header.php

Ce que c'est. Toute page HTML commence par un `<!DOCTYPE html>`, une balise `<html>`, un `<head>` (informations pour le navigateur : encodage, titre, feuilles de style) et un `<body>` (le contenu visible).

Pourquoi ici. Plutôt que de répéter ce squelette dans la soixantaine de pages, le projet le centralise dans `includes/header.php`, refermé par `includes/footer.php`. Chaque page définit quelques variables avant l'inclusion.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title><?php echo h(isset($pageTitle) ? $pageTitle . ' - ' . SITE_NAME : SITE_NAME); ?>
</title>
  ...
  <meta name="csrf-token" content="<?php echo generateCSRFToken(); ?>">
</head>
<body class="<?php echo h(isset($bodyClass) ? $bodyClass : ''); ?>">
```

Trois détails à savoir expliquer :

- `lang="fr"` indique la langue aux lecteurs d'écran et aux moteurs de recherche ;
- `charset="UTF-8"` permet d'afficher correctement les accents (« Échéances », « Réparé ») ;
- `meta viewport` est **indispensable au responsive** : sans elle, un téléphone afficherait la page comme un écran d'ordinateur rétréci, et les `@media` ne s'appliqueraient pas comme prévu.

Côté page, la configuration tient en quelques lignes (extrait de `client/abonnement.php`) :

```
$pageTitle = 'Abonnement';
$hideNavbar = true;
$bodyClass = 'v2';
$extraStylesheets = ['assets/css/client_v2.css'];
$extraFonts = ['https://fonts.googleapis.com/css2?family=Sora:wght@600;700;800&family=Manrope:wght@400;500;600;700&display=swap'];
include '../includes/header.php';
```

La classe du `<body>` (`v2`, `av2`, `gv2`, `tv2`) choisit l'habillage de l'espace (client, admin, garage, technicien). On remarque aussi que le titre passe par `h()` : c'est l'échappement HTML contre les attaques XSS (voir le chapitre sécurité).

5.2 HTML sémantique

Ce que c'est. Le HTML sémantique consiste à utiliser la balise qui a du sens plutôt qu'une `<div>` générique : `<nav>` pour un menu, `<aside>` pour un contenu latéral, `<main>` pour le contenu principal, `<button>` pour

une action, `<label>` pour le nom d'un champ.

Pourquoi. Le rendu visuel est le même, mais le navigateur, les lecteurs d'écran (personnes malvoyantes) et les moteurs de recherche comprennent la page. Un `<button>` est en plus utilisable au clavier (Tab, Entrée) sans code supplémentaire.

Dans le projet, la barre latérale du client (`client/includes/sidebar.php`) est un `<aside>` contenant des `<nav>` ; `header.php` ouvre `<main>`, que `footer.php` referme :

```
<aside class="v2-sidebar <?php echo $isEntreprise ? 'v2-sidebar-entreprise' : '' ; ?>">
    ...
    <nav class="v2-nav">
        ...
    </nav>
```

5.3 Les formulaires : type, required, placeholder, autocomplete

Ce que c'est. Un formulaire `<form>` regroupe des champs `<input>` envoyés au serveur. Les **attributs** HTML5 ajoutent une validation et une ergonomie gratuites :

Attribut	Rôle	Exemple dans le projet
<code>type="email"</code>	Le navigateur vérifie la présence d'un @ et affiche un clavier adapté sur mobile	connexion, mot de passe oublié
<code>type="password"</code>	Masque les caractères saisis	connexion
<code>type="tel" + inputmode="numeric"</code>	Clavier numérique sur téléphone	inscription (téléphone)
<code>required</code>	Bloque l'envoi si le champ est vide	tous les champs obligatoires
<code>maxlength</code>	Limite le nombre de caractères	téléphone : 15
<code>placeholder</code>	Texte d'exemple grisé, disparaît à la saisie	<code>exemple@gmail.com</code>
<code>autocomplete</code>	Aide le gestionnaire de mots de passe	<code>username</code> , <code>current-password</code>

Extrait réel du formulaire de connexion (`auth/login.php`) :

```
<form method="POST" id="loginForm">
    <input type="hidden" name="csrf_token" value="<?php echo generateCSRFToken(); ?>">

    <div class="authv2-field">
        <label for="loginEmail">Email <span class="authv2-required" aria-hidden="true">*</span>
    </label>
    <div class="authv2-input-wrap">
        <span class="authv2-icon"><i class="fas fa-envelope"></i></span>
        <input type="email" name="email" id="loginEmail" required autocomplete="username"
            value="<?php echo h($_POST['email'] ?? ''); ?>"
            placeholder="exemple@gmail.com">
        </div>
    </div>
```

Points à expliquer :

- `method="POST"` : les données partent dans le corps de la requête, pas dans l'URL (un mot de passe ne doit jamais apparaître dans l'historique) ;
- le champ caché `csrf_token` protège contre la falsification de requête (CSRF) ;
- `<label for="loginEmail">` est relié à l'`id` du champ : cliquer sur le texte place le curseur dans le champ, et le lecteur d'écran annonce « Email » ;
- `value="<?php echo h($_POST['email']) ?? ' '); ?>"` remet l'email saisi après une erreur, pour ne pas obliger l'utilisateur à tout retaper.

5.4 L'accessibilité avec ARIA

Ce que c'est. ARIA (Accessible Rich Internet Applications) est un ensemble d'attributs `aria-*` qui donnent des informations aux technologies d'assistance quand le HTML seul ne suffit pas.

- `aria-hidden="true"` sur l'astérisque `*` : il est purement décoratif, le lecteur d'écran ne doit pas lire « astérisque » (l'attribut `required` annonce déjà « obligatoire ») ;
- `aria-label="Afficher le mot de passe"` sur le bouton œil : le bouton ne contient qu'une icône, il lui faut un nom lisible ;
- `aria-expanded` et `aria-controls` sur le bouton du menu de la barre de navigation (`navbar-toggle` dans `header.php`) : ils indiquent si le menu est ouvert et quel élément il contrôle ; le bouton « hamburger » des espaces v2, créé par `main.js`, porte lui `aria-label="Ouvrir le menu"` et `aria-expanded`.

```
<input type="password" name="mot_de_passe" id="loginPassword" required autocomplete="current-password" placeholder=".....">
<button type="button" class="authv2-toggle" id="passwordToggle" aria-label="Afficher le mot de passe">
  <i class="fas fa-eye"></i>
</button>
```

Le `type="button"` est important : sans lui, un bouton placé dans un formulaire est par défaut un bouton d'envoi, et cliquer sur l'œil soumettrait le formulaire.

Piège fréquent

Croire que `required` ou `type="email"` suffisent à sécuriser. Ces contrôles tournent dans le navigateur : n'importe qui peut les retirer avec les outils de développement ou envoyer une requête directement. Ils servent au **confort** de l'utilisateur ; la **vraie** validation est refaite en PHP côté serveur (par exemple avec `filter_var` pour l'email) avant tout enregistrement.

5.5 Les variables CSS et la charte commune des 4 espaces

Ce que c'est. Une variable CSS (« propriété personnalisée ») se déclare avec deux tirets, ici dans `:root` (la racine du document, donc visible partout), et se lit avec `var()`.

Pourquoi. Une couleur est définie **une seule fois** : changer `--a-accent` recolore d'un coup tous les boutons, liens actifs et avatars de l'espace admin. C'est aussi ce qui a permis d'harmoniser les quatre espaces sur une même charte.

Extrait de `assets/css/admin_v2.css` :

```

:root {
  --a-accent: #3956E8;
  --a-accent-dark: #2540C4;
  --a-sidebar-bg: #131B3E;
  --a-bg: #F4F5FA;
  --a-text: #171B33;
  --a-border: #ECEE7;
  --a-success: #1E8A4C;
  --a-danger: #E5484D;
}

.av2 { font-family: 'Manrope', sans-serif; background: var(--a-bg); color: var(--a-text); font-size: 15px; }
.av2-navitem.active { background: var(--a-accent); }

```

Organisation en fichiers. Chaque espace a sa feuille `*_v2.css` et son préfixe de variables et de classes :

Espace	Fichier	Classe du body / préfixe	Variables
Client	<code>client_v2.css</code>	<code>v2 / v2-</code>	<code>--v2-accent ...</code>
Administrateur	<code>admin_v2.css</code>	<code>av2 / av2-</code>	<code>--a-accent ...</code>
Garage	<code>garage_v2.css</code>	<code>gv2 / gv2-</code>	<code>--g-accent ...</code>
Technicien	<code>technicien_v2.css</code>	<code>tv2 / tv2-</code>	<code>--t-accent ...</code>
Connexion / inscription	<code>auth_v2.css</code>	<code>authv2-</code>	—

Les quatre espaces partagent les mêmes valeurs (bleu `#3956E8`, barre latérale bleu nuit `#131B3E`, fond `#F4F5FA`). Le commentaire en tête de `admin_v2.css` justifie ce choix : l'admin réutilise le bleu du client « plutôt que d'ajouter une 4e couleur », et les classes sont préfixées « pour ne jamais entrer en collision avec les autres `*_v2.css` ». Les préfixes jouent le rôle d'**espaces de noms** : une règle `.av2-card` ne peut pas modifier par accident une carte client `.v2-card`.

À côté de ces feuilles, `assets/css/themes.css` définit des thèmes (clair, sombre, bleu, vert...) en redéfinissant les mêmes variables sous un sélecteur d'attribut comme `[data-theme="dark"]` : c'est le même mécanisme, appliqué au changement de thème choisi par l'utilisateur.

5.6 La mise en page : Flexbox et Grid

Flexbox (`display: flex`) aligne des éléments sur **une dimension** : une ligne ou une colonne. **Grid** (`display: grid`) découpe l'espace en **deux dimensions** : lignes et colonnes. Règle pratique : Flexbox pour aligner des éléments entre eux (une icône et son texte, une barre d'outils), Grid pour la grille générale d'une page (des cartes de statistiques, deux colonnes).

Flexbox : la structure barre latérale + contenu

```
.av2-shell { width: 100%; min-height: 100vh; display: flex; align-items: stretch; box-sizing: border-box; }
.av2-sidebar { width: 272px; flex-shrink: 0; background: var(--a-sidebar-bg); display: flex; flex-direction: column;
    padding: 22px 16px; position: sticky; top: 0; height: 100vh; overflow-y: auto; }
.av2-main { flex-grow: 1; min-width: 0; padding: 30px 40px 60px; display: flex; flex-direction: column; gap: 26px; }
.av2-topbar { display: flex; justify-content: space-between; align-items: flex-start; gap: 20px; flex-wrap: wrap; }
```

- `.av2-shell` place la barre latérale et le contenu côte à côte ;
- la barre latérale a une largeur fixe (`272px` , `flex-shrink: 0` : elle ne rétrécit jamais) et reste visible au défilement grâce à `position: sticky` ;
- le contenu prend tout le reste (`flex-grow: 1`) ; `min-width: 0` évite qu'un long tableau ne fasse déborder la page ;
- `justify-content: space-between` pousse le titre à gauche et les boutons à droite ; `gap` gère l'espacement sans marges ; `flex-wrap: wrap` autorise le passage à la ligne sur petit écran.

Grid : les cartes de statistiques et les colonnes

```
.av2-stats { display: grid; grid-template-columns: repeat(4, minmax(0, 1fr)); gap: 16px; }
.av2-body { display: grid; grid-template-columns: 2fr 1fr; gap: 22px; align-items: start; }
```

`1fr` signifie « une fraction de l'espace libre ». Quatre colonnes égales pour les statistiques ; pour le corps du tableau de bord, une colonne principale deux fois plus large que la colonne de droite (`2fr 1fr`). Ailleurs, `style.css` utilise une grille qui s'adapte seule : `repeat(auto-fit, minmax(250px, 1fr))` met autant de colonnes d'au moins 250 px que l'écran le permet.

5.7 Le responsive : @media et menu mobile

Ce que c'est. Une requête média `@media` (`max-width: 720px`) contient des règles qui ne s'appliquent que si l'écran fait au plus 720 px de large. On adapte ainsi la même page à l'ordinateur, la tablette et le téléphone, ce qui est essentiel au Cameroun où une grande partie des utilisateurs se connectent depuis un smartphone.

Extrait de `admin_v2.css` : trois paliers successifs.

```
@media (max-width: 1200px) {
    .av2-stats, .av2-stats.av2-stats-3 { grid-template-columns: repeat(2, minmax(0, 1fr)); }
}
@media (max-width: 1100px) {
    .av2-body { grid-template-columns: 1fr; }
}
@media (max-width: 720px) {
    .av2-shell { flex-direction: column; }
    .av2-sidebar { position: fixed; top: 0; left: 0; width: min(82vw, 300px); height: 100vh; z-index: 300;
        transform: translateX(-100%); transition: transform .25s ease; }
    .av2-sidebar.is-open { transform: translateX(0); box-shadow: 10px 0 28px rgba(15,18,38,0.25); }
    .av2-menu-toggle { display: flex; }
}
```

Sur téléphone, la barre latérale devient un **tiroir** : elle est placée hors de l'écran (`translateX(-100%)`) et glisse en place quand elle reçoit la classe `is-open`, avec une animation de 0,25 s. Le bouton « hamburger », caché sur ordinateur (`display: none`), devient visible.

C'est **JavaScript** qui ajoute ou retire cette classe. Le script de `assets/js/main.js` est écrit une seule fois pour les quatre espaces : il détecte le préfixe présent sur la page, crée le bouton et le rideau sombre, et tient `aria-expanded` à jour.

```
var prefix = ['v2', 'av2', 'gv2', 'tv2'].find(function (p) {
  return document.querySelector('.' + p + '-shell');
});
if (!prefix) return;
...
function openNav() {
  sidebar.classList.add('is-open');
  backdrop.classList.add('is-open');
  document.body.classList.add('nav-open');
  toggle.setAttribute('aria-expanded', 'true');
}
```

Le menu se ferme au clic sur le rideau, sur un lien ou avec la touche Échap, et `body.nav-open { overflow: hidden; }` empêche la page de défiler derrière le tiroir. Ce partage des rôles est à souligner devant le jury : **CSS décrit les deux états, JavaScript ne fait que basculer une classe.**

5.8 Les polices Google Fonts : Sora et Manrope

Ce que c'est. Google Fonts est un service gratuit qui héberge des polices web. On les charge par une balise `<link>` vers `fonts.googleapis.com`, puis on les utilise avec `font-family`.

Dans le projet, deux polices se partagent les rôles : **Sora** (géométrique, graisses 600 à 800) pour les titres, avatars et chiffres, **Manrope** (très lisible, 400 à 700) pour le texte courant. Les pages les demandent via `$extraFonts` (voir 5.1), et le CSS les applique :

```
.av2 { font-family: 'Manrope', sans-serif; ... }
.v2-h1 { margin: 0; font-family: 'Sora', sans-serif; font-size: 25px; font-weight: 700; letter-spacing: -0.3px; }
```

- l'URL ne demande que les graisses utilisées (`wght@600;700;800`), pour alléger le téléchargement ;
- `display=swap` affiche d'abord le texte dans une police système puis la remplace : le texte n'est jamais invisible pendant le chargement ;
- `sans-serif` en fin de liste est la police de secours si Google Fonts est inaccessible (connexion coupée).

`header.php` charge aussi la police **Inter** sur toutes les pages : c'est la police de base de `style.css` (`body { font-family: 'Inter', sans-serif; }`), donc celle des pages sans habillage v2, et elle sert de police de secours après Manrope dans certaines règles (`'Manrope', 'Inter', sans-serif`).

5.9 Les icônes Font Awesome

Ce que c'est. Font Awesome est une bibliothèque de plus de 2 000 icônes dessinées comme une police de caractères. Elle est chargée depuis un CDN (réseau de diffusion de contenu) dans `header.php` :

```
<!-- FontAwesome -->
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css">
...
<span class="authv2-icon"><i class="fas fa-envelope"></i></span>
<button type="button" class="authv2-toggle" id="passwordToggle" aria-label="Afficher le mot de
passe">
    <i class="fas fa-eye"></i>
</button>
```

Une icône s'obtient avec deux classes : le style (`fas` = solid) et le nom (`fa-envelope`, `fa-lock`, `fa-eye`, `fa-car` ...). Comme c'est une police, l'icône prend la couleur et la taille du texte via `color` et `font-size`. Les espaces v2 utilisent aussi des icônes SVG directement dans le HTML (menu hamburger, logo Google, navigation), plus nettes et colorables trait par trait (`.av2-navitem.active svg * { stroke: #FFFFFF }`).

5.10 Le versionnement des fichiers contre le cache : `asset_url()`

Le problème. Pour aller vite, le navigateur garde une copie des fichiers CSS et JS (le **cache**). Après une modification des couleurs, un utilisateur pouvait continuer à voir l'ancienne apparence, voire une page cassée (nouveau HTML avec ancien CSS).

La solution (dans `includes/header.php`) : ajouter à l'URL un paramètre `?v=` égal à la date de dernière modification du fichier.

```
function asset_url(string $rel): string {
    $file = __DIR__ . '/../' . ltrim($rel, '/');
    $version = is_file($file) ? (string)filemtime($file) : '';
    return SITE_URL . ltrim($rel, '/') . ($version !== '' ? '?v=' . $version : '');
}
...
<link rel="stylesheet" href="<?php echo h(asset_url('assets/css/style.css')); ?>">
<script src="<?php echo h(asset_url('assets/js/main.js')); ?>"></script>
```

`filemtime()` renvoie la date de modification sous forme de nombre (timestamp Unix). L'URL devient par exemple `assets/css/admin_v2.css?v=1759912345` (valeur d'illustration). Tant que le fichier ne change pas, l'URL est identique et le cache sert la copie (rapide). Dès que le fichier est modifié, le nombre change : pour le navigateur c'est une **nouvelle adresse**, il télécharge donc la nouvelle version. Aucun numéro à incrémenter à la main, et toutes les feuilles de `$extraStylesheets` en profitent automatiquement.

5.11 L'ordre de chargement et la cascade

Le « C » de CSS signifie Cascading (en cascade) : quand deux règles visent le même élément, la plus **spécifique** l'emporte, et à spécificité égale, la **dernière** chargée. C'est pourquoi `header.php` charge dans cet ordre : `style.css` (base commune), `themes.css` (thèmes), puis la feuille de l'espace (`client_v2.css` ...). La charte de l'espace peut ainsi surcharger la base sans `!important`, qui reste rare (une seule occurrence par feuille `*_v2.css`, aucune dans `auth_v2.css`).

À retenir

- HTML = structure et sens (balises sémantiques, formulaires) ; CSS = apparence ; les deux sont séparés.
- Un seul gabarit `includes/header.php` pour toutes les pages, paramétré par `$pageTitle`, `$bodyClass`, `$extraStylesheets`, `$extraFonts`.
- Les attributs `type`, `required`, `placeholder`, `autocomplete` améliorent l'ergonomie mais ne remplacent pas la validation PHP.
- ARIA (`aria-label`, `aria-hidden`, `aria-expanded`) rend les boutons à icône et le menu mobile compréhensibles par les lecteurs d'écran.
- Variables CSS dans `:root` + préfixes (`v2-`, `av2-`, `gv2-`, `tv2-`) = une charte commune aux 4 espaces, sans collision.
- Flexbox pour une dimension (barre latérale + contenu, barres d'outils), Grid pour deux dimensions (cartes, colonnes).
- `@media` (`max-width: 720px`) transforme la barre latérale en tiroir ; JavaScript ne fait que basculer la classe `is-open`.
- Sora (titres) et Manrope (texte) via Google Fonts ; icônes Font Awesome via CDN.
- `asset_url()` ajoute `?v=filemtime` pour forcer le rechargement d'un fichier modifié.

Questions du jury

Q. Pourquoi ne pas avoir utilisé un framework CSS comme Bootstrap ou Tailwind ?

Le projet a une charte graphique propre (barre latérale bleu nuit, cartes arrondies, polices Sora et Manrope) qu'il aurait fallu de toute façon redéfinir par-dessus Bootstrap. Avec du CSS écrit à la main, les feuilles restent légères (environ 220 à 310 lignes par espace), on maîtrise chaque règle et on n'ajoute pas de dépendance. Flexbox, Grid et les variables CSS, aujourd'hui supportés par tous les navigateurs, couvrent ce que les frameworks apportaient autrefois.

Q. Si l'attribut `required` empêche déjà d'envoyer un champ vide, pourquoi révéifier en PHP ?

Parce que tout ce qui se passe dans le navigateur est sous le contrôle de l'utilisateur : il peut supprimer l'attribut avec les outils de développement ou envoyer une requête POST directement, sans formulaire. La validation HTML est un confort, qui évite un aller-retour serveur pour une erreur simple. La validation PHP est la seule qui protège réellement la base de données.

Q. À quoi servent les variables CSS, et comment avez-vous unifié les quatre espaces ?

Une variable comme `--a-accent` est déclarée une fois dans `:root` et réutilisée partout avec `var(--a-accent)` ; changer sa valeur change toute l'interface. Chaque espace a sa feuille `*_v2.css` avec son préfixe, mais les variables portent les mêmes valeurs (bleu `#3956E8`, barre latérale `#131B3E`), ce qui donne une identité visuelle unique. Les préfixes évitent qu'une règle d'un espace modifie un autre espace.

Q. Quelle est la différence entre Flexbox et Grid ? Donnez un exemple de chaque dans votre

application.

Flexbox dispose les éléments sur un seul axe, Grid sur deux axes à la fois. Dans le projet, `.av2-shell` utilise Flexbox pour mettre la barre latérale de largeur fixe à côté du contenu qui prend le reste de la place. Les cartes de statistiques utilisent Grid avec `repeat(4, minmax(0, 1fr))` : quatre colonnes égales, qui passent à deux sous 1200 px de large ; et le corps du tableau de bord (`2fr 1fr`) passe à une seule colonne sous 1100 px.

Q. Comment votre application s'affiche-t-elle sur un téléphone ?

La balise `meta viewport` demande au téléphone d'utiliser sa vraie largeur. Ensuite des requêtes `@media` à 1200, 1100 et 720 px réduisent le nombre de colonnes (quatre cartes de statistiques deviennent deux, les deux colonnes du tableau de bord deviennent une), puis transforment la barre latérale en tiroir caché à gauche. Un bouton hamburger, injecté une seule fois par `main.js` pour les quatre espaces, ouvre ce tiroir en ajoutant la classe `is-open` et met à jour `aria-expanded`.

Q. Qu'est-ce que le `?v=` à la fin des URL de vos fichiers CSS et JS ?

C'est un numéro de version calculé par la fonction `asset_url()` à partir de `filemtime()`, la date de dernière modification du fichier. Le navigateur met les fichiers en cache par adresse : tant que le fichier ne change pas, l'adresse reste identique et la copie locale est réutilisée. Dès qu'on modifie le fichier, le numéro change, l'adresse est nouvelle et le navigateur télécharge la nouvelle version, sans que l'utilisateur ait à vider son cache.

Q. Qu'avez-vous fait pour l'accessibilité ?

On utilise des balises sémantiques (`<aside>`, `<nav>`, `<main>`, `<button>`) et chaque champ a un `<label for>` relié à son `id`. Les boutons qui ne contiennent qu'une icône reçoivent un `aria-label`, les astérisques décoratifs un `aria-hidden="true"`, et le menu mobile indique son état avec `aria-expanded`. On peut aussi fermer le menu avec la touche Échap.

Q. Que se passe-t-il si Google Fonts ou le CDN de Font Awesome sont inaccessibles ?

L'application continue de fonctionner : les polices sont déclarées avec une police de secours (`'Manrope', sans-serif`), donc le texte s'affiche dans la police système, et `display=swap` évite un texte invisible pendant le chargement. Les icônes Font Awesome ne s'afficheraient pas, mais les boutons gardent leur texte ou leur `aria-label`. Pour une version hors ligne, on pourrait héberger ces fichiers dans `assets/`.

6. Serveur et outils : XAMPP, Apache, Composer, PHPUnit, Git

Une application web ne se résume pas à son code. Il faut un **serveur** pour l'exécuter, un **gestionnaire de dépendances** pour installer les bibliothèques, des **tests automatiques** pour vérifier qu'elle fonctionne, et un **outil de versionnement** pour garder l'historique et travailler à plusieurs. Ce chapitre présente les cinq outils utilisés pour SmartAutoTrack et la façon dont ils sont configurés dans le projet.

6.1 XAMPP : tout le serveur sur un seul poste

Ce que c'est. XAMPP est une distribution gratuite qui installe d'un coup les logiciels nécessaires à un site PHP : **A**pache (serveur web), **M**ariaDB/MySQL (base de données), **P**HP (langage) et **P**erl. Le « X » signifie multiplateforme (Windows, Linux, macOS). Le panneau de contrôle XAMPP permet de démarrer et d'arrêter Apache et MySQL d'un clic.

Pourquoi ici. Le projet est développé sous Windows. XAMPP évite d'installer et de relier à la main Apache, PHP et MySQL. Le projet se trouve dans `C:\xampp\htdocs\HCH` : `htdocs` est la racine web d'Apache, donc l'application est accessible à l'adresse `http://localhost/HCH/`. Cette URL est d'ailleurs déclarée dans la configuration des tests :

```
<!-- phpunit.xml -->
<php>
  <const name="HCH_BASE_URL" value="http://localhost/HCH/" />
</php>
```

Comment ça marche. Le navigateur envoie une requête HTTP à Apache. Si le fichier demandé se termine par `.php`, Apache le confie à l'interpréteur PHP, qui exécute le code, interroge MySQL via PDO si besoin, puis renvoie du HTML. Apache transmet ce HTML au navigateur. Les fichiers statiques (CSS, JS, images) sont envoyés directement, sans passer par PHP.

Composant	Rôle dans SmartAutoTrack	Version / emplacement
Apache	Reçoit les requêtes, applique les <code>.htaccess</code> , sert les fichiers	Module de XAMPP, port 80
PHP	Exécute les pages et les scripts AJAX	PHP 8.2.12 (<code>C:\xampp\php\php.exe</code>)
MySQL / MariaDB	Stocke utilisateurs, véhicules, réparations, paiements...	Administré avec phpMyAdmin
phpMyAdmin	Interface web pour importer les scripts <code>.sql</code> et consulter les tables	<code>http://localhost/phpmyadmin</code>

Piège fréquent

XAMPP est un environnement de **développement** : son compte MySQL `root` est souvent sans mot de passe et phpMyAdmin est ouvert. Il ne faut pas le présenter comme un serveur de production. En production, on utilise un hébergement avec Apache (ou Nginx), PHP et MySQL correctement sécurisés, et HTTPS.

6.2 Apache et les fichiers .htaccess

Ce que c'est. Un fichier `.htaccess` est un fichier de configuration qu'Apache lit dans chaque dossier. Il permet de changer le comportement du serveur pour ce dossier et ses sous-dossiers, sans toucher à la configuration globale : interdire l'accès, ajouter des en-têtes HTTP, réécrire des URL, choisir la page d'index.

Pourquoi ici. Certains dossiers ne doivent **jamais** être lisibles depuis un navigateur : la configuration (identifiants de base de données, clés CamPay et OpenRouter), les scripts d'administration, les documents envoyés par les utilisateurs. Le projet contient six fichiers `.htaccess` : à la racine, dans `config/`, `scripts/`, `uploads/`, `uploads/techniciens/` et `api/`.

Protéger un dossier entier

```
# config/.htaccess
# Aucun fichier de configuration ne doit être accessible depuis le web
<IfModule mod_authz_core.c>
    Require all denied
</IfModule>
<IfModule !mod_authz_core.c>
    Order deny,allow
    Deny from all
</IfModule>
```

`Require all denied` est la syntaxe d'Apache 2.4 : toute requête HTTP vers ce dossier reçoit une erreur **403 Forbidden**. Le bloc `<IfModule !mod_authz_core.c>` fournit l'équivalent pour Apache 2.2 (`Deny from all`), au cas où l'hébergeur serait ancien. Attention : cela bloque seulement l'accès **web**. PHP, lui, peut toujours lire ces fichiers avec `require`, et c'est bien ce qu'on veut.

Pour les documents des techniciens, le principe va plus loin : le dossier est fermé et les fichiers passent par un script PHP qui vérifie la session et les droits.

```
# uploads/techniciens/.htaccess
# Aucun accès direct : les documents sont servis par ajax/download_document.php
# (vérification de session et des droits).
Options -Indexes
<IfModule mod_authz_core.c>
    Require all denied
</IfModule>
```

`Options -Indexes` empêche Apache d'afficher la liste des fichiers d'un dossier qui n'a pas de page d'index.

Bloquer des fichiers sensibles à la racine

```
# .htaccess (racine)
<FilesMatch "(^tmp_.*\.txt|\. (sql|lock|phar|md)|^composer(-setup)?\.
(json|php)|^phpunit\.xml|^\.gitignore)$">
    <IfModule mod_authz_core.c>
        Require all denied
    </IfModule>
</FilesMatch>
```

`FilesMatch` applique la règle aux fichiers dont le nom correspond à une **expression régulière**. Ici on bloque les scripts SQL (qui révèlent la structure de la base), `composer.lock`, `composer.phar`, les fichiers Markdown, `phpunit.xml`, `.gitignore` et les fichiers de cookies de test `tmp_*.txt`.

En-têtes de sécurité et de cache

```
# .htaccess (racine)
<IfModule mod_headers.c>
    Header set X-Content-Type-Options "nosniff"
    Header set X-Frame-Options "SAMEORIGIN"
    Header set Referrer-Policy "strict-origin-when-cross-origin"

    # Pages dynamiques (données personnelles) : jamais mises en cache
    <FilesMatch "\\.php$">
        Header set Cache-Control "no-store, no-cache, must-revalidate"
    </FilesMatch>
    <FilesMatch "\\. (css|js|png|jpe?g|gif|svg|webp|ico|woff2?)$">
        Header set Cache-Control "no-cache"
    </FilesMatch>
</IfModule>
```

En-tête	Effet	Attaque évitée
X-Content-Type-Options: nosniff	Le navigateur respecte le type annoncé et ne « devine » pas	Un fichier texte interprété comme script
X-Frame-Options: SAMEORIGIN	Le site ne peut être affiché dans une iframe que par lui-même	Clickjacking (faux bouton superposé)
Referrer-Policy	Limite l'URL transmise aux sites externes	Fuite d'URL internes vers des tiers
Cache-Control: no-store (PHP)	Les pages avec données personnelles ne sont pas gardées en cache	Bouton « Précédent » qui réaffiche un tableau de bord après déconnexion
Cache-Control: no-cache (statiques)	Le navigateur garde le fichier mais revalide (ETag) à chaque chargement	Ancien CSS/JS affiché après une mise à jour

Enfin, `DirectoryIndex index.php` indique la page servie quand on demande seulement `http://localhost/HCH/`.

À retenir

- XAMPP = Apache + MySQL/MariaDB + PHP installés ensemble, pour le développement local.
- `.htaccess` configure Apache dossier par dossier ; `Require all denied` renvoie 403.
- `config/`, `scripts/` et `uploads/` sont fermés au web ; PHP peut toujours les lire.
- Les documents privés sont servis par un script PHP qui vérifie la session, jamais par lien direct.
- En-têtes : `nosniff`, `SAMEORIGIN`, `Referrer-Policy`, et pas de cache pour les pages PHP.

6.3 Composer : le gestionnaire de dépendances PHP

Ce que c'est. Composer est l'outil standard pour installer des bibliothèques PHP (des « paquets », publiés sur Packagist). On déclare dans `composer.json` ce dont le projet a besoin ; Composer télécharge les paquets et leurs propres dépendances dans le dossier `vendor/` et génère un **autoloader** (`vendor/autoload.php`) qui charge les classes automatiquement.

Pourquoi ici. SmartAutoTrack n'utilise **aucune bibliothèque en production** : la section `require` est vide. Les appels à CamPay, OpenRouter, Brevo et Google sont écrits en PHP natif avec l'extension cURL (fonction `httpJsonRequest()` de `includes/http_client.php`), et les PDF sont produits par un petit générateur maison,

PdfLite (includes/pdf_lite.php), écrit en PHP pur. Composer sert uniquement aux **outils de développement** : PHPUnit pour les tests et Guzzle (client HTTP) pour les tests fonctionnels.

```
// composer.json
{
  "name": "hch/smartautotrack",
  "require": {},
  "require-dev": {
    "phpunit/phpunit": "^9.6",
    "guzzlehttp/guzzle": "^7.9"
  },
  "autoload": { "psr-4": { "HCH\\": "" } },
  "scripts": {
    "test": "phpunit -c phpunit.xml",
    "test:unit": "phpunit -c phpunit.xml --testsuite Unit",
    "test:integration": "...", "test:functional": "..."
  }
}
```

Élément	Signification
<code>require</code>	Dépendances nécessaires en production (vide ici)
<code>require-dev</code>	Dépendances de développement seulement ; ignorées avec <code>composer install --no-dev</code>
<code>^9.6</code>	Contrainte de version : 9.6 ou plus récente, mais inférieure à 10 (pas de changement majeur)
<code>autoload psr-4</code>	Règle de correspondance espace de noms → dossier pour charger les classes
<code>scripts</code>	Raccourcis : <code>composer test</code> lance toute la suite de tests
<code>composer.lock</code>	Versions exactes installées (ex. PHPUnit 9.6.29), pour que tout le monde ait les mêmes

Pourquoi vendor/ n'est pas versionné

Le fichier `.gitignore` contient la ligne `/vendor/`. Le dossier `vendor/` (PHPUnit, Guzzle, Symfony, Doctrine...) est volumineux, ne contient pas notre code et peut être **reconstruit** à l'identique avec `composer install` grâce à `composer.lock`. Le versionner alourdirait le dépôt et créerait des conflits inutiles. On versionne donc la **recette** (`composer.json` + `composer.lock`), pas le résultat.

```
# Après un git clone, pour retrouver les dépendances :
composer install      # lit composer.lock et remplit vendor/
composer update       # cherche des versions plus récentes et réécrit composer.lock
```

Piège fréquent

Confondre `install` et `update`. `composer install` respecte le fichier `lock` : c'est la commande à utiliser après un clone. `composer update` change les versions et modifie `composer.lock` : on ne l'utilise que lorsqu'on veut volontairement mettre à jour.

6.4 PHPUnit : les tests automatiques

Ce que c'est. PHPUnit est le framework de tests de référence en PHP. Un test est une méthode qui appelle une fonction du projet et **vérifie** le résultat avec des assertions (`assertSame`, `assertNull`, `assertTrue`...). Si une assertion échoue, PHPUnit signale le test en rouge.

Pourquoi ici. Les règles métier (politique de mot de passe, jeton CSRF, limitation des tentatives de connexion, calculs d'abonnement, échéances d'entretien, signature des webhooks CamPay, génération des PDF) doivent rester correctes quand on modifie le code. Relancer les tests après chaque modification permet de détecter immédiatement une régression.

Organisation

```
<!-- phpunit.xml -->
<phpunit bootstrap="tests/bootstrap.php" colors="true" cacheResultFile="tests/.phpunit.cache">
  <testsuites>
    <testsuite name="Unit"><directory>tests/Unit</directory></testsuite>
    <testsuite name="Integration"><directory>tests/Integration</directory></testsuite>
    <testsuite name="Functional"><directory>tests/Functional</directory></testsuite>
  </testsuites>
</phpunit>
```

Suite	Ce qui est testé	Exemples de fichiers
Unit (18 fichiers)	Une fonction isolée, sans base de données ni réseau	PasswordPolicyTest, Csrftest, CampayTest, MaintenanceTest, PdfLiteTest, AiTest
Integration (3 fichiers)	Le code avec la vraie base MySQL	DatabaseConnectionTest, SchemaTest, RolesTest
Functional (1 fichier)	L'application à travers HTTP, comme un navigateur (Guzzle)	HomepageTest

Le fichier `tests/bootstrap.php` est exécuté avant les tests : il charge `config/config.php` et `config/database.php` pour que les fonctions du projet soient disponibles.

Un test réel

```
// tests/Unit/PasswordPolicyTest.php
require_once __DIR__ . '/../../includes/password_policy.php';

final class PasswordPolicyTest extends TestCase
{
    public function testAcceptsAPasswordMeetingEveryRule(): void
    {
        $this->assertSame([], passwordPolicyMissing('Garage@2026'));
        $this->assertNull(passwordPolicyError('Garage@2026'));
    }

    /** @dataProvider weakPasswords */
    public function testRejectsAPasswordMissingOneRule(string $password, string $missingRule):
void
    {
        $this->assertSame([$missingRule], passwordPolicyMissing($password));
    }
}
```

```

public static function weakPasswords(): array
{
    return [
        'trop court'      => ['Ga@2026', 'au moins ' . PASSWORD_MIN_LENGTH . ' caractères'],
        'sans minuscule' => ['GARAGE@2026', 'une lettre minuscule'],
        'sans majuscule' => ['garage@2026', 'une lettre majuscule'],
        'sans chiffre'   => ['Garage@Yaounde', 'un chiffre'],
        // ... (cas « sans caractère spécial »)
    ];
}

```

À lire ainsi : la classe hérite de `TestCase` ; chaque méthode qui commence par `test` est un test. Le premier vérifie qu'un mot de passe conforme ne déclenche aucune erreur. Le second utilise un **data provider** : la même méthode est exécutée une fois par ligne du tableau `weakPasswords()`, chaque mot de passe faible devant manquer exactement la règle indiquée. On écrit une méthode et on obtient plusieurs tests.

Le test d'intégration montre une autre technique : s'il n'y a pas de base disponible, le test est **ignoré** au lieu d'échouer.

```

// tests/Integration/DatabaseConnectionTest.php
$db = new Database();
$conn = $db->getConnection();
if (!$conn) {
    $this->markTestSkipped('Connexion DB indisponible dans cet environnement.');
```

Lancer les tests et résultat actuel

```

C:\xampp\php\php.exe vendor/bin/phpunit -c phpunit.xml           # tout
C:\xampp\php\php.exe vendor/bin/phpunit -c phpunit.xml --testsuite Unit # unitaires
composer test                                                    # raccourci

```

```

PHPUnit 9.6.29 by Sebastian Bergmann and contributors.
..... 63 / 207 ( 30%)
..... 126 / 207 ( 60%)
..... 189 / 207 ( 91%)
..... 207 / 207 (100%)
OK (207 tests, 950 assertions)

```

État mesuré au moment de la rédaction (PHP 8.2.12) : **207 tests unitaires** (950 assertions) et **234 tests au total** avec les suites Integration et Functional (1002 assertions), tous verts. Chaque point représente un test réussi ; un **F** signale un échec, un **E** une erreur, un **S** un test ignoré.

Piège fréquent

Dire « l'application n'a pas de bug car tous les tests passent ». Les tests prouvent seulement que les cas **prévus** fonctionnent. Ils réduisent le risque de régression, mais ne remplacent pas les tests manuels de l'interface ni une revue de sécurité.

À retenir

- Composer installe les dépendances dans `vendor/` d'après `composer.json` et `composer.lock`.
- Ici, seuls PHPUnit et Guzzle sont installés, en `require-dev` : aucune dépendance en production.
- `vendor/` n'est pas versionné : on le reconstruit avec `composer install`.
- Trois suites de tests : Unit, Integration, Functional ; 207 tests unitaires, 234 au total.
- Un test = appel d'une fonction + assertion ; `@dataProvider` rejoue un test sur plusieurs jeux de données.

6.5 Git et GitHub : versionner et collaborer

Ce que c'est. **Git** est un logiciel de gestion de versions : il enregistre l'historique de tous les fichiers sous forme de commits (photos datées et commentées du projet). **GitHub** est un site qui héberge une copie du dépôt Git en ligne. Le dépôt distant du projet s'appelle `origin` et pointe vers `https://github.com/Charles-625/SmartAutoTrack.git`.

Pourquoi ici. Git permet de revenir à une version antérieure si une modification casse quelque chose, de savoir qui a changé quoi et pourquoi, de travailler à plusieurs sans s'écraser mutuellement, et de sauvegarder le travail sur GitHub.

Les commandes essentielles

Commande	Rôle
<code>git status</code>	Voir les fichiers modifiés, ajoutés, non suivis
<code>git add fichier</code>	Placer les modifications dans la zone de préparation (staging)
<code>git commit -m "message"</code>	Enregistrer un commit local avec un message explicatif
<code>git push origin main</code>	Envoyer les commits locaux vers GitHub
<code>git pull origin main</code>	Récupérer les commits de GitHub et les fusionner localement (<code>fetch</code> + <code>merge</code>)
<code>git branch / git switch -c nom</code>	Lister / créer une branche
<code>git merge nom</code>	Fusionner une branche dans la branche courante
<code>git log --oneline --graph</code>	Afficher l'historique sous forme de graphe

Branches et fusions dans l'historique réel

Une **branche** est une ligne de développement parallèle : on peut développer une fonctionnalité sans toucher à `main`, puis la **fusionner** (merge) quand elle est prête. Le dépôt contient la branche principale `main`, une branche distante `claude/gallant-euler-6y5qb0` et une branche locale de sauvegarde `backup-before-pull`.
Extrait de l'historique (25 commits au total) :

```
* 7cdab83 Nouveau logo SAT, échéances et rappels d'entretien, modèle IA de secours
* a05b7fe Fusion de claude/gallant-euler-6y5qb0 : assistant IA sur OpenRouter, ...
|\
| * 39eacda Google OAuth : session conservée au retour de Google
| * ca60b07 Nouveau logo vectoriel SmartAutoTrack
| * 49a5961 Assistant IA : passage de Hugging Face à OpenRouter
* | 829f03b Charte graphique commune, fenêtre « Affecter la demande », ...
|/
* 2facf63 Fusion de origin/main : mot de passe oublié (Brevo) et connexion Google
```

On y voit deux lignes parallèles : sur la branche, l'assistant IA passe de Hugging Face à OpenRouter ; sur `main`, on travaille sur la charte graphique. Le commit `a05b7fe` est un **commit de fusion** qui réunit les deux. Si les deux côtés avaient modifié les mêmes lignes d'un fichier, Git aurait signalé un **conflit** à résoudre à la main (c'est arrivé au commit `9aed507` « Resolve merge conflicts »). La branche `backup-before-pull` illustre une bonne pratique : créer une sauvegarde avant un `pull` risqué.

.gitignore et config/local.php

Le fichier `.gitignore` liste ce que Git ne doit **pas** suivre :

```
# .gitignore (extrait)
config/local.php
uploads/techniciens/*
!uploads/techniciens/.htaccess
!uploads/techniciens/index.php
tmp_*.txt

# Dependencies
/vendor/
# Composer
composer.phar
composer-setup.php
```

La ligne la plus importante est `config/local.php`. Ce fichier contient les **secrets** : identifiants MySQL, clés CamPay, clé OpenRouter, clés Brevo et Google. Un dépôt GitHub peut être public ou partagé, et l'historique Git garde tout pour toujours : un secret committé une seule fois est considéré comme compromis. On versionne donc à la place un **modèle** vide, `config/local.example.php` :

```
// config/local.example.php (modèle, sans valeur secrète)
/**
 * Modèle de configuration locale.
 * Copiez ce fichier en config/local.php et renseignez vos identifiants.
 * Une variable d'environnement du même nom (si définie) a priorité.
 */
return [
    'CAMPAY_APP_ID' => '',
    'CAMPAY_TOKEN' => '',
    'OPENROUTER_API_KEY' => '', // clé sk-or-v1-... créée sur https://openrouter.ai/keys
    // ...
];
```

Le code lit ces valeurs par une seule fonction, `appConfig()`, qui donne la priorité aux variables d'environnement (pratique sur un vrai serveur) puis se rabat sur `local.php` :

```
// config/config.php
function appConfig(string $key, $default = null) {
    $env = getenv($key);
    if ($env !== false && $env !== '') {
        return $env;
    }
    static $local = null;
    if ($local === null) {
        $file = __DIR__ . '/local.php';
        $loaded = is_file($file) ? require $file : [];
        $local = is_array($loaded) ? $loaded : [];
    }
    return $local[$key] ?? $default;
}
```

Les deux protections sont complémentaires : `.gitignore` empêche le secret d'arriver sur GitHub, et `config/.htaccess` empêche de le lire par le navigateur. De même, les documents envoyés par les techniciens (`uploads/techniciens/*`) sont des données personnelles et ne sont pas versionnés, sauf le `.htaccess` et l'`index.php` de protection (le `!` crée une exception).

Piège fréquent

Ajouter un fichier au `.gitignore` **après** l'avoir déjà committé ne le retire pas du dépôt : Git continue à le suivre. Il faut aussi faire `git rm --cached fichier`. C'est le cas dans le projet pour `tests/.phpunit.cache` : il figure dans `.gitignore` mais reste suivi, d'où sa présence en « modifié » dans `git status` après chaque lancement des tests.

À retenir

- Git = historique local en commits ; GitHub = copie distante (`origin`) pour sauvegarder et partager.
- Cycle : `add` → `commit` → `push` ; `pull` pour récupérer le travail des autres.
- Branche = développement parallèle ; `merge` la réunit à `main` ; conflit si les mêmes lignes ont changé.
- `config/local.php` (secrets) et `vendor/` sont exclus par `.gitignore` ; `local.example.php` sert de modèle.
- `appConfig()` : variable d'environnement d'abord, puis `local.php`.

Questions du jury

Q. Pourquoi avoir utilisé XAMPP, et votre application peut-elle tourner ailleurs ?

XAMPP installe en une fois Apache, MySQL et PHP sous Windows, ce qui simplifie le développement local. L'application n'en dépend pas : c'est du PHP 8 standard avec PDO et MySQL, donc elle fonctionne sur n'importe quel hébergement Apache/PHP/MySQL. Il suffit d'importer la base, de créer `config/local.php` (ou des variables d'environnement) et d'activer HTTPS.

Q. Comment empêchez-vous un visiteur de télécharger votre fichier de configuration ou les documents des techniciens ?

Les dossiers `config/`, `scripts/` et `uploads/` contiennent un `.htaccess` avec `Require all denied` : Apache renvoie 403 à toute requête directe. PHP peut toujours lire ces fichiers côté serveur. Les

documents des techniciens sont servis par `ajax/download_document.php`, qui vérifie la session et les droits avant d'envoyer le fichier.

Q. À quoi servent les en-têtes X-Frame-Options et Cache-Control dans votre .htaccess ?

X-Frame-Options: SAMEORIGIN interdit à un autre site d'afficher nos pages dans une iframe, ce qui bloque le clickjacking. Cache-Control: no-store sur les pages PHP empêche le navigateur de conserver des pages contenant des données personnelles, par exemple un tableau de bord réaffiché avec le bouton Précédent après déconnexion. Pour les CSS et JS, no-cache force une revalidation pour que les mises à jour soient visibles.

Q. Pourquoi le dossier vendor n'est-il pas sur GitHub ?

Il contient uniquement du code tiers (PHPUnit, Guzzle et leurs dépendances), lourd et reconstituible. On versionne `composer.json` et `composer.lock` ; n'importe qui retrouve exactement les mêmes versions avec `composer install`. Cela garde le dépôt léger et évite des conflits sur des fichiers qu'on n'a pas écrits.

Q. Votre application utilise-t-elle des bibliothèques externes en production ?

Non, la section `require` de `composer.json` est vide. Les intégrations CamPay, OpenRouter, Brevo et Google OAuth sont écrites avec cURL, et les PDF sont générés par un module maison. Composer ne sert qu'aux outils de développement déclarés en `require-dev` : PHPUnit et Guzzle.

Q. Comment testez-vous votre application ? Combien de tests avez-vous ?

Avec PHPUnit 9.6, en trois suites : unitaires (fonctions isolées), intégration (avec la vraie base) et fonctionnels (requêtes HTTP avec Guzzle). Au moment de la rédaction, il y a 207 tests unitaires et 234 au total, tous verts. On les lance avec `vendor/bin/phpunit -c phpunit.xml` ou `composer test`, après chaque modification importante.

Q. Quelle est la différence entre un test unitaire et un test d'intégration ?

Un test unitaire vérifie une seule fonction, sans base ni réseau, par exemple que `passwordPolicyMissing('garage@2026')` signale l'absence de majuscule. Il est rapide et déterministe. Un test d'intégration vérifie que plusieurs éléments fonctionnent ensemble, par exemple la connexion PDO à MySQL et la présence des tables ; s'il n'y a pas de base, notre test est marqué « ignoré » avec `markTestSkipped`.

Q. Où sont vos clés CamPay et OpenRouter, et comment éviter qu'elles fuient ?

Elles sont dans `config/local.php`, exclu de Git par `.gitignore` et fermé au web par `config/.htaccess`. Le dépôt contient seulement `local.example.php`, un modèle sans valeur. Le code les lit via `appConfig()`, qui accepte aussi des variables d'environnement, la méthode recommandée sur un serveur de production. Si une clé avait été committée, il faudrait la révoquer et en générer une nouvelle.

7. Services externes et API

SmartAutoTrack ne fait pas tout lui-même : il confie quatre tâches à des services spécialisés. **CamPay** encaisse les paiements Mobile Money (MTN, Orange), **OpenRouter** fournit les modèles d'intelligence artificielle de l'assistant, **Brevo** envoie les emails (mot de passe oublié, rappels d'échéances) et **Google** permet de se connecter avec un compte Google. L'application leur parle à travers des **API**. Ce chapitre explique les notions de base, puis chaque intégration avec un extrait réel du code.

7.1 Les notions de base : API REST, HTTP, JSON, clé d'API

API et REST

Une **API** (Application Programming Interface) est une « porte d'entrée » qu'un service ouvre aux programmes : au lieu d'un humain qui clique sur un site, c'est notre code PHP qui envoie une requête et reçoit une réponse lisible par une machine. Une API **REST** suit une convention simple : chaque ressource a une URL (par exemple `/api/transaction/{référence}/` chez CamPay) et on agit dessus avec les **verbes HTTP** : **GET** pour lire, **POST** pour créer ou déclencher une action. Le serveur ne garde pas de « conversation » entre deux appels (stateless) : chaque requête doit porter son authentification.

HTTP : requête, en-têtes, code de statut

Une requête HTTP contient une méthode, une URL, des **en-têtes** (`Authorization`, `Content-Type` ...) et éventuellement un **corps**. La réponse contient un **code de statut** qui résume le résultat :

Code	Sens	Exemple dans le projet
200 / 2xx	Succès	Réponse normale de CamPay, OpenRouter, Brevo
400	Requête mal formée	Webhook sans <code>reference</code>
401	Non authentifié (clé refusée)	Clé OpenRouter invalide : on arrête tout
402	Paiement requis	Modèle IA payant sans crédit : on passe au suivant
403	Interdit	Notre webhook rejette une signature invalide
404	Introuvable	Modèle IA retiré, paiement d'un autre client
429	Trop de requêtes	Modèle gratuit saturé, limite de débit de l'assistant
5xx	Panne côté serveur	Fournisseur IA en panne

JSON

JSON (JavaScript Object Notation) est le format texte des échanges : objets `{"clé": valeur}`, tableaux `[...]`, chaînes, nombres, booléens. En PHP, `json_encode()` transforme un tableau en texte JSON et `json_decode($texte, true)` fait l'inverse (le `true` donne un tableau associatif plutôt qu'un objet).

Clé d'API et secret

Une **clé d'API** identifie et authentifie notre application auprès du service : elle est envoyée à chaque appel dans un en-tête (`Authorization: Bearer ...` pour OpenRouter, `Authorization: Token ...` pour CamPay, `api-key: ...` pour Brevo). C'est un **secret** : quiconque la possède peut utiliser le service à notre nom (et à nos frais). Elle ne doit donc exister que côté serveur.

Un client HTTP commun : cURL

Tous les appels sortants passent par l'extension PHP **cURL**. Pour CamPay et OpenRouter, le projet a factorisé l'appel dans une seule fonction qui envoie du JSON et renvoie le statut, les données décodées et le texte brut.

```
// includes/http_client.php
function httpJsonRequest(string $method, string $url, array $headers = [], $body = null, int $timeout = 30): array {
    $ch = curl_init($url);
    $httpHeaders = ['Accept: application/json'];
    foreach ($headers as $name => $value) {
        $httpHeaders[] = $name . ': ' . $value;
    }
    if ($body !== null) {
        $httpHeaders[] = 'Content-Type: application/json';
        curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode($body, JSON_UNESCAPED_UNICODE));
    }
    // ... CURLOPT_CUSTOMREQUEST, CURLOPT_TIMEOUT ...
    $raw = curl_exec($ch);
    $status = (int)curl_getinfo($ch, CURLINFO_HTTP_CODE);
    // ...
    return ['status' => $status, 'data' => json_decode($raw, true), 'raw' => (string)$raw];
}
```

Le **délai maximal** (`CURLOPT_TIMEOUT`) est important : sans lui, une API lente bloquerait la page du client indéfiniment. Si le serveur distant est injoignable, la fonction lève une `RuntimeException`, que chaque module transforme en message compréhensible.

À retenir

- API REST = URL par ressource + verbes HTTP + réponse JSON, sans état entre deux appels.
- Le code de statut HTTP dit si l'appel a réussi ; le projet réagit différemment à 401, 402, 429, 5xx.
- La clé d'API est un secret serveur ; elle voyage dans un en-tête HTTP, jamais dans le navigateur.

7.2 CamPay : le paiement Mobile Money

Pourquoi CamPay ? Au Cameroun, le moyen de paiement le plus répandu est le Mobile Money (MTN MoMo, Orange Money), pas la carte bancaire. CamPay est un agrégateur camerounais qui offre une seule API pour les deux opérateurs, en francs CFA (XAF). Le client paie une réparation terminée ou un abonnement Premium sans quitter l'application.

Le déroulement d'un paiement

1. **Lancement** : `paymentStart()` (`includes/payments.php`) vérifie le numéro, crée une ligne `paiement` au statut `EN_ATTENTE`, puis appelle `campayCollect()`.
2. **Collect** : CamPay envoie au téléphone du client une invite ; le client tape son code secret Mobile Money sur son téléphone (l'application ne le voit jamais).
3. **Suivi du statut** : la page interroge toutes les 5 secondes `ajax/campay_status.php`, qui demande à CamPay l'état de la transaction.
4. **Webhook** : en parallèle, CamPay prévient lui-même notre serveur à la fin de la transaction (`webhooks/campay.php`).
5. **Résultat** : `SUCCESSFUL` → `PAYE` (définitif, notifications, reçu PDF) ; `FAILED` → `ECHOUÉ`.

Authentification et appel « collect »

CamPay accepte soit un jeton permanent (`CAMPAY_TOKEN`), soit un couple identifiant / mot de passe échangé contre un jeton temporaire sur `/api/token/`. Ensuite, chaque appel porte l'en-tête `Authorization: Token ...`.

```
// includes/campay.php
function campayCollect(int $amount, string $phone, string $description, string
$externalReference): array {
    if (campayIsSimulation()) {
        return ['reference' => 'SIM-' . $externalReference, 'ussd_code' => null, 'operator' =>
'SIMULATION'];
    }
    $data = campayRequest('POST', '/api/collect/', [
        'amount' => (string)$amount,
        'currency' => 'XAF',
        'from' => $phone,
        'description' => mb_substr($description, 0, 100),
        'external_reference' => $externalReference,
    ]);
    // ... vérifie que $data['reference'] existe
}
```

Deux références identifient un paiement : la **référence externe** générée par nous (`'SAT-' . $repairId . '-' . bin2hex(random_bytes(6))`) et la **référence CamPay** renvoyée par l'API. Elles sont toutes deux stockées en base, ce qui permet de retrouver le paiement quand CamPay nous rappelle. Le numéro est aussi normalisé au format `2376XXXXXXX` par `campayNormalizePhone()`.

Le webhook signé

Un **webhook** est l'inverse d'un appel d'API : c'est le service qui appelle notre URL quand un événement se produit. Problème : cette URL est publique, n'importe qui pourrait l'appeler en prétendant « le paiement X est réussi ». CamPay joint donc une **signature** : un **JWT** signé en **HMAC-SHA256** avec une clé partagée (`CAMPAY_WEBHOOK_KEY`) que seuls CamPay et nous connaissons.

```
// includes/campay.php - campayVerifyWebhookSignature()
[$header64, $payload64, $signature64] = explode('.', $jwt);
$header = json_decode(campayBase64UrlDecode($header64), true);
if (!is_array($header) || ($header['alg'] ?? '') !== 'HS256') {
    return false;
}
$expected = hash_hmac('sha256', $header64 . '.' . $payload64, $key, true);
if (!hash_equals($expected, campayBase64UrlDecode($signature64))) {
    return false;
}
$payload = json_decode(campayBase64UrlDecode($payload64), true);
if (is_array($payload) && isset($payload['exp']) && (int)$payload['exp'] < ($now ?? time())) {
    return false; // signature expirée
}
```

On recalcule la signature avec notre clé et on compare avec `hash_equals()` (comparaison en temps constant, qui ne révèle rien par sa durée). On impose aussi l'algorithme `HS256` et on refuse un jeton expiré. Mais le projet va plus loin : même avec une signature valide, **il ne croit pas le statut reçu**. Il redemande le statut à CamPay :

```
// webhooks/campay.php
if (!campayVerifyWebhookSignature((string)($data['signature'] ?? ''))) {
    http_response_code(403);
    exit(json_encode(['status' => 'rejected']));
}
// ...
// Le statut appliqué est celui que CamPay renvoie pour cette référence,
// pas celui des paramètres reçus : une signature rejouée ne suffit pas.
$statut = paymentApplyCampayStatus($conn, $paiement, campayTransactionStatus((string)
($paiement['referenceCampay'] ?: $reference)));
```

Idempotence : webhook et suivi peuvent arriver en même temps

Le webhook et le suivi AJAX peuvent confirmer le même paiement à la même seconde.

`paymentApplyCampayStatus()` est **idempotent** (l'appliquer deux fois donne le même résultat) : la mise à jour SQL contient `WHERE idPaiement = ? AND statut <> 'PAYE'`, et seule la requête pour laquelle `rowCount() === 1` envoie les notifications. Elle contrôle aussi le **montant** confirmé : il est comparé au montant attendu (`campayChargedAmount()` du montant dû, donc plafonné en démo) ; un montant différent passe le paiement en `ECHOUÉ` pour vérification manuelle.

Mode démo et mode simulation

- **Démo** (`CAMPAY_USE_DEMO`, activé par défaut) : on appelle `https://demo.campay.net`. Les échanges sont réels, mais la démo refuse les montants de plus de 25 XAF ; `campayChargedAmount()` plafonne donc le montant débité, alors que le montant dû enregistré en base reste le vrai coût.
- **Simulation** (`CAMPAY_SIMULATION`) : aucun appel réseau, chaque paiement réussit. Ce mode sert à développer hors ligne.
- **Production** : `https://www.campay.net`, argent réel.

Piège fréquent

En local (XAMPP sur `localhost`), CamPay **ne peut pas joindre** le webhook : votre machine n'est pas accessible depuis Internet. C'est pour cela que le projet ne dépend pas uniquement du webhook : le suivi par `ajax/campay_status.php` fait passer le paiement à `PAYE` même sans lui. Le webhook prend le relais quand le client ferme la page avant la confirmation.

7.3 OpenRouter : l'assistant IA

OpenRouter est une passerelle qui donne accès à des centaines de modèles de langage (de plusieurs fournisseurs) à travers **une seule API**, compatible avec celle d'OpenAI (`/chat/completions`). Changer de modèle revient à changer une chaîne de caractères dans la configuration. Plusieurs modèles sont **gratuits** (identifiant terminé par `:free`), ce qui convient à un projet étudiant.

L'appel

```
// includes/ai.php - aiOpenRouterRequest()
return httpJsonRequest(
    'POST',
    (string)appConfig('OPENROUTER_API_URL', 'https://openrouter.ai/api/v1/chat/completions'),
    [
        'Authorization' => 'Bearer ' . trim((string)appConfig('OPENROUTER_API_KEY')),
        'HTTP-Referer' => SITE_URL,
        'X-Title' => SITE_NAME,
    ],
    [
        'model' => $model,
        'messages' => $messages,
        'max_tokens' => $maxTokens,
        'temperature' => 0.4,
    ],
    AI_MODEL_TIMEOUT
);
```

Le corps contient la liste `messages` : un message `system` (consignes + contexte de l'utilisateur), l'historique récent (10 messages gardés en session), puis la question `user`. `temperature` à 0,4 rend les réponses plutôt sobres et stables ; `max_tokens` limite leur longueur. La réponse se lit dans `choices[0].message.content`.

Modèles gratuits et modèles de secours

Les modèles gratuits sont souvent **saturés** (429) ou **retirés** sans préavis (404). Le projet essaie donc une liste ordonnée : le modèle principal (`OPENROUTER_MODEL`), puis les modèles de secours (`OPENROUTER_FALLBACK_MODELS`), quatre au maximum (`AI_MAX_MODELS`).

```
// includes/ai.php - aiChat()
foreach ($models as $model) {
    try {
        // ... $response = $transport($model, $messages, $maxTokens);
        return aiParseResponse($model, $response);
    } catch (AiException $e) {
        if (!$e->retryable) {
            throw $e;          // ex. clé refusée (401) : inutile d'essayer un autre modèle
        }
        $failures[] = $e->detail;
        $last = $e;
    }
}
```

La distinction est fine : une clé refusée (401) vaut pour tous les modèles, donc on s'arrête ; un 402, 403, 404, 429, 5xx ou une réponse vide ne concerne que ce modèle, donc on passe au suivant. Le paramètre `$transport` peut être remplacé dans les tests unitaires, ce qui permet de tester cette logique sans appeler Internet.

Pourquoi l'appel se fait côté serveur

- **Protéger la clé** : un appel depuis le JavaScript du navigateur exposerait `OPENROUTER_API_KEY` à tout visiteur (outils de développement, onglet Réseau).
- **Contrôler les données** : c'est le serveur qui construit le contexte (`aiClientContext()` / `aiAdminContext()`) ; un client ne reçoit que ses véhicules, l'admin seulement des totaux.

- **Limiter l'usage** : longueur maximale (1 000 caractères), limite de débit (20 messages / 10 minutes, `aiAllowMessage()`) et quota quotidien pour la formule gratuite.
- **Fiabilité des faits** : les échéances (vidange, assurance...) sont calculées par l'application et données au modèle déjà calculées ; le modèle est chargé de les rappeler sans les recalculer (un modèle de langage peut toujours se tromper, d'où ce choix de lui fournir les dates toutes faites).

Côté navigateur, `assets/js/assistant.js` envoie seulement la question à `ajax/assistant_chat.php` (POST + jeton CSRF), qui fait tout le reste.

Piège fréquent

Dire « l'IA connaît les données de l'utilisateur » est inexact. Un modèle de langage ne lit pas la base de données : il ne connaît que le texte qu'on lui envoie dans le message `system`. C'est justement ce qui permet de maîtriser ce qu'il voit.

7.4 Brevo : l'envoi d'emails

Envoyer un email avec la fonction `mail()` de PHP depuis XAMPP ne marche pas sans serveur SMTP, et les messages finissent souvent en spam. **Brevo** (ex-Sendinblue) est un service d'emails **transactionnels** (emails déclenchés par une action : lien de réinitialisation du mot de passe, rappel d'échéance) qui gère la délivrabilité. On l'appelle par son API HTTP v3.

```
// includes/mailer.php - sendBrevoEmail()
$ch = curl_init('https://api.brevo.com/v3/smtp/email');
curl_setopt_array($ch, [
    CURLOPT_POST => true,
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_HTTPHEADER => [
        'accept: application/json',
        'api-key: ' . $apiKey,
        'content-type: application/json',
    ],
    CURLOPT_POSTFIELDS => json_encode($payload),
    CURLOPT_TIMEOUT => 15,
    // ...
]);
```

Le `$payload` contient l'expéditeur (`sender`, une adresse validée chez Brevo), le destinataire (`to`), le sujet, le contenu HTML et une version texte. La fonction renvoie toujours un tableau `{success, messageId, error}` au lieu de lever une exception : un email non parti ne doit pas faire planter la page. Les gabarits échappent les données avec `htmlspecialchars()`. Elle est utilisée par `auth/forgot_password.php` et par les rappels d'échéances (`includes/maintenance.php`), qui ne tentent l'envoi que si `BREVO_API_KEY` est configurée.

7.5 Google OAuth 2.0 : « Se connecter avec Google »

OAuth 2.0 est un protocole de délégation : l'utilisateur prouve son identité à Google, et Google nous confirme qui il est, **sans que nous voyions jamais son mot de passe Google**. Le projet utilise le flux le plus sûr pour un site serveur, l'**Authorization Code Flow**, codé avec cURL sans bibliothèque externe (`includes/google_oauth.php`).

Les étapes

1. `auth/google_login.php` génère un jeton **state** aléatoire, le met en session et redirige vers la page de consentement de Google.
2. L'utilisateur choisit son compte et accepte de partager `openid email profile`.
3. Google redirige le navigateur vers l'**URI de redirection** `auth/google_callback.php?code=...&state=...`.
4. Le callback vérifie que `state` est identique à celui de la session (protection CSRF).
5. Le serveur échange le `code` contre un **access token** (requête serveur à serveur, avec le `client_secret`).
6. Avec ce jeton, il lit le profil (`sub`, `email`, prénom, nom), retrouve le compte par `googleId` ou par email, ou en crée un (rôle client), puis ouvre la session.

```
// auth/google_login.php
$state = bin2hex(random_bytes(16));
$_SESSION['google_oauth_state'] = $state;
$authUrl = buildGoogleAuthUrl($state);
header('Location: ' . $authUrl);

// includes/google_oauth.php - exchangeGoogleCode()
$postData = [
    'code' => $code,
    'client_id' => $clientId,
    'client_secret' => $clientSecret,
    'redirect_uri' => getGoogleRedirectUri(),
    'grant_type' => 'authorization_code',
];
```

```
// auth/google_callback.php
$expectedState = $_SESSION['google_oauth_state'] ?? null;
unset($_SESSION['google_oauth_state']); // jeton à usage unique
$receivedState = $_GET['state'] ?? '';
// ...
if (!hash_equals($expectedState, $receivedState)) {
    redirect('auth/login.php?google_error=' . urlencode('Session expirée ou requête invalide
    (erreur de jeton CSRF Google).'));
}
```

L'URI de redirection

C'est l'adresse où Google renvoie l'utilisateur. Elle doit être déclarée **à l'identique** dans la Google Cloud Console (« URI de redirection autorisés ») ; sinon Google affiche `redirect_uri_mismatch`. Le projet la déduit par défaut (`SITE_URL . 'auth/google_callback.php'`) ou la lit dans `GOOGLE_REDIRECT_URI`. Il gère aussi un piège réel : si l'on démarre sur `127.0.0.1` alors que l'URI déclarée utilise `localhost`, le cookie de session n'est pas renvoyé au retour, et le `state` est introuvable. `google_login.php` redirige donc d'abord vers la bonne origine (`googleLoginUrlOnRedirectOrigin()`) et vérifie la casse du dossier (`googleRedirectPathMatchesBase()`).

Point à connaître : lorsqu'un compte existe déjà avec la même adresse, le callback lui associe le `googleId` sans contrôler le champ `email_verified` du profil Google. Les comptes Gmail ont toujours une adresse vérifiée, mais pour être rigoureux il faudrait refuser la liaison si `email_verified` n'est pas vrai ; c'est une amélioration à proposer si le jury pose la question.

Piège fréquent

Le `code` renvoyé par Google n'est pas une preuve d'identité à lui seul : il ne vaut quelque chose qu'une fois échangé par le serveur avec le `client_secret`. C'est pourquoi l'échange se fait côté serveur et pourquoi le secret ne doit jamais apparaître dans une page HTML ou un fichier JavaScript.

7.6 Où sont stockées les clés ?

Toutes les clés (CamPay, OpenRouter, Brevo, Google) et les identifiants de la base sont lus par une seule fonction, `appConfig()` :

```
// config/config.php
function appConfig(string $key, $default = null) {
    $env = getenv($key);
    if ($env !== false && $env !== '') {
        return $env; // 1. variable d'environnement
    }
    static $local = null;
    if ($local === null) {
        $file = __DIR__ . '/local.php';
        $loaded = is_file($file) ? require $file : [];
        $local = is_array($loaded) ? $loaded : [];
    }
    return $local[$key] ?? $default; // 2. config/local.php, sinon 3. défaut
}
```

- `config/local.php` contient les vraies valeurs. Il est listé dans `.gitignore` : il n'est **jamais envoyé dans Git** (l'historique du dépôt ne contient aucune version de ce fichier).
- `config/local.example.php`, lui, est versionné : c'est un modèle avec toutes les clés vides et commentées. Un nouveau développeur le copie en `local.php` et le remplit.
- `config/.htaccess` contient `Require all denied` : même si quelqu'un tape l'URL du dossier `config/`, Apache refuse de servir ses fichiers.
- Les **variables d'environnement** ont priorité : c'est la méthode recommandée chez un hébergeur.
- Le mot-clé `static` fait que le fichier n'est lu qu'une fois par requête.

À retenir

- CamPay : collect → validation sur le téléphone → suivi du statut + webhook signé (JWT HS256) → PAYE idempotent ; démo plafonnée à 25 XAF, simulation hors ligne.
- OpenRouter : une API pour beaucoup de modèles ; liste principal + secours ; 401 arrête tout, 429/404/5xx font passer au suivant ; appel toujours côté serveur.
- Brevo : emails transactionnels par API HTTP, échec géré sans planter la page.
- Google OAuth 2.0 : flux « code d'autorisation », state anti-CSRF, échange code → jeton côté serveur, URI de redirection identique à la console Google.
- Les secrets vivent dans `config/local.php` (ignoré par Git, protégé par `.htaccess`) ou dans l'environnement.

Piège fréquent

Dans `includes/mailler.php` et `includes/google_oauth.php`, les appels `cURL` désactivent la vérification du certificat (`CURLOPT_SSL_VERIFYPEER => false`, « compatibilité environnements locaux »), car XAMPP sous Windows n'a souvent pas de fichier de certificats racine. C'est acceptable en développement, mais pas en production : un attaquant placé sur le réseau pourrait se faire passer pour Google ou Brevo. Si le jury le relève, reconnaissez-le et expliquez la correction : configurer `curl.cainfo` dans `php.ini` et retirer cette option (le client commun `http_client.php`, utilisé pour CamPay et OpenRouter, garde la vérification par défaut).

Questions du jury

Q. Qu'est-ce qu'une API REST et comment votre application l'utilise-t-elle ?

Une API REST est une interface web où chaque ressource a une URL et où l'on agit avec les verbes HTTP (GET pour lire, POST pour créer), les données étant échangées en JSON. Mon serveur PHP appelle ainsi CamPay (POST `/api/collect/`, GET `/api/transaction/{ref}/`), OpenRouter (POST `/chat/completions`) et Brevo (POST `/v3/smtplib/email`) avec `cURL`. Le code de statut HTTP de la réponse me dit si l'appel a réussi.

Q. Comment êtes-vous sûr qu'une notification de paiement vient vraiment de CamPay ?

Le webhook doit contenir une signature JWT signée en HMAC-SHA256 avec une clé que seuls CamPay et moi connaissons. Je recalcule la signature, je la compare avec `hash_equals()`, j'impose l'algorithme HS256 et je refuse un jeton expiré ; sinon je réponds 403. En plus, je ne fais pas confiance au statut reçu : je redemande à l'API CamPay le statut de la transaction, et je vérifie que le montant confirmé correspond au montant attendu (le montant dû, plafonné en démo).

Q. Que se passe-t-il si le webhook et la page du client confirment le même paiement en même temps ?

La mise à jour est idempotente : la requête SQL ne modifie la ligne que si elle n'est pas déjà PAYE (AND statut <> 'PAYE'). Une seule des deux requêtes obtient `rowCount() === 1`, et seule celle-là envoie les notifications et écrit le journal. Le client n'est donc jamais notifié deux fois, et un paiement payé ne peut plus être modifié.

Q. Comment testez-vous les paiements sans dépenser d'argent ?

Par défaut, l'application utilise l'environnement de démonstration de CamPay, où l'argent n'est pas réel ; comme cette démo refuse plus de 25 XAF, le montant débité est plafonné, mais le montant dû reste le vrai coût en base. Il existe aussi un mode simulation sans aucun appel réseau, où chaque paiement réussit, pour développer hors ligne. Le passage en production se fait par configuration (`CAMPAY_USE_DEMO` à `false`), sans changer le code.

Q. Pourquoi l'assistant IA n'appelle-t-il pas OpenRouter directement depuis le navigateur ?

Parce que la clé d'API serait visible par n'importe quel visiteur dans le code JavaScript ou l'onglet Réseau, et pourrait être utilisée à mes frais. Côté serveur, je contrôle aussi ce que le modèle reçoit : un client ne

voit que ses propres véhicules, l'admin seulement des statistiques. Enfin, je peux y appliquer l'authentification, le jeton CSRF, la limite de débit et le quota quotidien.

Q. Que se passe-t-il si le modèle d'IA gratuit est indisponible ?

L'application essaie une liste ordonnée de modèles : le principal, puis jusqu'à trois modèles de secours configurés. Si un modèle répond 429 (saturé), 404 (retiré), 402, 5xx ou renvoie une réponse vide, on passe au suivant ; si la clé est refusée (401), on s'arrête, car tous les modèles échoueraient. L'utilisateur voit un message simple, et l'administrateur voit le détail de chaque essai.

Q. Expliquez la connexion avec Google. Voyez-vous le mot de passe Google de l'utilisateur ?

Non, jamais : l'utilisateur s'authentifie chez Google, qui me renvoie seulement un code d'autorisation sur mon URI de redirection. Mon serveur échange ce code contre un jeton d'accès en envoyant mon `client_secret`, puis lit le profil (identifiant `sub`, email, nom). Le paramètre `state`, aléatoire et gardé en session, empêche une attaque CSRF où un pirate injecterait son propre code. Ensuite je retrouve ou crée le compte et j'ouvre une session normale.

Q. Où sont vos clés d'API et comment évitez-vous qu'elles fuient ?

Elles sont dans `config/local.php`, lu par la fonction `appConfig()`, ou dans des variables d'environnement qui ont priorité. Ce fichier est dans le `.gitignore`, donc jamais publié sur GitHub ; seul un modèle vide, `local.example.php`, est versionné. Le dossier `config/` est en plus interdit d'accès web par un `.htaccess`, et les clés ne sont jamais envoyées au navigateur : tous les appels aux API partent du serveur.

8. Sécurité transversale et questions finales

La sécurité de SmartAutoTrack ne tient pas dans un seul fichier. Chaque protection est portée par une technologie précise : PDO pour les requêtes SQL, `htmlspecialchars` et `textContent` contre l'injection de HTML, la session PHP pour l'identité, `password_hash` pour les mots de passe, HMAC pour le webhook CamPay, Apache (`.htaccess`) pour les fichiers. Ce chapitre fait la synthèse : pour chaque attaque, **ce que c'est, la technologie qui protège, l'extrait réel du projet et ce qu'il faut savoir répondre**. Le principe de départ : **on ne fait jamais confiance au navigateur**. Tout ce qui arrive en `$_GET`, `$_POST`, en cookie ou en en-tête peut avoir été fabriqué à la main par un attaquant.

Menace	Protection	Où dans le code
Injection SQL	Requêtes préparées PDO, émulation désactivée	<code>config/database.php</code> , toutes les pages
XSS (injection de script)	<code>h()</code> côté PHP, <code>textContent</code> côté JS	<code>config/config.php</code> , <code>assets/js/main.js</code>
CSRF (requête forcée)	Jeton aléatoire en session, comparé avec <code>hash_equals</code>	<code>config/config.php</code> , <code>includes/header.php</code>
Vol / fixation de session	Cookie HttpOnly + SameSite, <code>session_regenerate_id(true)</code>	<code>config/config.php</code> , <code>auth/login.php</code>
Vol de mots de passe	<code>password_hash</code> (bcrypt) + règles de robustesse	<code>config/config.php</code> , <code>includes/password_policy.php</code>
Accès à l'espace d'un autre rôle	<code>requireRole()</code> , rôle déduit côté serveur	<code>config/config.php</code> , <code>config/roles.php</code>
IDOR (objet d'autrui)	Contrôle de propriété dans la requête SQL	<code>client/interventions.php</code> , <code>ajax/download_document.php</code>
Fichiers malveillants / exposés	Type réel (<code>finfo</code>), nom aléatoire, dossier interdit, script de téléchargement	<code>auth/register.php</code> , <code>uploads/.htaccess</code>
Faux avis de paiement	Webhook signé (JWT HS256) + re-vérification chez CamPay	<code>includes/campay.php</code> , <code>webhooks/campay.php</code>
Fuite des clés	Secrets dans <code>config/local.php</code> non versionné	<code>.gitignore</code> , <code>appConfig()</code>
Force brute sur le login	Compteurs par email et par IP, HTTP 429	<code>includes/login_throttle.php</code>

8.1 Injection SQL : les requêtes préparées PDO

Ce que c'est. Une injection SQL consiste à glisser du code SQL dans une saisie. Si on écrit `"... WHERE email = ' ' . $email . ' ' "` et que l'attaquant tape `' OR '1'='1`, la requête change de sens et peut renvoyer tous les comptes.

Comment c'est fait ici. Toutes les requêtes qui contiennent une donnée externe utilisent `prepare()` avec des marqueurs `?`, puis `execute([...])`. La requête et les valeurs sont envoyées **séparément** à MySQL : une valeur ne peut jamais devenir du code. La connexion désactive en plus l'émulation des requêtes préparées, ce qui oblige MySQL à faire lui-même la préparation :

```
// config/database.php
$this->conn = new PDO(
    "mysql:host=" . $this->host . ";dbname=" . $this->db_name . ";charset=utf8mb4",
    $this->username,
    $this->password,
    array(
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false
    )
);
```

```
// ajax/download_document.php
$stmt = $conn->prepare("SELECT id, technicien_id, nom_fichier, chemin_fichier FROM
technician_documents WHERE id = ?");
$stmt->execute([$doc_id]);
$doc = $stmt->fetch();
```

Autre détail : en cas d'échec de connexion, le message technique part dans `error_log()` et jamais à l'écran. Un message d'erreur SQL affiché aide énormément un attaquant.

Piège fréquent

Un marqueur `?` ne remplace qu'une **valeur**, pas un nom de colonne ni un mot-clé comme `ASC / DESC`. Quand un tri ou un filtre dépend de l'utilisateur, la valeur doit être prise dans une liste blanche (comme `in_array($reqType, $allowedTypes, true)` dans `client/interventions.php`), jamais concaténée telle quelle.

8.2 XSS : échapper à l'affichage

Ce que c'est. Le Cross-Site Scripting, c'est faire exécuter du JavaScript dans le navigateur d'une autre personne. Par exemple, un client qui écrit `<script>...</script>` dans la description d'une panne : si l'admin affiche ce texte brut, le script tourne avec la session de l'admin.

Côté PHP, toute valeur affichée passe par `h()`, qui transforme `<`, `>`, `&` et les guillemets en entités HTML :

```
// config/config.php
function h($value) {
    return htmlspecialchars((string)($value ?? ''), ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8', false);
}
```

`ENT_QUOTES` échappe aussi les apostrophes (indispensable dans un attribut `value='...'`). Le dernier paramètre `false` (pas de double encodage) existe parce que `sanitize()` encode déjà les saisies **avant** leur enregistrement : sans ce paramètre, une apostrophe déjà stockée sous la forme `'` serait ré-encodée et l'écran afficherait littéralement `'` au lieu de `'`. (`sanitize()` applique aussi `strip_tags` et `trim`.)

Côté JavaScript, les fenêtres de rapport construisent le contenu avec `textContent`, qui insère du texte et jamais du HTML :

```
// assets/js/main.js
function el(tag, className, text) {
  const node = document.createElement(tag);
  if (className) node.className = className;
  if (text !== undefined) node.textContent = text;
  return node;
}
```

L'assistant IA fait de même (`e1.textContent = text` dans `assets/js/assistant.js`) : une réponse du modèle ne peut pas injecter de balises.

Piège fréquent

Dire « on nettoie à l'entrée, donc c'est sûr ». La vraie règle est d'**échapper à la sortie**, selon le contexte (HTML, attribut, JS). Ici les deux sont faits, mais c'est `h()` à l'affichage qui protège réellement, y compris pour des données qui n'auraient pas été saisies via un formulaire (import SQL, API, IA).

8.3 CSRF : le jeton de formulaire

Ce que c'est. Le Cross-Site Request Forgery : un site piégé fait envoyer à votre navigateur, à votre insu, un formulaire vers SmartAutoTrack. Comme le navigateur joint automatiquement le cookie de session, la requête arrive « authentifiée ». Le jeton CSRF est un secret que le site piégé ne peut pas lire, donc pas recopier.

```
// config/config.php
function generateCSRFToken() {
  if (!isset($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
  }
  return $_SESSION['csrf_token'];
}

function verifyCSRFToken($token) {
  return isset($_SESSION['csrf_token']) && is_string($token) &&
  hash_equals($_SESSION['csrf_token'], $token);
}
```

Le jeton est placé dans un champ caché de chaque formulaire (`<input type="hidden" name="csrf_token" ...>` dans `auth/login.php`) et dans `<meta name="csrf-token">` de `includes/header.php`. Pour l'AJAX, `main.js` l'ajoute en en-tête `X-CSRF-Token` à chaque requête jQuery **vers le même site seulement** (`if (!settings.crossDomain)`), et `verifyRequestCSRF()` accepte l'un ou l'autre. `random_bytes(32)` donne 256 bits imprévisibles ; `hash_equals` compare en temps constant pour ne pas laisser deviner le jeton caractère par caractère via le temps de réponse.

Le même principe protège la connexion Google : le paramètre `state` envoyé à Google est comparé au retour avec `hash_equals($expectedState, $receivedState)` dans `auth/google_callback.php`.

8.4 Sessions : cookie protégé et identifiant régénéré

```
// config/config.php
session_name('HCHSESSID');
session_set_cookie_params([
    'lifetime' => 0,
    'path' => BASE_PATH,
    'domain' => '',
    'secure' => (isset($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off'),
    'httponly' => true,
    'samesite' => 'Lax',
]);
session_start();
```

- **httponly** : JavaScript ne peut pas lire le cookie, donc même un XSS ne peut pas voler la session via `document.cookie`.
- **samesite Lax** : le cookie n'est pas envoyé sur les POST venant d'un autre site, une deuxième barrière anti-CSRF.
- **secure** : activé automatiquement en HTTPS, le cookie ne circule alors jamais en clair.
- **lifetime 0** : cookie de session, supprimé à la fermeture du navigateur ; **path** limité au dossier de l'application.

À la connexion réussie, `auth/login.php` appelle `session_regenerate_id(true)` **avant** d'écrire l'identité en session : c'est la protection contre la **fixation de session** (un attaquant qui aurait imposé un identifiant connu à la victime le voit devenir inutile). Enfin `enforceActiveSession()`, exécutée à chaque requête, relit le profil en base : un compte suspendu ou supprimé perd l'accès immédiatement, pas seulement à sa prochaine connexion.

8.5 Mots de passe : hachage et règles

```
// config/config.php
function hashPassword($password) {
    return password_hash($password, PASSWORD_DEFAULT);
}
function verifyPassword($password, $hash) {
    return password_verify($password, $hash);
}
```

`password_hash` produit un hachage **bcrypt** avec un **sel aléatoire** intégré et un coût volontairement lent. Le mot de passe n'est jamais stocké ni déchiffrable ; à la connexion on recalcule et compare avec `password_verify`. Le sel fait que deux utilisateurs avec le même mot de passe ont deux hachages différents (les tables arc-en-ciel précalculées deviennent inutiles).

Les règles de robustesse sont centralisées et appliquées **côté serveur** ; le contrôle JavaScript n'est qu'un confort :

```
// includes/password_policy.php
function passwordPolicyMissing(string $password): array {
    $missing = [];
    if (mb_strlen($password, 'UTF-8') < PASSWORD_MIN_LENGTH) $missing[] = 'au moins ' .
PASSWORD_MIN_LENGTH . ' caractères';
    if (!preg_match('/[a-z]/', $password)) $missing[] = 'une lettre minuscule';
    if (!preg_match('/[A-Z]/', $password)) $missing[] = 'une lettre majuscule';
    if (!preg_match('/[0-9]/', $password)) $missing[] = 'un chiffre';
    if (!preg_match('/[^A-Za-z0-9]/', $password)) $missing[] = 'un caractère spécial (ex. ! @ # $
%)';
    return $missing;
}
```

Le message d'échec de connexion est volontairement générique (« Email ou mot de passe incorrect ») pour ne pas révéler quels emails existent.

8.6 Contrôle d'accès par rôle et par propriété (IDOR)

Par rôle. Chaque page commence par une garde. Le rôle vient de la session, et la session l'a reçu du serveur : `getUserRole()` regarde dans quelle table (administrateur, client, technicien, garage) l'utilisateur figure réellement. Le formulaire ne transmet jamais de rôle.

```
// config/config.php
function requireRole($role) {
    requireAuth();
    if ($_SESSION['role'] !== $role) {
        redirect('dashboard.php');
    }
}
```

Pour les endpoints AJAX, `requireJsonAuth()` fait la même chose mais répond en JSON avec les codes HTTP **401** (non connecté) ou **403** (mauvais rôle).

Par propriété. Être client ne suffit pas : il faut être **le** client propriétaire. L'**IDOR** (Insecure Direct Object Reference) consiste à changer un identifiant dans l'URL ou le formulaire (`?id=12` devient `?id=13`) pour atteindre l'objet de quelqu'un d'autre. La parade est d'ajouter le propriétaire dans la requête SQL elle-même :

```
// client/interventions.php
// Contrôle de propriété : le véhicule doit appartenir au client connecté.
if (empty($requestErrors)) {
    $stmt = $conn->prepare("SELECT idVehicule FROM vehicule WHERE idVehicule = ? AND idClient =
?");
    $stmt->execute([$reqVehiculeId, $_SESSION['user_id']]);
    if (!$stmt->fetch()) {
        $requestErrors[] = 'Ce véhicule ne vous appartient pas.';
    }
}
```

Toutes les listes de l'espace client filtrent de la même façon par `WHERE idClient = ?` avec l'identifiant de session.

8.7 Téléchargements contrôlés

Les documents des techniciens (diplômes, pièces) sont sensibles. Quatre couches :

1. **À l'envoi** (auth/register.php) : le type est déterminé par le **contenu** avec `finfo` (pas par l'extension), seuls PDF/JPEG/PNG sont admis, 5 Mo maximum, et le fichier reçoit un nom aléatoire : `$user_id . '_' . bin2hex(random_bytes(16)) . '.' . $file_ext`.
2. **Sur le disque** : `uploads/.htaccess` contient `Require all denied`. Apache refuse toute URL directe vers ces fichiers, même si on devine le nom. `config/.htaccess` fait de même pour la configuration.
3. **Au téléchargement** : `ajax/download_document.php` vérifie la session puis les droits (admin ou technicien propriétaire), et répond **404** sinon, pour ne même pas révéler que le document existe.
4. **Anti-traversée de chemin** : `realpath()` doit tomber dans `uploads/techniciens/` ; le type MIME est revérifié avec `finfo` (réponse 415 sinon), le nom renvoyé est nettoyé, puis des en-têtes durcissent la réponse.

```
// ajax/download_document.php
$baseDir = realpath(UPLOAD_PATH . 'techniciens');
$file = realpath(__DIR__ . '/../' . $doc['chemin_fichier']);
if ($baseDir === false || $file === false || strpos($file, $baseDir . DIRECTORY_SEPARATOR) !== 0
|| !is_file($file)) {
    http_response_code(404);
    exit('Fichier introuvable');
}
// ...
header('X-Content-Type-Options: nosniff');
header("Content-Security-Policy: default-src 'none'; img-src 'self'; style-src 'unsafe-inline';
sandbox");
```

`nosniff` empêche le navigateur de « deviner » qu'un fichier est du HTML ; la CSP (`default-src 'none' + sandbox`) fait traiter la réponse comme un document isolé, sans droit d'exécuter de script ni d'accéder aux cookies du site, même si un fichier piégé s'ouvrait dans le navigateur.

8.8 Webhook CamPay signé

Le webhook est une URL que CamPay appelle pour dire « ce paiement a réussi ». Elle est publique, sans session ni jeton CSRF : n'importe qui pourrait l'appeler. Deux défenses :

```
// includes/campay.php
[$header64, $payload64, $signature64] = explode('.', $jwt);
$header = json_decode(campayBase64UrlDecode($header64), true);
if (!is_array($header) || ($header['alg'] ?? '') !== 'HS256') {
    return false;
}
$expected = hash_hmac('sha256', $header64 . '.' . $payload64, $key, true);
if (!hash_equals($expected, campayBase64UrlDecode($signature64))) {
    return false;
}
```

- La **signature** est un JWT HS256 : un HMAC-SHA256 calculé avec la clé secrète `CAMPAY_WEBHOOK_KEY` que seuls CamPay et le serveur connaissent. L'algorithme est imposé (`HS256`), la date d'expiration `exp` est contrôlée, la comparaison est en temps constant.
- Même signée, la notification n'est **pas crue sur parole** : `webhooks/campay.php` redemande le statut à l'API CamPay (`campayTransactionStatus()`) et applique celui-là. Une ancienne notification rejouée ne peut donc pas valider un autre paiement.

8.9 Secrets hors du dépôt

Les clés (base de données, CamPay, OpenRouter, Brevo, Google) ne sont jamais écrites dans le code versionné. `appConfig()` lit d'abord une variable d'environnement, sinon `config/local.php`, fichier listé dans `.gitignore`. Le dépôt ne contient que le modèle vide `config/local.example.php`.

```
// config/config.php
function appConfig(string $key, $default = null) {
    $env = getenv($key);
    if ($env !== false && $env !== '') {
        return $env;
    }
    static $local = null;
    if ($local === null) {
        $file = __DIR__ . '/local.php';
        $loaded = is_file($file) ? require $file : [];
        $local = is_array($loaded) ? $loaded : [];
    }
    return $local[$key] ?? $default;
}
```

Les appels à OpenRouter et CamPay partent du **serveur** PHP : la clé API n'est jamais envoyée au navigateur.

8.10 Limitation des tentatives de connexion

Contre la **force brute** (essayer des milliers de mots de passe), `includes/login_throttle.php` compte les échecs sur une fenêtre glissante de 15 minutes : 5 par email, 20 par adresse IP. Les compteurs sont dans des fichiers et non en session, car un attaquant n'a qu'à ne pas renvoyer son cookie pour repartir de zéro.

```
// includes/login_throttle.php
const LOGIN_THROTTLE_WINDOW = 900;           // secondes (15 min)
const LOGIN_THROTTLE_MAX_PER_EMAIL = 5;
const LOGIN_THROTTLE_MAX_PER_IP = 20;

function loginThrottleKeys(string $email, string $ip): array {
    return [
        'email:' . strtolower(trim($email)) => LOGIN_THROTTLE_MAX_PER_EMAIL,
        'ip:' . $ip => LOGIN_THROTTLE_MAX_PER_IP,
    ];
}
```

L'email est normalisé (minuscules) pour qu'une variante de casse ne contourne pas la limite, et haché en SHA-256 pour nommer le fichier (il n'apparaît pas en clair sur le disque). Dans `auth/login.php`, si la limite est atteinte, la réponse est **HTTP 429 Too Many Requests** avec l'en-tête `Retry-After`. Une connexion réussie efface le compteur de l'email, mais pas celui de l'IP. Le comportement est vérifié par `tests/Unit/LoginThrottleTest.php`.

À retenir

- Règle d'or : **le serveur décide**. Rôle, propriété, type de fichier, statut de paiement sont toujours recalculés côté serveur.
- Données vers SQL : requête préparée. Données vers HTML : `h()` ou `textContent`.
- Toute action qui modifie quelque chose exige un jeton CSRF ; le webhook, lui, exige une signature HMAC.
- Session : `HttpOnly` + `SameSite` + régénération à la connexion. Mots de passe : `bcrypt` via `password_hash`.
- Comparaisons de secrets : toujours `hash_equals` (temps constant).
- Défense en profondeur : plusieurs couches indépendantes (ex. fichiers : `finfo` + nom aléatoire + `.htaccess` + script contrôlé).

8.11 Les 12 questions techniques que le jury posera presque sûrement

Q. Comment empêchez-vous les injections SQL ?

Par des requêtes préparées PDO : la requête contient des marqueurs `?` et les valeurs sont passées séparément à `execute()`, donc MySQL ne peut jamais les interpréter comme du code. J'ai désactivé `PDO::ATTR_EMULATE_PREPARES` pour que la préparation soit faite par MySQL lui-même. Quand une partie non paramétrable dépend de l'utilisateur (un type, un tri), je la vérifie contre une liste blanche.

Q. Qu'est-ce qu'une faille XSS et comment l'évitez-vous ?

C'est l'injection de JavaScript dans une page vue par un autre utilisateur, par exemple via la description d'une panne. J'échappe toute valeur affichée avec ma fonction `h()`, basée sur `htmlspecialchars` avec `ENT_QUOTES` et UTF-8. Côté JavaScript, je construis le contenu dynamique avec `textContent` et non `innerHTML`. Et le cookie de session étant `HttpOnly`, un script ne pourrait pas le lire.

Q. À quoi sert le jeton CSRF, et pourquoi le webhook CamPay n'en a pas ?

Le jeton prouve que le formulaire a bien été affiché par mon site : un site tiers ne peut pas le lire, donc ne peut pas forger une requête valide avec le cookie de la victime. Il est généré avec `random_bytes(32)` et vérifié avec `hash_equals`. Le webhook est appelé par le serveur de CamPay, sans session ni navigateur : il est donc authentifié autrement, par une signature JWT HMAC-SHA256 avec une clé secrète partagée, et le statut est ensuite revérifié auprès de l'API CamPay.

Q. Comment sont stockés les mots de passe ? Pourriez-vous me donner le mien ?

Non, c'est impossible : je ne stocke qu'un hachage `bcrypt` produit par `password_hash`, avec un sel aléatoire et un coût de calcul volontairement lent. À la connexion, `password_verify` recalcule et compare. En cas d'oubli, l'utilisateur passe par une réinitialisation (`auth/forgot_password.php`), jamais par la récupération de l'ancien mot de passe. Les règles (8 caractères, minuscule, majuscule, chiffre, caractère spécial) sont validées côté serveur.

Q. Pourquoi appeler `session_regenerate_id(true)` à la connexion ?

Pour éviter la fixation de session : si un attaquant a réussi à imposer un identifiant de session à la victime avant sa connexion, cet identifiant devient inutile dès qu'elle se connecte. Le paramètre `true` supprime l'ancienne session côté serveur. Je le fais avant d'écrire l'identité en session.

Q. Un client peut-il voir les véhicules ou les rapports d'un autre client en changeant l'identifiant dans l'URL ?

Non, c'est la faille IDOR, et je la bloque dans la requête SQL elle-même : par exemple `WHERE idVehicule = ? AND idClient = ?` avec l'identifiant pris dans la session, jamais dans le formulaire. Si rien ne correspond, l'action est refusée. Pour les documents, je renvoie même une 404 pour ne pas révéler leur existence.

Q. Comment est déterminé le rôle d'un utilisateur ? Peut-on se faire passer pour admin ?

Le rôle n'est jamais envoyé par le navigateur : `config/roles.php` le déduit de la table où l'utilisateur apparaît réellement (administrateur, client, technicien, garage), puis il est stocké en session. Chaque page appelle `requireRole()` et les endpoints AJAX `requireJsonAuth()` qui répond 401 ou 403. En plus, `enforceActiveSession()` revérifie le compte à chaque requête, donc un compte suspendu est déconnecté immédiatement.

Q. Comment sécurisez-vous les fichiers envoyés par les techniciens ?

Le type est vérifié sur le contenu réel avec `finfo` (PDF, JPEG, PNG uniquement), la taille est limitée à 5 Mo et le fichier est renommé avec un nom aléatoire. Le dossier `uploads/` est interdit d'accès direct par un `.htaccess`. On ne peut les lire que via `ajax/download_document.php`, qui contrôle la session, les droits et le chemin avec `realpath()`, et ajoute `nosniff` et une CSP `sandbox`.

Q. Comment vous protégez-vous contre la force brute sur la page de connexion ?

`includes/login_throttle.php` compte les échecs sur 15 minutes : 5 par email et 20 par IP. Au-delà, la page répond HTTP 429 avec `Retry-After`. Les compteurs sont stockés dans des fichiers et non en session, parce qu'un attaquant peut simplement ne pas renvoyer son cookie. Le message d'erreur reste générique pour ne pas révéler quels emails sont inscrits.

Q. Où sont vos clés API, et que se passe-t-il si votre dépôt Git devient public ?

Elles sont dans `config/local.php`, exclu par `.gitignore`, ou dans des variables d'environnement lues en priorité par `appConfig()`. Le dépôt ne contient que `local.example.php` avec des valeurs vides. Les appels à CamPay et OpenRouter partent du serveur, donc les clés ne sont jamais envoyées au navigateur. Si une clé fuitait malgré tout, il faudrait la révoquer et en générer une nouvelle chez le fournisseur.

Q. Comment êtes-vous sûr qu'un paiement Mobile Money est réellement payé ?

Je ne crois jamais le navigateur ni la seule notification. Le webhook doit porter une signature JWT HS256 valide et non expirée, calculée avec la clé webhook. Ensuite, le serveur interroge lui-même l'API CamPay

sur cette référence et applique le statut renvoyé ; un paiement passé à PAYE est définitif. Une notification rejouée ou fabriquée ne change donc rien.

Q. Votre application est-elle totalement sécurisée ? Que feriez-vous en plus ?

Aucune application ne l'est totalement ; j'ai couvert les risques principaux de l'OWASP Top 10 avec plusieurs couches. En production, j'imposerais HTTPS (le cookie passerait automatiquement en `secure`), j'ajouterais une Content-Security-Policy sur toutes les pages, une double authentification pour les administrateurs, et un audit des dépendances Composer. Les protections sensibles sont déjà couvertes par des tests PHPUnit (`CsrfTest` , `EscapeTest` , `LoginThrottleTest` , `PasswordPolicyTest` , `CampayTest`).